NEXT ENGINEER · _SHEERAT_

COMPLETE TEXTBOOK

Data Structure

BCS301

Full syllabus, unit by unit: theory in question-answer form, diagrams, algorithms, C programs, solved numericals and 2-mark questions

UNIT 1 Arrays, Complexity and Linked Lists

UNIT 2 Stacks, Recursion and Queues

UNIT 3 Searching, Hashing and Sorting

UNIT 4 Trees

UNIT 5 Graphs

Program: B.Tech **Branches:** CSE, IT, CSE-AIML, CSE-DS, AIDS **Semester:** 3 **Credits:** 4

L-T-P: 3-1-0 **Scheme:** AKTU B.Tech 2nd year, 2023-24 onwards

First edition, 2026

Contents

How to Use This Book

Unit 1 Arrays, Complexity and Linked Lists

Part 1: Basic Terminology, Data Organisation and ADT
Part 2: Algorithms, Complexity and Asymptotic Notations
Part 3: Arrays and Address Calculation
Part 4: Sparse Matrices
Part 5: Linked Lists
Part 6: Polynomial Representation and Arithmetic

Unit 2 Stacks, Recursion and Queues

Part 1: The Stack ADT and its Implementation
Part 2: Expressions: Conversion and Evaluation
Part 3: Recursion
Part 4: Queues

Unit 3 Searching, Hashing and Sorting

Part 1: Searching
Part 2: Hashing
Part 3: Simple Sorting Algorithms
Part 4: Divide and Conquer Sorting: Quick Sort and Merge Sort
Part 5: Heap Sort and Radix Sort

Unit 4 Trees

Part 1: Tree Terminology and Binary Trees
Part 2: Tree Traversals
Part 3: Binary Search Trees
Part 4: Threaded Binary Trees
Part 5: Huffman Coding
Part 6: AVL Trees
Part 7: B-Trees
Part 8: Binary Heaps and Conversion of General Trees

Unit 5 Graphs

Part 1: Graph Terminology and Representation
Part 2: Graph Traversal
Part 3: Spanning Trees and Minimum Cost Spanning Trees
Part 4: Transitive Closure and Shortest Paths

Model Question Paper with Answer Guide

Last-Day Revision Checklist

NEXT ENGINEER _sheerat_

How to Use This Book

This book covers the complete BCS301 syllabus, unit by unit, in the question-and-answer style that university exams use. Every topic is explained from the beginning, so you can study from this book alone. It is written in original words for Next Engineer.

How each unit is organised

- The unit opens with a short **overview** that tells you what the unit is about and which topics carry the most marks.
- The unit is divided into **Parts**. Each Part groups a few related syllabus topics.
- Inside each Part, the theory is written as **long-answer questions** (marked *Que 1.1*, *Que 1.2*, and so on). Each answer is written the way you should write it in the exam: definition, explanation, diagram, algorithm, example, complexity.
- At the end of each unit there are **2-mark short-answer questions** with model answers, and a list of **practice questions** for self-testing.

The exam pattern (theory, 70 marks, 3 hours)

Section	What is asked	Marks
A	7 short questions, all compulsory, 2 marks each	14
B	Attempt any 3 out of 5 questions, 7 marks each	21
C	One question per unit, each with two parts; attempt one part of each, 7 marks each	35

Because Section C has one question from every unit, you cannot skip any unit. Units 3 and 4 (sorting and trees) usually give the most numerical questions, so practise their dry runs by hand.

How to write a 7-mark answer

1. Start with a one or two line definition.
2. Draw a neat diagram (a tree, a linked list, a stack, a table of passes).
3. Write the algorithm in numbered steps or in C.
4. Show a worked example or a dry run.
5. End with the time and space complexity, and one or two advantages or applications.

Prescribed reference books (for further reading)

- Aaron M. Tenenbaum, Yedidyah Langsam and Moshe J. Augenstein, *Data Structures Using C and C++*, PHI Learning.
- Jean Paul Tremblay and Paul G. Sorenson, *An Introduction to Data Structures with Applications*, McGraw Hill.
- Ellis Horowitz and Sartaj Sahni, *Fundamentals of Data Structures*, Galgotia Publications.
- Reema Thareja, *Data Structures Using C*, Oxford University Press.
- Seymour Lipschutz, *Data Structures*, Schaum's Outline Series, McGraw Hill.
- Thomas H. Cormen et al., *Introduction to Algorithms*, PHI Learning.

UNIT 1

Arrays, Complexity and Linked Lists

Unit overview

This unit lays the foundation for the whole subject. It starts with what a data structure is and how we measure the efficiency of an algorithm. It then covers arrays in depth, including how a compiler finds the address of any element, and closes with linked lists and their use in polynomial arithmetic.

Topics in this unit: basic terminology, elementary data organisation, built-in data types in C, abstract data types, algorithms and their efficiency, time and space complexity, asymptotic notations, time-space trade-off, arrays (1-D, 2-D, 3-D and n-D), row-major and column-major order, index formulae, sparse matrices, singly, doubly and circular linked lists, insertion, deletion and traversal, polynomial representation and arithmetic.

Most asked: address calculation numericals, asymptotic notations with graphs, sparse matrix representation, linked list insertion/deletion code, and polynomial addition using linked lists.

Part 1: Basic Terminology, Data Organisation and ADT

Que 1.1 **What is a data structure? Explain the basic terminology used in data structures.**

ANSWER

A **data structure** is a particular way of storing and organising data in a computer's memory so that it can be used efficiently. The choice of data structure decides how quickly we can insert, delete, search or update the data.

A data structure is defined by three things:

1. **The logical relationship** between the data elements (for example, a sequence, a hierarchy or a network).
2. **The way the data is stored** in memory (contiguously or with links).
3. **The operations** that are allowed on it and how efficient they are.

Basic terminology

- **Data:** raw values or facts, such as 25, "Ravi" or 7.5. On their own they carry no meaning.
- **Data item:** a single unit of value. A data item that cannot be divided further is an **elementary item** (for example, a roll number). One that can be divided into sub-items is a **group item** (for example, a date, which has day, month and year).
- **Entity:** something that has attributes, each of which takes a value. A student is an entity with attributes roll number, name and branch.
- **Entity set:** a collection of entities of the same type, such as all students of a class.
- **Information:** processed data that has a meaning for the user.

- **Field:** a single elementary unit of information that represents one attribute of an entity.
- **Record:** a collection of field values of a single entity. One student's roll number, name and branch together form a record.
- **File:** a collection of records of the entities of one entity set.
- **Key (primary key):** a field whose value uniquely identifies a record, such as a roll number.

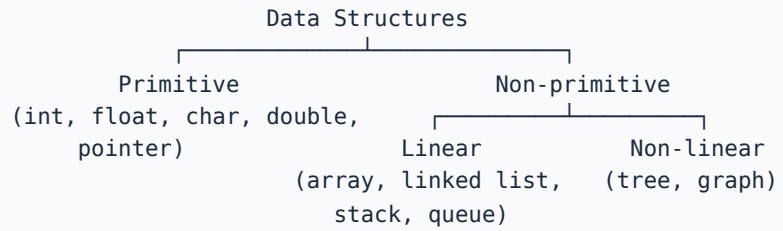
Roll No (key)	Name	Branch	CGPA
101	Ananya	CSE	8.4
102	Rahul	IT	7.9

In the table above each row is a record, each column is a field, and the whole table is a file.

Que 1.2 **Explain the classification of data structures with examples.**

ANSWER

Data structures are classified as follows:



1. Primitive data structures: These are the basic types that the machine operates on directly. In C these are `int`, `char`, `float`, `double` and pointers. Each stores a single value.

2. Non-primitive data structures: These are built from primitive types. They are further divided into:

- **Linear data structures:** the elements form a sequence, and each element (except the first and the last) has exactly one predecessor and one successor. They can be traversed in one pass. Examples: arrays, linked lists, stacks and queues.
- **Non-linear data structures:** an element can be connected to more than one other element, so the data does not form a sequence. Examples: trees (hierarchical relationship) and graphs (network relationship).

Another classification

Basis	Type 1	Type 2
Memory allocation	**Static:** size fixed at compile time (array)	**Dynamic:** size changes at run time (linked list)
Element types	**Homogeneous:** all elements of one type (array)	**Non-homogeneous:** elements of different types (structure)
Arrangement	**Linear** (stack, queue)	**Non-linear** (tree, graph)

Operations on data structures

1. **Traversing:** visiting each element exactly once.
2. **Searching:** finding the location of an element with a given value.
3. **Inserting:** adding a new element.
4. **Deleting:** removing an element.
5. **Sorting:** arranging the elements in some order.
6. **Merging:** combining two sorted structures into one sorted structure.

Que 1.3 **What is elementary data organisation? Describe the built-in data types of C.**

ANSWER

Elementary data organisation refers to the way basic data is grouped: data items are grouped into fields, fields into records, and records into files (see Que 1.1). Data structures then decide how these records are arranged in memory and accessed.

Built-in data types in C

Type	Typical size (64-bit GCC)	Range / precision	Format specifier
`char`	1 byte	−128 to 127	`%c`
`unsigned char`	1 byte	0 to 255	`%c`
`short int`	2 bytes	−32,768 to 32,767	`%hd`
`int`	4 bytes	−2,147,483,648 to 2,147,483,647	`%d`
`long int`	8 bytes	about $\pm 9.2 \times 10^{18}$	`%ld`
`float`	4 bytes	about 6 significant digits	`%f`
`double`	8 bytes	about 15 significant digits	`%lf`
`void`	none	no value	—

Sizes depend on the compiler and machine; older 16-bit compilers (such as Turbo C) used 2 bytes for `int`. The `sizeof` operator gives the actual size on a given system.

C also gives **derived types** (arrays, pointers, functions) and **user-defined types** (`struct`, `union`, `enum`, `typedef`). Data structures such as linked lists and trees are built using structures and pointers.

Que 1.4 **What is an abstract data type (ADT)? Explain with the example of a list ADT.**

ANSWER

An **abstract data type** is a mathematical model of a data type that is defined only by:

1. the **set of values** it can hold, and
2. the **set of operations** that can be performed on those values, along with what each operation does.

It says *what* the operations do, not *how* they are implemented. The implementation is hidden from the user. This separation is called **abstraction** and **encapsulation**.

For example, the **Stack ADT** has the operations push, pop, peek, isEmpty and isFull. A user can use a stack without knowing whether it is built with an array or with a linked list.

List ADT

- **Values:** a finite ordered sequence of elements a_1, a_2, ..., a_n.
- **Operations:**
- `createList()` : makes an empty list.
- `insert(L, x, pos)` : inserts x at position pos.
- `delete(L, pos)` : removes the element at position pos.
- `find(L, x)` : returns the position of x, or −1 if absent.
- `get(L, pos)` : returns the element at position pos.
- `length(L)` : returns the number of elements.
- `isEmpty(L)` : returns true if the list has no elements.

The same List ADT can be implemented with an array (fast access by position, slow insertion in the middle) or with a linked list (fast insertion, slow access by position).

Advantages of ADT

- The implementation can be changed without changing the programs that use it.
- Programs become easier to understand, test and maintain.
- It encourages reuse of code.

Part 2: Algorithms, Complexity and Asymptotic Notations

Que 1.5 **Define an algorithm. What are its characteristics? How is the efficiency of an algorithm measured?**

ANSWER

An **algorithm** is a finite sequence of well-defined instructions that, when followed, solves a given problem in a finite amount of time.

Characteristics of an algorithm

1. **Input:** it takes zero or more inputs.
2. **Output:** it produces at least one output.
3. **Definiteness:** each step is clear and unambiguous.
4. **Finiteness:** it stops after a finite number of steps.
5. **Effectiveness:** each step is basic enough to be carried out exactly, in principle even with pen and paper.

Example: algorithm to find the largest of n numbers.

```
Algorithm LARGEST(A, n)
1. Set max = A[0]
2. Repeat for i = 1 to n - 1
3.     If A[i] > max then set max = A[i]
4. Return max
```

Efficiency of an algorithm

The efficiency of an algorithm is the amount of computing resources it uses. Two resources are measured:

- **Time complexity:** the amount of time the algorithm takes to run, expressed as a function of the input size n.
- **Space complexity:** the amount of memory the algorithm needs, as a function of n.

We do not measure time in seconds because that depends on the machine, compiler and load. Instead we count the number of **basic operations** (comparisons, assignments, arithmetic) as a function of n. This is called **a priori analysis**. Running the program and measuring time is called **a posteriori analysis** (profiling).

Best, worst and average case

- **Worst case:** the maximum time over all inputs of size n. This gives a guarantee and is the most commonly used.
- **Best case:** the minimum time over all inputs of size n.
- **Average case:** the expected time over all inputs of size n, assuming some probability distribution.

For linear search in an array of n elements: best case 1 comparison (the key is first), worst case n comparisons (the key is last or absent), average case about n/2 comparisons.

Que 1.6 **What is time complexity and space complexity? Explain with examples how to compute them.**

ANSWER

Time complexity is the number of elementary steps an algorithm performs as a function of the input size n.

Space complexity S(P) of a program P is the total memory it needs:

S(P) = C + S_P(I)

where C is the **fixed part** (space for code, simple variables and constants, independent of the input) and S_P(I) is the **variable part** (space that depends on the instance I, such as arrays whose size depends on n and the recursion stack).

Rules for finding time complexity from code

Code pattern	Complexity
Simple statement	O(1)
`for (i = 0; i < n; i++)` with O(1) body	O(n)

Code pattern	Complexity
Two nested loops each running n times	$O(n^2)$
`for (i = 1; i < n; i = i * 2)`	O(log n)
Outer loop n times, inner loop doubling	O(n log n)
Consecutive blocks	add them, keep the largest
if-else	take the larger branch

Example 1: sum of an array.

```
int sum(int a[], int n) {
    int s = 0;                  /* 1 step        */
    for (int i = 0; i < n; i++) /* n + 1 checks  */
        s = s + a[i];           /* n steps       */
    return s;                   /* 1 step        */
}
```

Total steps = 1 + (n + 1) + n + 1 = 2n + 3, so the time complexity is **O(n)**. Space: the array is passed by address, and the variables s, i, n take constant space, so the extra space is **O(1)**.

Example 2:

```
for (i = 1; i <= n; i++)
    for (j = 1; j <= i; j++)
        count++;
```

The inner loop runs 1 + 2 + ... + n = n(n + 1)/2 times, so the complexity is **$O(n^2)$**.

Example 3:

```
for (i = n; i > 1; i = i / 2)
    count++;
```

i takes the values n, n/2, n/4, ..., 2. After k iterations $i = n/2^k$, and the loop stops when $n/2^k \leq 1$, that is when $k = \log_2 n$. So the complexity is **O(log n)**.

Example 4: recursive factorial. Each call does constant work and makes one recursive call, so T(n) = T(n − 1) + c, which gives **O(n)** time. The recursion goes n levels deep, so the stack needs **O(n)** space.

Que 1.7 **Explain asymptotic notations Big O, Big Omega and Big Theta with graphs and examples.**

ANSWER

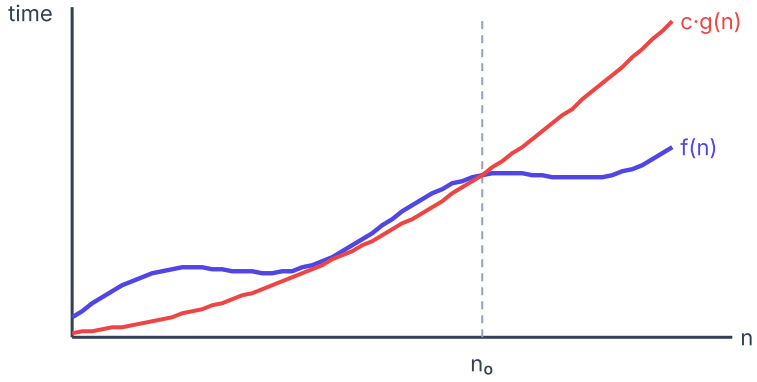
Asymptotic notations describe how the running time of an algorithm grows as the input size n becomes very large. They ignore constant factors and lower-order terms, which do not matter for large n.

1. Big O notation (upper bound)

$f(n) = O(g(n))$ if there exist positive constants c and n_0 such that

$0 \le f(n) \le c \cdot g(n)$ for all $n \ge n_0$.

It means f grows **no faster** than g. It is used for the **worst case**.



Example: show that $f(n) = 3n^2 + 5n + 2 = O(n^2)$. For $n \ge 1$: $3n^2 + 5n + 2 \le 3n^2 + 5n^2 + 2n^2 = 10n^2$. So $c = 10$, $n_0 = 1$, and $f(n) = O(n^2)$.

2. Big Omega notation (lower bound)

$f(n) = \Omega(g(n))$ if there exist positive constants c and n_0 such that

$0 \le c \cdot g(n) \le f(n)$ for all $n \ge n_0$.

It means f grows **at least as fast** as g. It is used for the **best case**.

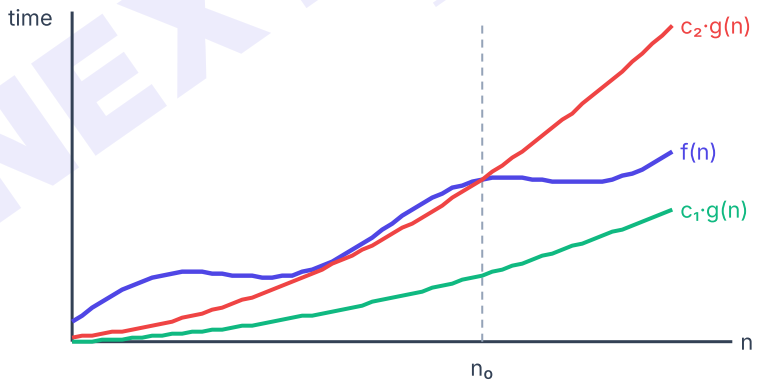
Example: $3n^2 + 5n + 2 \ge 3n^2$ for all $n \ge 1$, so with $c = 3$, $n_0 = 1$, $f(n) = \Omega(n^2)$.

3. Big Theta notation (tight bound)

$f(n) = \Theta(g(n))$ if there exist positive constants c_1, c_2 and n_0 such that

$0 \le c_1 \cdot g(n) \le f(n) \le c_2 \cdot g(n)$ for all $n \ge n_0$.

It means f grows **at the same rate** as g. $f(n) = \Theta(g(n))$ if and only if $f(n) = O(g(n))$ and $f(n) = \Omega(g(n))$.



Example: from the two examples above, $3n^2 + 5n + 2 = \Theta(n^2)$ with $c_1 = 3$, $c_2 = 10$, $n_0 = 1$.

Little o and little omega: $f(n) = o(g(n))$ means f grows strictly slower than g (the limit of f/g is 0). $f(n) = \omega(g(n))$ means f grows strictly faster than g (the limit of f/g is infinity).

Common growth rates (slowest to fastest)

$1 < \log n < \sqrt{n} < n < n \log n < n^2 < n^3 < 2^n < n! < n^n$

NEXT ENGINEER _sheerat_

n	$\log_2 n$	$n \log_2 n$	n^2	2^n
8	3	24	64	256
16	4	64	256	65,536
32	5	160	1,024	about 4.3×10^9

The table shows why an $O(2^n)$ algorithm becomes useless even for moderate n.

Que 1.8 **What is time-space trade-off? Explain with examples.**

ANSWER

A **time-space trade-off** is a situation in which we can reduce the running time of an algorithm by using more memory, or reduce the memory used at the cost of more running time. Usually we cannot minimise both at once, so the choice depends on which resource is more valuable for the application.

Examples

1. **Lookup tables:** instead of computing a value (such as sin x or the factorial of n) every time, we compute all values once and store them in a table. Each later query takes O(1) time, but the table takes extra memory.
2. **Hashing:** a hash table with many empty slots gives near O(1) search, but wastes memory. A sorted array uses exactly n slots but needs O(log n) binary search.
3. **Compressed vs uncompressed data:** compressed files save storage but take time to decompress before use.
4. **Memoization in recursion:** computing Fibonacci numbers recursively takes $O(2^n)$ time and little extra memory. Storing already computed values in an array reduces the time to O(n) but uses O(n) memory.
5. **Sparse matrix storage:** storing only the non-zero elements saves space, but accessing an element at (i, j) now needs a search instead of direct indexing.

Part 3: Arrays and Address Calculation

Que 1.9 **What is an array? Explain one-dimensional and multi-dimensional arrays and their operations.**

ANSWER

An **array** is a finite collection of elements of the same data type, stored in **contiguous** memory locations, and accessed by an index (subscript).

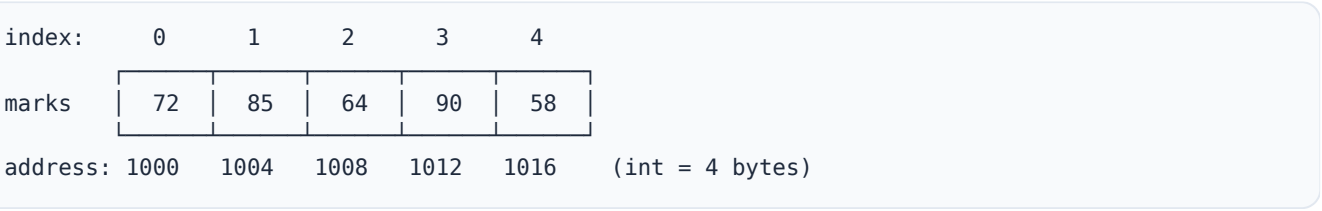
Properties

- All elements are of the same type (homogeneous).
- The size is fixed when the array is declared (static allocation).
- Any element can be accessed directly in O(1) time using its index, because its address can be calculated.

- In C, the index starts at 0, so an array of size n has indices 0 to n − 1.

One-dimensional array

```
int marks[5] = {72, 85, 64, 90, 58};
```



The number of elements (length) of an array with lower bound LB and upper bound UB is **UB − LB + 1**.

Multi-dimensional arrays

A **two-dimensional array** stores data in rows and columns, like a matrix. `int a[3][4];` has 3 rows and 4 columns, that is 12 elements. A **three-dimensional array** `int a[2][3][4];` can be pictured as 2 pages, each a 3 × 4 matrix, giving 24 elements. In general an n-dimensional array has n subscripts.

Operations on arrays and their complexity

Operation	Method	Time
Access A[i]	address formula	O(1)
Traversal	visit each element	O(n)
Insertion at position k	shift elements k..n−1 right by one	O(n)
Deletion at position k	shift elements k+1..n−1 left by one	O(n)
Linear search	compare one by one	O(n)
Binary search (sorted)	halve the range	O(log n)

Insertion algorithm

```
INSERT(A, n, k, item)        // insert item at index k
1. Set j = n − 1
2. Repeat while j ≥ k:
3.     A[j + 1] = A[j];  j = j − 1
4. A[k] = item
5. n = n + 1
```

Deletion algorithm

```
DELETE(A, n, k)              // delete element at index k
1. item = A[k]
2. Repeat for j = k to n − 2:
3.     A[j] = A[j + 1]
4. n = n − 1
5. Return item
```

Applications of arrays

- Storing lists of data such as marks, temperatures or prices.
- Representing matrices and performing matrix operations.
- Implementing other data structures: stacks, queues, heaps, hash tables.
- Lookup tables and sorting/searching algorithms.
- Representing polynomials (coefficient at index = power).

Limitations: fixed size, costly insertion and deletion in the middle, and wasted memory if the array is declared larger than needed.

Que 1.10 **Explain row-major and column-major representation of a 2-D array. Derive the address formula for each.**

ANSWER

Memory is one-dimensional, so a 2-D array must be **linearised** (laid out as one long row) before it can be stored. There are two ways.

Consider `A[3][3]`:

```
        col 0  col 1  col 2
row 0 |  a00    a01    a02 |
row 1 |  a10    a11    a12 |
row 2 |  a20    a21    a22 |
```

Row-major order: the elements are stored row by row. First all elements of row 0, then all of row 1, and so on.

`a00 a01 a02 | a10 a11 a12 | a20 a21 a22`

C, C++ and Java (for arrays of arrays) use row-major order.

Column-major order: the elements are stored column by column.

`a00 a10 a20 | a01 a11 a21 | a02 a12 a22`

FORTRAN and MATLAB use column-major order.

Derivation of the row-major formula

Let the array be $A[L_1 \dots U_1][L_2 \dots U_2]$, with base address B and W bytes per element.

- Number of rows $M = U_1 - L_1 + 1$, number of columns $N = U_2 - L_2 + 1$.
- To reach element A[i][j], we must skip $(i - L_1)$ complete rows. Each row has N elements, so that is $(i - L_1) \times N$ elements.
- In row i we must skip $(j - L_2)$ more elements.
- Total elements before A[i][j] = $(i - L_1)N + (j - L_2)$.

So, **$Loc(A[i][j]) = B + W [N(i - L_1) + (j - L_2)]$**

Derivation of the column-major formula

- To reach A[i][j] we skip $(j - L_2)$ complete columns, each of M elements, and then $(i - L_1)$ elements in column j.

So, **Loc(A[i][j]) = B + W [M(j − L_2) + (i − L_1)]**

When indices start at 0 (as in C), $L_1 = L_2 = 0$ and the formulae become B + W(N·i + j) and B + W(M·j + i).

Que 1.11 **Derive the address formula for 1-D, 3-D and n-dimensional arrays.**

ANSWER

1-D array A[L ... U] with base address B and element size W. To reach A[i] we skip (i − L) elements:

Loc(A[i]) = B + W (i − L)

3-D array $A[L_1...U_1][L_2...U_2][L_3...U_3]$. Let $M = U_1 - L_1 + 1$, $N = U_2 - L_2 + 1$ and $P = U_3 - L_3 + 1$. Write $E_1 = i - L_1$, $E_2 = j - L_2$, $E_3 = k - L_3$ (these are called effective indices).

Row-major (the last index changes fastest): to reach A[i][j][k] we skip E_1 complete "pages" of N × P elements, then E_2 rows of P elements, then E_3 elements:

Loc = B + W [E_1·N·P + E_2·P + E_3] which can also be written as B + W [(E_1·N + E_2)·P + E_3]

Column-major (the first index changes fastest):

Loc = B + W [E_3·M·N + E_2·M + E_1] or B + W [(E_3·N + E_2)·M + E_1]

n-dimensional array $A[L_1..U_1][L_2..U_2]...[L_n..U_n]$. Let the length of dimension r be $Len_r = U_r - L_r + 1$ and $E_r = i_r - L_r$.

Row-major: Loc = B + W [(... ((E_1·Len_2 + E_2)·Len_3 + E_3)·Len_4 + ...)·Len_n + E_n]

Column-major: Loc = B + W [(... ((E_n·Len_{n-1} + E_{n-1})·Len_{n-2} + E_{n-2}) ...)·Len_1 + E_1]

This nested form (Horner's form) is what compilers actually use, because it needs only one multiplication and one addition per dimension.

Que 1.12 **Solved numericals on address calculation.**

ANSWER

Example 1 (1-D). An array A[−5 ... 20] of integers has base address 2000 and each element takes 4 bytes. Find the address of A[10] and the number of elements.

- Number of elements = 20 − (−5) + 1 = **26**.
- Loc(A[10]) = 2000 + 4 × (10 − (−5)) = 2000 + 4 × 15 = **2060**.

Example 2 (2-D, both orders). A 2-D array X[−4 ... 6][−2 ... 12] stores elements of 2 bytes each, base address 1200. Find the address of X[2][5] in row-major and column-major order.

- M (rows) = 6 − (−4) + 1 = 11, N (columns) = 12 − (−2) + 1 = 15.
- $E_1 = 2 - (-4) = 6$, $E_2 = 5 - (-2) = 7$.
- **Row-major:** 1200 + 2 × [15 × 6 + 7] = 1200 + 2 × 97 = **1394**.
- **Column-major:** 1200 + 2 × [11 × 7 + 6] = 1200 + 2 × 83 = **1366**.

Example 3 (finding the base address). In an array A[1...25][1...30] stored in row-major order with 2 bytes per element, the address of A[5][7] is 1800. Find the address of A[12][15].

- N = 30. Loc(A[5][7]) = B + 2 × [30 × 4 + 6] = B + 252 = 1800, so B = **1548**.

- Loc(A[12][15]) = 1548 + 2 × [30 × 11 + 14] = 1548 + 2 × 344 = **2236**.

Example 4 (3-D). An array A[2...8][−4...1][6...10] has base address 200 and W = 4. Find the address of A[5][−1][8] in row-major and column-major order.

- M = 8 − 2 + 1 = 7, N = 1 − (−4) + 1 = 6, P = 10 − 6 + 1 = 5.
- E_1 = 5 − 2 = 3, E_2 = −1 − (−4) = 3, E_3 = 8 − 6 = 2.
- **Row-major:** 200 + 4 × [3 × 6 × 5 + 3 × 5 + 2] = 200 + 4 × [90 + 15 + 2] = 200 + 4 × 107 = **628**.
- **Column-major:** 200 + 4 × [2 × 7 × 6 + 3 × 7 + 3] = 200 + 4 × [84 + 21 + 3] = 200 + 4 × 108 = **632**.

Exam tip: always write M, N, P and the effective indices E_1, E_2, E_3 on separate lines before substituting. Most mistakes come from using the wrong dimension length when the lower bound is negative.

Part 4: Sparse Matrices

Que 1.13 **What is a sparse matrix? Explain its representations with examples.**

ANSWER

A **sparse matrix** is a matrix in which most of the elements are zero. A common rule of thumb: if more than about two-thirds of the elements are zero, the matrix is sparse. Storing all the zeros wastes memory and time, so we store only the non-zero elements.

Example: consider the 4 × 5 matrix

```
0   0   3   0   4
0   0   5   7   0
0   0   0   0   0
0   2   6   0   0
```

It has 20 elements, of which only 6 are non-zero.

1. Triplet (3-tuple / array) representation

Each non-zero element is stored as a row of three values: (row, column, value). The first row (row 0 of the table) is a header that stores the number of rows, number of columns and number of non-zero elements.

Row	Column	Value
4	5	6
0	2	3
0	4	4
1	2	5
1	3	7

Row	Column	Value
3	1	2
3	2	6

Memory used: 7 × 3 = 21 integers instead of 20, which seems no better here, but for a 1000 × 1000 matrix with 2,000 non-zeros the triplet form needs about 6,000 integers instead of 1,000,000.

2. Linked list representation

Each non-zero element is a node with four fields: row, column, value and a pointer to the next node.

```
head → [0|2|3|•] → [0|4|4|•] → [1|2|5|•] → [1|3|7|•] → [3|1|2|•] → [3|2|6|NULL]
```

This handles insertion of new non-zero elements easily.

3. Orthogonal (multi-linked) list representation

Each node has five fields: row, column, value, a pointer to the next non-zero element in the same row (right) and a pointer to the next non-zero element in the same column (down). There is a header node for every row and every column. This allows fast traversal of both rows and columns, which helps in matrix multiplication.

Special sparse matrices

- **Lower triangular matrix:** all elements above the main diagonal are zero (A[i][j] = 0 for j > i). Only n(n + 1)/2 elements need to be stored. In row-major order, A[i][j] (indices from 1) is stored at position i(i − 1)/2 + j in a 1-D array.
- **Upper triangular matrix:** all elements below the main diagonal are zero (A[i][j] = 0 for i > j).
- **Tridiagonal matrix:** non-zero elements only on the main diagonal and the diagonals just above and below it ($|i - j| \le 1$). It has 3n − 2 non-zero elements.

Program: convert a matrix to triplet form

```c
#include <stdio.h>
#define R 4
#define C 5

int main(void) {
    int a[R][C] = {{0,0,3,0,4},{0,0,5,7,0},{0,0,0,0,0},{0,2,6,0,0}};
    int t[R * C + 1][3], k = 1;

    for (int i = 0; i < R; i++)
        for (int j = 0; j < C; j++)
            if (a[i][j] != 0) {
                t[k][0] = i; t[k][1] = j; t[k][2] = a[i][j];
                k++;
            }
    t[0][0] = R; t[0][1] = C; t[0][2] = k - 1;   /* header row */

    for (int i = 0; i < k; i++)
        printf("%d %d %d\n", t[i][0], t[i][1], t[i][2]);
    return 0;
}
```

Transpose of a sparse matrix in triplet form: swap the row and column of every triplet, then sort the triplets by the new row. The simple method scans the triplet list once for each column, which takes O(columns × non-zeros). The **fast transpose** method first counts the non-zeros in each column, computes the starting position of each column in the result, and then places every element directly, which takes O(columns + non-zeros).

Part 5: Linked Lists

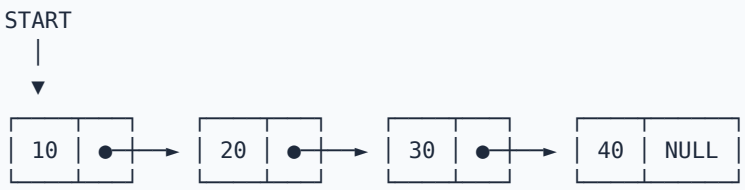
Que 1.14 **What is a linked list? Compare arrays and linked lists. Explain the array implementation of a linked list.**

ANSWER

A **linked list** is a linear collection of data elements called **nodes**, where the order is given by pointers rather than by physical position in memory. Each node has two parts:

1. **Data (info) field:** the value stored.
2. **Link (next) field:** the address of the next node.

A pointer called **START** (or head) holds the address of the first node. The link of the last node is **NULL**.



Array vs linked list

Basis	Array	Linked list
Memory	Contiguous	Scattered; nodes linked by pointers
Size	Fixed at compile time	Grows and shrinks at run time
Access	Direct (random), O(1)	Sequential, O(n)
Insertion/deletion	O(n), needs shifting	O(1) once the position is known
Extra memory	None	One pointer per node
Memory waste	Unused slots if over-declared	None (allocated as needed)
Cache performance	Good (locality)	Poorer
Binary search	Possible	Not practical

Array implementation (cursor implementation) of a linked list

Before dynamic memory was common, linked lists were built inside arrays. We keep two parallel arrays, INFO[] and LINK[]. LINK[k] stores the **index** of the next node instead of an address, and −1 (or

0 with 1-based indices) stands for NULL. A variable START holds the index of the first node, and another list called **AVAIL** links together the free slots.

Index	INFO	LINK
1	30	5
2	(free)	6
3	10	4
4	20	1
5	40	−1
6	(free)	−1

Here START = 3, so the list is 10 → 20 → 30 → 40, and AVAIL = 2 (free slots 2 → 6). To insert, we take a slot from AVAIL; to delete, we return the slot to AVAIL. This is how linked lists are implemented in languages without pointers.

Pointer implementation uses a self-referential structure and dynamic memory allocation with `malloc()` and `free()`:

```
struct node {
    int data;
    struct node *next;
};
```

Que 1.15 **Write algorithms and C functions for insertion in a singly linked list at the beginning, at the end and after a given node.**

ANSWER

We use `struct node` as defined above and a global pointer `start`.

Creating a node

```
#include <stdio.h>
#include <stdlib.h>

struct node { int data; struct node *next; };
struct node *start = NULL;

struct node *getnode(int x) {
    struct node *p = (struct node *)malloc(sizeof(struct node));
    if (p == NULL) { printf("Overflow\n"); exit(1); }
    p->data = x;
    p->next = NULL;
    return p;
}
```

1. Insertion at the beginning

Steps:

1. Create a new node p and store x in it.
2. Set p→next = start.
3. Set start = p.

```
Before:  start → [10] → [20] → NULL
After:   start → [5] → [10] → [20] → NULL
```

```
void insert_begin(int x) {
    struct node *p = getnode(x);
    p->next = start;
    start = p;
}
```

Time complexity: **O(1)**.

2. Insertion at the end

Steps:

1. Create a new node p with data x and next = NULL.
2. If the list is empty, set start = p and stop.
3. Otherwise move a pointer q from start until q→next is NULL.
4. Set q→next = p.

```
void insert_end(int x) {
    struct node *p = getnode(x), *q = start;
    if (start == NULL) { start = p; return; }
    while (q->next != NULL)
        q = q->next;
    q->next = p;
}
```

Time complexity: **O(n)**, because we traverse to the last node. Keeping a separate `tail` pointer makes this O(1).

3. Insertion after a node containing a given key

```
void insert_after(int key, int x) {
    struct node *q = start;
    while (q != NULL && q->data != key)
        q = q->next;
    if (q == NULL) { printf("Key not found\n"); return; }
    struct node *p = getnode(x);
    p->next = q->next;      /* step 1: link new node to the successor */
    q->next = p;            /* step 2: link predecessor to new node   */
}
```

The order of the two link statements matters. If we write `q->next = p` first, the address of the rest of the list is lost.

4. Insertion at position pos (1-based)

```c
void insert_pos(int pos, int x) {
    if (pos == 1) { insert_begin(x); return; }
    struct node *q = start;
    for (int i = 1; i < pos - 1 && q != NULL; i++)
        q = q->next;
    if (q == NULL) { printf("Invalid position\n"); return; }
    struct node *p = getnode(x);
    p->next = q->next;
    q->next = p;
}
```

Que 1.16 **Write algorithms and C functions for deletion from a singly linked list (first node, last node, node with a given value). Also write traversal, counting and reversal functions.**

ANSWER

1. Delete the first node

```c
int delete_begin(void) {
    if (start == NULL) { printf("Underflow\n"); return -1; }
    struct node *p = start;
    int x = p->data;
    start = start->next;
    free(p);
    return x;
}
```

Time: **O(1)**.

2. Delete the last node

We need a pointer to the second-last node, so we keep a trailing pointer `prev`.

```c
int delete_end(void) {
    if (start == NULL) { printf("Underflow\n"); return -1; }
    struct node *p = start, *prev = NULL;
    while (p->next != NULL) { prev = p; p = p->next; }
    int x = p->data;
    if (prev == NULL) start = NULL;      /* only one node */
    else prev->next = NULL;
    free(p);
    return x;
}
```

Time: **O(n)**.

3. Delete the node with a given key

```
void delete_key(int key) {
    struct node *p = start, *prev = NULL;
    while (p != NULL && p->data != key) { prev = p; p = p->next; }
    if (p == NULL) { printf("Not found\n"); return; }
    if (prev == NULL) start = p->next;   /* deleting the first node */
    else prev->next = p->next;
    free(p);
}
```

```
Delete 20:
start → [10] → [20] → [30] → NULL
         prev    p
prev->next = p->next   gives   start → [10] → [30] → NULL, then free(p)
```

4. Traversal and counting

```
void display(void) {
    for (struct node *p = start; p != NULL; p = p->next)
        printf("%d -> ", p->data);
    printf("NULL\n");
}

int count(void) {
    int c = 0;
    for (struct node *p = start; p != NULL; p = p->next) c++;
    return c;
}
```

5. Search

```
struct node *search(int key) {
    struct node *p = start;
    while (p != NULL && p->data != key) p = p->next;
    return p;          /* NULL if not found */
}
```

6. Reverse a singly linked list (in place)

We walk through the list once and turn each `next` pointer backwards, using three pointers.

```
void reverse(void) {
    struct node *prev = NULL, *cur = start, *nxt;
    while (cur != NULL) {
        nxt = cur->next;   /* remember the rest of the list */
        cur->next = prev;  /* reverse the link              */
        prev = cur;        /* move prev one step            */
        cur = nxt;         /* move cur one step             */
    }
    start = prev;
}
```

Dry run on 10 → 20 → 30:

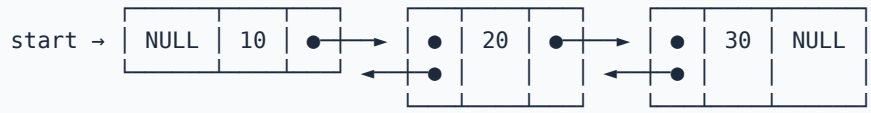
Step	prev	cur	List so far (reversed part)
start	NULL	10	
1	10	20	10 → NULL
2	20	30	20 → 10 → NULL
3	30	NULL	30 → 20 → 10 → NULL

Time **O(n)**, extra space **O(1)**.

Que 1.17 **What is a doubly linked list? Write C functions for insertion and deletion in a doubly linked list.**

ANSWER

In a **doubly linked list (DLL)** each node has three fields: a pointer to the previous node (**prev**), the data, and a pointer to the next node (**next**). The list can be traversed in both directions.




```
struct dnode {
    struct dnode *prev;
    int data;
    struct dnode *next;
};
struct dnode *head = NULL;
```

Insertion at the beginning

```
void dll_insert_begin(int x) {
    struct dnode *p = malloc(sizeof(struct dnode));
    p->data = x;
    p->prev = NULL;
    p->next = head;
    if (head != NULL) head->prev = p;
    head = p;
}
```

Insertion after node q (four links change)

```
void dll_insert_after(struct dnode *q, int x) {
    struct dnode *p = malloc(sizeof(struct dnode));
    p->data = x;
    p->prev = q;                /* 1 */
    p->next = q->next;          /* 2 */
    if (q->next != NULL)
        q->next->prev = p;      /* 3 */
    q->next = p;                /* 4 */
}
```

Deletion of a given node p

The big advantage of a DLL is that we can delete a node when we have only a pointer to it, because we can reach its predecessor through `prev`.

```
void dll_delete(struct dnode *p) {
    if (p->prev != NULL) p->prev->next = p->next;
    else head = p->next;                  /* p was the first node */
    if (p->next != NULL) p->next->prev = p->prev;
    free(p);
}
```

Time: **O(1)** once p is known.

Advantages of DLL over SLL

- Traversal in both directions.
- Deletion of a given node in O(1) without searching for its predecessor.
- Insertion before a given node is easy.

Disadvantages: an extra pointer per node, and every insertion or deletion must update more pointers, so there are more chances of errors.

Applications: browser back/forward navigation, undo/redo in editors, music playlists, LRU cache implementation.

Que 1.18 **What is a circular linked list? Explain its insertion and deletion operations.**

ANSWER

In a **circular linked list** the last node points back to the first node instead of NULL. There is no NULL in the list, so we can reach any node from any node.



It is convenient to keep a pointer to the **last** node, because last→next is the first node. Then both insertion at the beginning and at the end take O(1).

Insertion at the beginning

```
struct node *last = NULL;

void cll_insert_begin(int x) {
    struct node *p = getnode(x);
    if (last == NULL) { last = p; p->next = p; return; }  /* first node points to itself */
    p->next = last->next;
    last->next = p;
}
```

Insertion at the end: same as above, followed by `last = p;`.

Deletion of the first node

```
int cll_delete_begin(void) {
    if (last == NULL) { printf("Underflow\n"); return -1; }
    struct node *first = last->next;
    int x = first->data;
    if (first == last) last = NULL;      /* only one node */
    else last->next = first->next;
    free(first);
    return x;
}
```

Traversal must stop when we come back to the starting node:

```
void cll_display(void) {
    if (last == NULL) return;
    struct node *p = last->next;
    do {
        printf("%d ", p->data);
        p = p->next;
    } while (p != last->next);
}
```

Circular doubly linked list: the last node's next points to the first node and the first node's prev points to the last node.

Header linked list: a list that has a special **header node** at the beginning which does not hold list data (it may hold the count of nodes). In a **grounded header list** the last node points to NULL; in a **circular header list** the last node points back to the header. A header node removes the special case of an empty list in insertion and deletion.

Applications of circular lists: round-robin CPU scheduling, multiplayer games where turns rotate, circular buffers, and repeating playlists.

Part 6: Polynomial Representation and Arithmetic

Que 1.19 **How is a polynomial represented using a linked list? Write an algorithm and C function to add two polynomials.**

ANSWER

A polynomial in one variable such as

$P(x) = 5x^4 + 3x^2 - 7x + 2$

is a sum of terms, each with a **coefficient** and an **exponent**. Using a linked list, each term is a node with three fields: coefficient, exponent and link. The nodes are kept in **decreasing order of exponent**.

```
struct term {
    int coef;
    int exp;
    struct term *next;
};
```

```
P → [5|4|●] → [3|2|●] → [−7|1|●] → [2|0|NULL]
```

Why a linked list and not an array? An array indexed by exponent would need 1001 slots for $x^{1000} + 1$, almost all zeros. A linked list stores only the non-zero terms.

Algorithm for addition (P + Q = R)

1. Let p point to the first term of P and q to the first term of Q.
2. While both p and q are not NULL: - If p→exp == q→exp: add the coefficients. If the sum is not zero, append a term (sum, exp) to R. Advance both p and q. - If p→exp > q→exp: append a copy of p's term to R and advance p. - Otherwise: append a copy of q's term to R and advance q.
3. Append the remaining terms of whichever list is not finished.

NEXT ENGINEER _sheerat_

```
struct term *newterm(int c, int e) {
    struct term *t = malloc(sizeof(struct term));
    t->coef = c; t->exp = e; t->next = NULL;
    return t;
}

/* append a term at the end of list whose last node is *tail */
void attach(int c, int e, struct term **tail) {
    struct term *t = newterm(c, e);
    (*tail)->next = t;
    *tail = t;
}

struct term *poly_add(struct term *p, struct term *q) {
    struct term dummy = {0, 0, NULL}, *tail = &dummy;
    while (p != NULL && q != NULL) {
        if (p->exp == q->exp) {
            int s = p->coef + q->coef;
            if (s != 0) attach(s, p->exp, &tail);
            p = p->next; q = q->next;
        } else if (p->exp > q->exp) {
            attach(p->coef, p->exp, &tail); p = p->next;
        } else {
            attach(q->coef, q->exp, &tail); q = q->next;
        }
    }
    for (; p != NULL; p = p->next) attach(p->coef, p->exp, &tail);
    for (; q != NULL; q = q->next) attach(q->coef, q->exp, &tail);
    return dummy.next;
}
```

Example: P = $5x^4 + 3x^2 - 7x + 2$ and Q = $4x^3 - 3x^2 + 2x + 6$.

Step	p term	q term	Action	Term added to R
1	$5x^4$	$4x^3$	4 > 3, copy p	$5x^4$
2	$3x^2$	$4x^3$	2 < 3, copy q	$4x^3$
3	$3x^2$	$-3x^2$	equal, 3 + (−3) = 0	(none)
4	$-7x$	$2x$	equal, −7 + 2 = −5	$-5x$
5	2	6	equal, 2 + 6 = 8	8

R = $5x^4 + 4x^3 - 5x + 8$.

Complexity: if P has m terms and Q has n terms, each step consumes at least one term, so the time is **O(m + n)**.

Subtraction P − Q is done the same way, except that when exponents are equal we compute p→coef − q→coef, and when a term is copied from Q its coefficient is negated.

Que 1.20 **Explain polynomial multiplication using linked lists. How is a polynomial in two variables represented?**

ANSWER

Multiplication of single-variable polynomials

To multiply P and Q, multiply every term of P with every term of Q: the coefficients are multiplied and the exponents are added. Each product term is inserted into the result list in decreasing order of exponent; if a term with the same exponent already exists, the coefficients are added (and the node removed if the sum becomes zero).

```
/* insert (c, e) into an ordered list, combining like terms */
struct term *insert_term(struct term *r, int c, int e) {
    struct term dummy = {0, 0, r}, *prev = &dummy, *cur = r;
    while (cur != NULL && cur->exp > e) { prev = cur; cur = cur->next; }
    if (cur != NULL && cur->exp == e) {
        cur->coef += c;
        if (cur->coef == 0) { prev->next = cur->next; free(cur); }
    } else {
        struct term *t = newterm(c, e);
        t->next = cur;
        prev->next = t;
    }
    return dummy.next;
}

struct term *poly_mul(struct term *p, struct term *q) {
    struct term *r = NULL;
    for (struct term *a = p; a != NULL; a = a->next)
        for (struct term *b = q; b != NULL; b = b->next)
            r = insert_term(r, a->coef * b->coef, a->exp + b->exp);
    return r;
}
```

Example: $(2x^2 + 3)(x - 4)$

- $2x^2 \cdot x = 2x^3$
- $2x^2 \cdot (-4) = -8x^2$
- $3 \cdot x = 3x$
- $3 \cdot (-4) = -12$

Result: $2x^3 - 8x^2 + 3x - 12$.

Complexity: there are m·n products, and each insertion can take up to O(m·n) time in the result list, so the worst case is $O(m^2n^2)$; in practice it is close to O(m·n·k) where k is the length of the result.

Polynomials in two variables

A term of a polynomial in x and y, such as $6x^3y^2$, has a coefficient and two exponents. Each node therefore has four fields:

```
struct term2 {
    int coef;
    int xexp;
    int yexp;
    struct term2 *next;
};
```

$P(x, y) = 6x^3y^2 + 4x^2y^5 - 2xy + 9$ is stored as

```
[6|3|2|●] → [4|2|5|●] → [−2|1|1|●] → [9|0|0|NULL]
```

The terms are kept in decreasing order of x-exponent, and for equal x-exponents in decreasing order of y-exponent. Two terms are **like terms** only if both exponents match.

- **Addition:** the same merge as before, but the comparison first compares xexp and, if equal, compares yexp.
- **Multiplication:** multiply coefficients and add the x-exponents and the y-exponents separately: $(3x^2y)(4xy^3) = 12x^3y^4$.

NEXT ENGINEER
sheerat

Short-Answer Questions (2 marks)

Q1. Define data structure. Give two examples of linear and non-linear data structures.

A data structure is a way of organising and storing data in memory so that operations on it can be performed efficiently. Linear: array, queue. Non-linear: tree, graph.

Q2. What is an abstract data type?

An ADT is a data type defined only by its values and the operations on them, without specifying the implementation. Example: Queue ADT with enqueue, dequeue and isEmpty.

Q3. Differentiate between time and space complexity.

Time complexity is the amount of time (number of steps) an algorithm takes as a function of input size. Space complexity is the amount of memory it needs as a function of input size.

Q4. Define Big O notation.

$f(n) = O(g(n))$ if there exist constants $c > 0$ and n_0 such that $f(n) \le c \cdot g(n)$ for all $n \ge n_0$. It gives an upper bound on growth.

Q5. What is the complexity of the code `for(i=1;i<n;i*=2)` ?

i doubles each time, so the loop runs about $\log_2 n$ times. The complexity is O(log n).

Q6. What is a sparse matrix?

A matrix in which most elements are zero. Only the non-zero elements are stored, usually as (row, column, value) triplets.

Q7. Find the number of elements in A[−3:7, 2:10].

Rows = 7 − (−3) + 1 = 11, columns = 10 − 2 + 1 = 9, total = 99 elements.

Q8. What are the advantages of a linked list over an array?

Dynamic size, efficient insertion and deletion without shifting, and no wasted memory from over-declaration.

Q9. What is a header linked list?

A linked list with an extra node at the front, the header node, which does not store list data but may store information such as the number of nodes.

Q10. Why is a doubly linked list preferred for deletion?

Because each node has a pointer to its predecessor, a node can be deleted in O(1) when only a pointer to it is known, without traversing from the start.

Q11. What is the time-space trade-off?

A situation where running time can be reduced by using more memory, or memory saved by spending more time, as in lookup tables or memoization.

Q12. Write the row-major address formula for A[i][j].

$Loc(A[i][j]) = B + W[N(i - L_1) + (j - L_2)]$, where N is the number of columns.

NEXT ENGINEER _sheerat_

Practice Questions

1. Explain asymptotic notations with suitable graphs. Prove that $2n^3 + 4n + 7 = \Theta(n^3)$.
2. Consider A[5:15, −10:20, 3:9] with base address 900 and W = 2. Find the address of A[8][5][6] in both row-major and column-major order.
3. Derive the address formula of an n-dimensional array in column-major order.
4. Write a C program to represent a sparse matrix in triplet form and find its transpose.
5. Write a C function to insert a node at a given position in a doubly linked list.
6. Write a C program to add two polynomials using linked lists and trace it for $3x^4 + 2x^2 + 1$ and $5x^3 - 2x^2 + 4$.
7. Explain the array (cursor) implementation of a singly linked list with the AVAIL list.
8. Write a function to delete alternate nodes of a singly linked list.
9. How can you detect whether a singly linked list has a loop? (Hint: use two pointers moving at different speeds.)
10. Compare singly, doubly and circular linked lists in a table.

NEXT ENGINEER
sheerat

UNIT 2

Stacks, Recursion and Queues

Unit overview

This unit studies two restricted linear structures. A **stack** allows insertion and deletion at one end only, and a **queue** inserts at one end and deletes at the other. Because of these rules they model many real processes: function calls, undo, expression evaluation, printer jobs, CPU scheduling. The unit also covers **recursion**, which is closely tied to the stack because every recursive call uses the system stack.

Topics in this unit: Stack ADT and its operations, array and linked implementation, complexity, applications of stacks, infix, prefix and postfix expressions, conversion and evaluation, recursion principles, tail recursion, removal of recursion, binary search, Fibonacci and Tower of Hanoi, iteration versus recursion, queue operations (create, add, delete, full, empty), circular queues, array and linked queues in C, deque and priority queue.

Most asked: infix to postfix conversion with a stack trace table, postfix evaluation, Tower of Hanoi, circular queue code, and the difference between recursion and iteration.

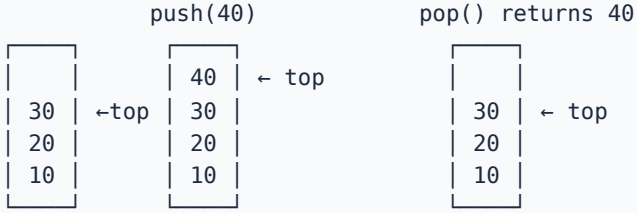
Part 1: The Stack ADT and its Implementation

Que 2.1 **What is a stack? Explain the Stack ADT and the primitive operations on a stack.**

ANSWER

A **stack** is a linear data structure in which insertion and deletion take place only at one end, called the **top** of the stack. The element inserted last is the first to be removed, so a stack follows the **LIFO (Last In, First Out)** principle.

A pile of plates is a stack: you put a plate on top and take a plate from the top.



Stack ADT

- **Values:** an ordered collection of elements, with one end marked as the top.
- **Operations:**
- `push(S, x)` : insert x on top of S. If the stack is full, this is an **overflow**.

- `pop(S)` : remove and return the top element. If the stack is empty, this is an **underflow**.
- `peek(S)` (or `top`): return the top element without removing it.
- `isEmpty(S)` : true if S has no elements.
- `isFull(S)` : true if no more elements can be added (array implementation only).
- `size(S)` : number of elements.

All these operations take **O(1)** time, because they touch only the top element.

Que 2.2 **Write algorithms and a C program to implement a stack using an array.**

ANSWER

We use an array `stack[MAX]` and an integer `top` that holds the index of the top element. An empty stack has `top = -1`, and a full stack has `top = MAX - 1`.

PUSH algorithm

```
PUSH(STACK, TOP, MAX, ITEM)
1. If TOP = MAX – 1, print "Overflow" and exit
2. Set TOP = TOP + 1
3. Set STACK[TOP] = ITEM
4. Exit
```

POP algorithm

```
POP(STACK, TOP, ITEM)
1. If TOP = –1, print "Underflow" and exit
2. Set ITEM = STACK[TOP]
3. Set TOP = TOP – 1
4. Return ITEM
```

C program

```c
#include <stdio.h>
#define MAX 100

int stack[MAX];
int top = -1;

int isEmpty(void) { return top == -1; }
int isFull(void)  { return top == MAX - 1; }

void push(int x) {
    if (isFull()) { printf("Stack overflow\n"); return; }
    stack[++top] = x;
}

int pop(void) {
    if (isEmpty()) { printf("Stack underflow\n"); return -1; }
    return stack[top--];
}

int peek(void) {
    if (isEmpty()) { printf("Stack is empty\n"); return -1; }
    return stack[top];
}

void display(void) {
    for (int i = top; i >= 0; i--) printf("%d\n", stack[i]);
}

int main(void) {
    push(10); push(20); push(30);
    printf("Popped: %d\n", pop());   /* 30 */
    printf("Top: %d\n", peek());     /* 20 */
    display();
    return 0;
}
```

Complexity: push, pop, peek, isEmpty and isFull are all O(1). The space is O(MAX), fixed in advance.

Drawback: the size is fixed. If MAX is too small we get overflow; if too large, memory is wasted. The linked implementation removes this limit.

Que 2.3 **Implement a stack using a linked list. Compare the array and linked list implementations.**

ANSWER

In the linked implementation each element is a node, and the **top** pointer points to the first node. Push inserts at the beginning of the list and pop deletes from the beginning, so both are O(1).

```
top → [30|●] → [20|●] → [10|NULL]
```

```c
#include <stdio.h>
#include <stdlib.h>

struct node { int data; struct node *next; };
struct node *top = NULL;

void push(int x) {
    struct node *p = malloc(sizeof(struct node));
    if (p == NULL) { printf("Overflow: no memory\n"); return; }
    p->data = x;
    p->next = top;
    top = p;
}

int pop(void) {
    if (top == NULL) { printf("Underflow\n"); return -1; }
    struct node *p = top;
    int x = p->data;
    top = top->next;
    free(p);
    return x;
}

int peek(void) { return top ? top->data : -1; }
```

Comparison

Basis	Array stack	Linked stack
Size	Fixed (MAX)	Grows until memory runs out
Overflow	When top = MAX − 1	Only when `malloc` fails
Memory per element	Only the data	Data plus one pointer
Speed	Slightly faster (no allocation)	Allocation and free on each operation
Implementation	Simple	Needs pointers

Que 2.4 **What are the applications of stacks? Write a program to check balanced parentheses.**

ANSWER

Applications of stacks

1. **Function calls and recursion:** each call pushes an activation record (return address, parameters, local variables) on the system stack; return pops it.
2. **Expression conversion:** infix to postfix and infix to prefix.
3. **Expression evaluation:** evaluating postfix and prefix expressions.
4. **Parenthesis matching:** checking that brackets in an expression or program are balanced.
5. **Reversing data:** pushing characters of a string and popping them reverses it.
6. **Undo in editors** and **back button** in browsers.

7. **Backtracking:** maze solving, the N-queens problem, depth-first search.
8. **Syntax parsing** in compilers.

Balanced parentheses

Scan the expression from left to right. Push every opening bracket. For every closing bracket, the stack must be non-empty and its top must be the matching opening bracket; pop it. At the end, the stack must be empty.

```c
#include <stdio.h>
#include <string.h>

int match(char open, char close) {
    return (open == '(' && close == ')') ||
           (open == '[' && close == ']') ||
           (open == '{' && close == '}');
}

int balanced(const char *e) {
    char st[100];
    int top = -1;
    for (int i = 0; e[i] != '\0'; i++) {
        char c = e[i];
        if (c == '(' || c == '[' || c == '{')
            st[++top] = c;
        else if (c == ')' || c == ']' || c == '}') {
            if (top == -1 || !match(st[top], c)) return 0;
            top--;
        }
    }
    return top == -1;
}

int main(void) {
    printf("%d\n", balanced("{a*[b+(c-d)]}"));   /* 1: balanced     */
    printf("%d\n", balanced("(a+b]"));            /* 0: not balanced */
    return 0;
}
```

Trace for `{[()]}`:

Symbol	Action	Stack
{	push	{
[	push	{ [
(	push	{ [(
)	top (matches, pop	{ [
]	top [matches, pop	{
}	top { matches, pop	empty

The stack is empty at the end, so the expression is balanced. Time O(n).

Part 2: Expressions: Conversion and Evaluation

Que 2.5 **What are infix, prefix and postfix notations? Why are postfix expressions preferred by compilers?**

ANSWER

An arithmetic expression has operands and operators. Depending on where the operator is written:

Notation	Operator position	Example for A + B	Example for (A + B) * C
Infix	between operands	A + B	(A + B) * C
Prefix (Polish)	before operands	+ A B	* + A B C
Postfix (Reverse Polish)	after operands	A B +	A B + C *

Polish notation is named after the Polish logician Jan Łukasiewicz, who introduced it.

Why postfix is preferred

1. **No brackets are needed.** The order of operations is fixed by the position of operators, so precedence and brackets do not have to be checked again.
2. **Single left-to-right scan:** a postfix expression can be evaluated in one pass with one stack, in O(n) time.
3. **Easy code generation:** postfix maps directly onto stack machine instructions (push operand, apply operator), which is why compilers first convert infix to postfix.

Operator precedence and associativity used in conversion

Operator	Precedence	Associativity
^ (exponent)	highest (3)	right to left
*, /, %	2	left to right
+, −	lowest (1)	left to right

Que 2.6 **Write the algorithm to convert an infix expression to postfix. Convert (A + B) * C − D / E ^ F + G to postfix showing the stack at each step.**

ANSWER

Algorithm INFIX_TO_POSTFIX(Q, P)

Q is the infix expression and P is the postfix expression produced.

1. Push "(" onto the stack and add ")" at the end of Q.
2. Scan Q from left to right and repeat steps 3 to 6 for each symbol until the stack is empty.
3. If the symbol is an **operand**, add it to P.
4. If the symbol is a **left parenthesis**, push it.

5. If the symbol is an **operator** ⊗: - Repeatedly pop from the stack and add to P every operator on top that has **higher precedence than ⊗, or equal precedence when ⊗ is left-associative**. - Push ⊗.
6. If the symbol is a **right parenthesis**: - Repeatedly pop operators and add them to P until a left parenthesis is on top. - Pop and discard the left parenthesis.
7. Exit.

(In the trace below we simply pop everything left on the stack at the end instead of using the extra brackets; the result is the same.)

Trace for (A + B) * C − D / E ^ F + G

Step	Symbol	Stack	Postfix output
1	(	(	
2	A	(	A
3	+	(+	A
4	B	(+	A B
5	)	(empty)	A B +
6	*	*	A B +
7	C	*	A B + C
8	−	−	A B + C *
9	D	−	A B + C * D
10	/	− /	A B + C * D
11	E	− /	A B + C * D E
12	^	− / ^	A B + C * D E
13	F	− / ^	A B + C * D E F
14	+	+	A B + C * D E F ^ / −
15	G	+	A B + C * D E F ^ / − G
16	end	(empty)	A B + C * D E F ^ / − G +

At step 8, − has lower precedence than *, so * is popped first. At step 14, + pops ^, / and − (all of higher or equal precedence) before being pushed.

Postfix: A B + C * D E F ^ / − G +

Complexity: each symbol is pushed and popped at most once, so the time is **O(n)** and the stack needs O(n) space.

C program

```c
#include <stdio.h>
#include <ctype.h>

char st[100]; int top = -1;
void push(char c) { st[++top] = c; }
char pop(void)    { return st[top--]; }

int prec(char c) {
    if (c == '^') return 3;
    if (c == '*' || c == '/' || c == '%') return 2;
    if (c == '+' || c == '-') return 1;
    return 0;                       /* '(' */
}

void infix_to_postfix(const char *in, char *out) {
    int k = 0;
    for (int i = 0; in[i]; i++) {
        char c = in[i];
        if (c == ' ') continue;
        if (isalnum(c)) out[k++] = c;
        else if (c == '(') push(c);
        else if (c == ')') {
            while (top != -1 && st[top] != '(') out[k++] = pop();
            pop();                  /* discard '(' */
        } else {                    /* operator */
            while (top != -1 && st[top] != '(' &&
                   (prec(st[top]) > prec(c) ||
                    (prec(st[top]) == prec(c) && c != '^')))
                out[k++] = pop();
            push(c);
        }
    }
    while (top != -1) out[k++] = pop();
    out[k] = '\0';
}

int main(void) {
    char out[100];
    infix_to_postfix("(A+B)*C-D/E^F+G", out);
    printf("%s\n", out);            /* AB+C*DEF^/-G+ */
    return 0;
}
```

Que 2.7 **Write the algorithm to evaluate a postfix expression. Evaluate 7 3 4 + * 2 5 * − 9 3 / +.**

ANSWER

Algorithm EVALUATE_POSTFIX(P)

1. Add a sentinel ")" at the end of P (or simply scan until the end).
2. Scan P from left to right. For each symbol: - If it is an **operand**, push it. - If it is an **operator** ⊗: pop the top element A, then pop the next element B, compute **B** ⊗ **A**, and push the result.
3. At the end, the single value on the stack is the result.

Note the order: the first value popped is the **right** operand.

Trace for 7 3 4 + * 2 5 * − 9 3 / +

Symbol	Action	Stack (bottom → top)
7	push	7
3	push	7, 3
4	push	7, 3, 4
+	3 + 4 = 7	7, 7
*	7 * 7 = 49	49
2	push	49, 2
5	push	49, 2, 5
*	2 * 5 = 10	49, 10
−	49 − 10 = 39	39
9	push	39, 9
3	push	39, 9, 3
/	9 / 3 = 3	39, 3
+	39 + 3 = 42	42

Result = 42. Time O(n).

```
#include <stdio.h>
#include <ctype.h>
#include <math.h>

int eval_postfix(const char *p) {        /* single-digit operands */
    int st[100], top = -1;
    for (int i = 0; p[i]; i++) {
        char c = p[i];
        if (c == ' ') continue;
        if (isdigit(c)) st[++top] = c - '0';
        else {
            int a = st[top--];            /* right operand */
            int b = st[top--];            /* left operand  */
            switch (c) {
                case '+': st[++top] = b + a; break;
                case '-': st[++top] = b - a; break;
                case '*': st[++top] = b * a; break;
                case '/': st[++top] = b / a; break;
                case '^': st[++top] = (int)pow(b, a); break;
            }
        }
    }
    return st[top];
}
```

Que 2.8 **Explain the conversion of infix to prefix. Convert (P + Q) * (R − S / T) to prefix and evaluate the prefix expression − * 2 3 / 8 4.**

ANSWER

Method (reverse, convert, reverse)

1. Reverse the infix expression. While reversing, change every "(" to ")" and every ")" to "(".
2. Convert the reversed expression to postfix, with one change: when the incoming operator has the **same** precedence as the top of the stack, **do not pop** (push it), except for ^ which pops on equal precedence. This keeps left-to-right associativity correct after the final reversal.
3. Reverse the postfix result. This gives the prefix expression.

Example: (P + Q) * (R − S / T)

Step 1, reversed with brackets swapped: **(T / S − R) * (Q + P)**

Step 2, convert to postfix:

Symbol	Stack	Output
(	(	
T	(	T
/	(/	T
S	(/	T S
−	(−	T S /
R	(−	T S / R
)		T S / R −
*	*	T S / R −
(	* (	T S / R −
Q	* (	T S / R − Q
+	* (+	T S / R − Q
P	* (+	T S / R − Q P
)	*	T S / R − Q P +
end		T S / R − Q P + *

Step 3, reverse: *** + P Q − R / S T**

Check: * applied to (+ P Q) and (− R (/ S T)) is (P + Q) * (R − S / T). Correct.

Evaluation of a prefix expression

Scan the prefix expression from **right to left**. Push operands. On an operator, pop A (the **left** operand this time, because we scan backwards), pop B, compute A ⊗ B, and push the result.

Evaluate − * 2 3 / 8 4:

Symbol (right to left)	Action	Stack (bottom → top)
4	push	4
8	push	4, 8
/	8 / 4 = 2	2
3	push	2, 3
2	push	2, 3, 2
*	2 * 3 = 6	2, 6
–	6 – 2 = 4	4

Result = 4. This is (2 * 3) – (8 / 4).

Que 2.9 **Convert the following without a stack (by bracketing): A * (B + C) / D – E to prefix and postfix.**

ANSWER

Manual (bracketing) method: fully bracket the expression according to precedence, then move each operator to the position of its left bracket (for prefix) or right bracket (for postfix), and remove the brackets.

Fully bracketed: **(((A * (B + C)) / D) – E)**

- Innermost: B + C → postfix **B C +**, prefix **+ B C**.
- A * (B + C) → postfix **A B C + ***, prefix *** A + B C**.
- ... / D → postfix **A B C + * D /**, prefix **/ * A + B C D**.
- ... – E → postfix **A B C + * D / E –**, prefix **– / * A + B C D E**.

Postfix: A B C + * D / E – Prefix: – / * A + B C D E

This method is quick for checking answers, but in the exam show the stack table when the question says "using a stack".

Part 3: Recursion

Que 2.10 **What is recursion? Explain the principles of recursion and how the system stack is used. Differentiate between direct and indirect recursion.**

ANSWER

Recursion is a technique in which a function calls itself, directly or indirectly, to solve a smaller instance of the same problem. A recursive solution has two parts:

1. **Base case:** a condition under which the function returns an answer without calling itself. Without it the recursion never stops (stack overflow).
2. **Recursive case:** the function calls itself on a smaller input that moves towards the base case.

Example: factorial. n! = n × (n − 1)! for n > 0, and 0! = 1.

```
long fact(int n) {
    if (n == 0) return 1;          /* base case      */
    return n * fact(n - 1);        /* recursive case */
}
```

Use of the system stack

Every call creates an **activation record (stack frame)** holding the parameters, local variables and return address, and pushes it on the system stack. When the call returns, its frame is popped and control goes back to the caller.

```
fact(3) calls fact(2) calls fact(1) calls fact(0)


┌──────────────┐
│ fact(0) → 1  │ ← top   (base case returns 1)
│ fact(1) 1*?  │          1 * 1 = 1
│ fact(2) 2*?  │          2 * 1 = 2
│ fact(3) 3*?  │          3 * 2 = 6
│ main         │
└──────────────┘
```

The calls "wind" up the stack until the base case, and then "unwind", each frame completing its multiplication.

Types of recursion

- **Direct recursion:** the function calls itself, as `fact` does.
- **Indirect (mutual) recursion:** function A calls B and B calls A.

```
int isOdd(int n);
int isEven(int n) { return n == 0 ? 1 : isOdd(n - 1); }
int isOdd(int n)  { return n == 0 ? 0 : isEven(n - 1); }
```

- **Linear recursion:** each call makes at most one recursive call (factorial).
- **Tree (binary) recursion:** a call makes two or more recursive calls (Fibonacci).
- **Tail recursion:** the recursive call is the last operation (see Que 2.11).
- **Nested recursion:** the argument of the recursive call is itself a recursive call, as in Ackermann's function.

Que 2.11 **What is tail recursion? Why is it important? How can recursion be removed?**

ANSWER

A recursive function is **tail recursive** if the recursive call is the **very last** thing it does, so nothing remains to be computed after the call returns.

The factorial above is **not** tail recursive, because after `fact(n - 1)` returns we still multiply by n. A tail-recursive version carries the partial result in an extra parameter (an **accumulator**):

```
long fact_tail(int n, long acc) {
    if (n == 0) return acc;
    return fact_tail(n - 1, n * acc);    /* last operation is the call */
}
/* call as fact_tail(5, 1) */
```

Why it matters: in a tail call, the current frame is no longer needed. A compiler can reuse the same frame instead of pushing a new one (**tail call optimisation**), turning the recursion into a loop that uses O(1) stack space.

Removal of recursion

Recursion can always be replaced by iteration.

1. Tail recursion → loop. Replace the recursive call with updates of the parameters and a jump back to the start:

```
long fact_iter(int n) {
    long acc = 1;
    while (n > 0) {          /* same as fact_tail(n - 1, n * acc) */
        acc = n * acc;
        n = n - 1;
    }
    return acc;
}
```

2. General recursion → explicit stack. Simulate what the system does:

- Create an empty stack.
- Wherever the function would call itself, push the current state (parameters, local variables, and which point to return to) and start working on the new parameters.
- When a base case is reached, pop a saved state and continue from where it left off.
- Stop when the stack is empty.

For example, the recursive inorder traversal of a binary tree can be rewritten with an explicit stack (shown in Unit 4), and the Tower of Hanoi can be solved by pushing (n, from, to, via) tuples.

Que 2.12 **Write recursive and iterative solutions for binary search and Fibonacci numbers. Analyse their complexity.**

ANSWER

1. Binary search (on a sorted array)

Compare the key with the middle element. If equal, we are done; if the key is smaller, search the left half; otherwise search the right half.

```c
/* recursive */
int bsearch_rec(int a[], int low, int high, int key) {
    if (low > high) return -1;                  /* base case: not found */
    int mid = low + (high - low) / 2;
    if (a[mid] == key) return mid;
    if (key < a[mid]) return bsearch_rec(a, low, mid - 1, key);
    return bsearch_rec(a, mid + 1, high, key);
}

/* iterative */
int bsearch_iter(int a[], int n, int key) {
    int low = 0, high = n - 1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == key) return mid;
        if (key < a[mid]) high = mid - 1; else low = mid + 1;
    }
    return -1;
}
```

Recurrence: T(n) = T(n/2) + c, which solves to **O(log n)** time. The recursive version uses O(log n) stack space; the iterative version uses O(1). (Both are tail recursive in nature, so the conversion is direct.)

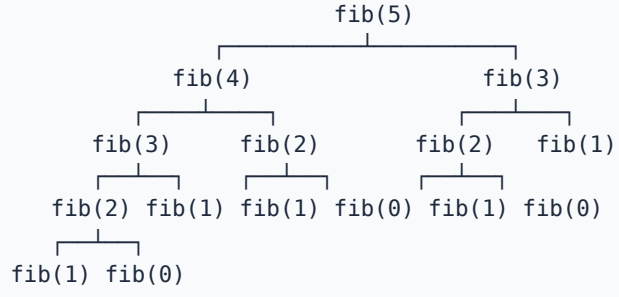
2. Fibonacci numbers

F(0) = 0, F(1) = 1, F(n) = F(n − 1) + F(n − 2). The sequence is 0, 1, 1, 2, 3, 5, 8, 13, 21, ...

```c
/* recursive (tree recursion) */
int fib_rec(int n) {
    if (n <= 1) return n;
    return fib_rec(n - 1) + fib_rec(n - 2);
}

/* iterative */
int fib_iter(int n) {
    int a = 0, b = 1;
    for (int i = 0; i < n; i++) {
        int t = a + b;
        a = b;
        b = t;
    }
    return a;
}
```

Recursion tree for fib(5):



fib(3) is computed twice and fib(2) three times. The number of calls grows roughly like 1.618^n, so the recursive version takes **exponential time $O(2^n)$** (more precisely $O(\varphi^n)$), with O(n) stack depth. The iterative version takes **O(n)** time and O(1) space. Storing computed values in an array (memoization) also brings the recursive version down to O(n).

Que 2.13 **Explain the Tower of Hanoi problem. Write a recursive algorithm, trace it for 3 discs and find its complexity.**

ANSWER

Problem: there are three pegs, Source (A), Auxiliary (B) and Destination (C). n discs of decreasing size are stacked on A, the largest at the bottom. Move all discs to C, following these rules:

1. Only one disc can be moved at a time.
2. Only the top disc of a peg can be moved.
3. A larger disc can never be placed on a smaller disc.

Recursive idea: to move n discs from A to C,

1. Move the top n − 1 discs from A to B, using C as auxiliary.
2. Move the largest disc from A to C.
3. Move the n − 1 discs from B to C, using A as auxiliary.

```c
#include <stdio.h>

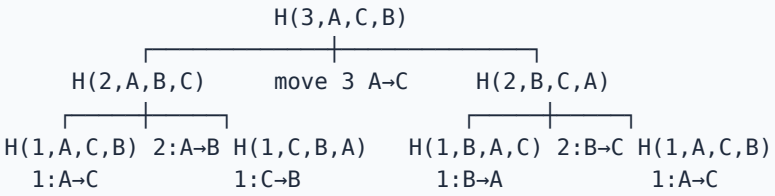
void hanoi(int n, char from, char to, char via) {
    if (n == 0) return;
    hanoi(n - 1, from, via, to);
    printf("Move disc %d from %c to %c\n", n, from, to);
    hanoi(n - 1, via, to, from);
}

int main(void) { hanoi(3, 'A', 'C', 'B'); return 0; }
```

Moves for n = 3

Move	Disc	From	To
1	1	A	C
2	2	A	B
3	1	C	B
4	3	A	C
5	1	B	A
6	2	B	C
7	1	A	C

Recursion tree (H(n, from, to, via))



Complexity

$T(n) = 2T(n - 1) + 1$, $T(0) = 0$.

Expanding: $T(n) = 2T(n - 1) + 1 = 4T(n - 2) + 2 + 1 = \ldots = 2^n\, T(0) + 2^{n-1} + \ldots + 2 + 1 = \mathbf{2^n - 1}$.

So the number of moves is $2^n - 1$ and the time complexity is **$O(2^n)$**. The recursion depth is n, so the space is **O(n)**. For n = 3 we get 7 moves, matching the table.

Que 2.14 **Compare recursion and iteration. What are the trade-offs between them?**

ANSWER

Basis	Recursion	Iteration
Definition	A function calls itself	A set of statements is repeated with a loop
Termination	When the base case is reached	When the loop condition becomes false
Memory	Uses the system stack, one frame per call: O(depth)	Usually O(1) extra memory
Speed	Slower, because of call and return overhead	Faster
Risk	Stack overflow if depth is too large or no base case	Infinite loop if the condition never fails
Code size	Short and close to the mathematical definition	Can be longer
Best for	Problems that are naturally recursive: trees, Tower of Hanoi, quick sort, merge sort, DFS	Simple repetition: sums, linear search, factorial

Trade-offs

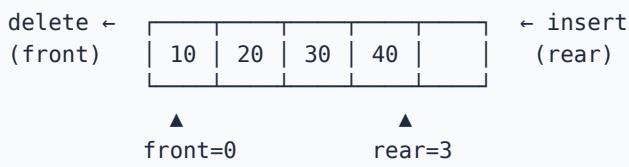
- Recursion makes the code **clear and short** for problems with recursive structure, but costs time and stack memory.
- Some naive recursive solutions repeat work (Fibonacci), making them exponential; iteration or memoization fixes this.
- Every recursive program can be converted to an iterative one (using a loop for tail recursion, or an explicit stack in general), and every iterative program can be written recursively.
- Choose recursion when clarity matters and the depth is small (for example O(log n)); choose iteration when performance and memory matter.

Part 4: Queues

Que 2.15 **What is a queue? Explain the operations on a queue and write a C program for a queue using an array.**

ANSWER

A **queue** is a linear data structure in which insertion is done at one end, called the **rear**, and deletion at the other end, called the **front**. The first element inserted is the first one removed, so a queue follows **FIFO (First In, First Out)**. A line of people at a ticket counter is a queue.



Operations

- **Create:** make an empty queue (front = rear = −1).
- **Add (enqueue / insert):** insert at the rear. If the queue is full, **overflow**.
- **Delete (dequeue):** remove from the front. If the queue is empty, **underflow**.
- **isFull** and **isEmpty**: test the conditions.
- **Peek (front):** read the front element without removing it.

Algorithms (linear queue)

```
ENQUEUE(Q, ITEM)
1. If REAR = MAX − 1, print "Overflow" and exit
2. If FRONT = −1, set FRONT = 0
3. REAR = REAR + 1
4. Q[REAR] = ITEM

DEQUEUE(Q)
1. If FRONT = −1 or FRONT > REAR, print "Underflow" and exit
2. ITEM = Q[FRONT]
3. FRONT = FRONT + 1
4. Return ITEM
```

C program

```c
#include <stdio.h>
#define MAX 5

int q[MAX], front = -1, rear = -1;

int isEmpty(void) { return front == -1 || front > rear; }
int isFull(void)  { return rear == MAX - 1; }

void enqueue(int x) {
    if (isFull()) { printf("Queue overflow\n"); return; }
    if (front == -1) front = 0;
    q[++rear] = x;
}

int dequeue(void) {
    if (isEmpty()) { printf("Queue underflow\n"); return -1; }
    return q[front++];
}

void display(void) {
    for (int i = front; i <= rear && front != -1; i++) printf("%d ", q[i]);
    printf("\n");
}
```

All operations are **O(1)**.

Drawback of the linear array queue: after some deletions, front moves forward and the slots before it are wasted. With MAX = 5, after inserting 5 elements and deleting 3, rear = 4 = MAX − 1, so the queue reports "full" even though 3 slots are empty. The **circular queue** solves this.

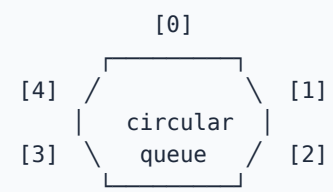
Que 2.16 **What is a circular queue? Write a C program to implement it. Trace a sequence of operations.**

ANSWER

In a **circular queue** the array is treated as a circle: after the last index MAX − 1 comes index 0 again. Positions are advanced with modulo arithmetic:

rear = (rear + 1) % MAX, front = (front + 1) % MAX

so freed slots at the beginning are reused.



Conditions

- Empty: front == −1.
- Full: (rear + 1) % MAX == front.

- One element: front == rear.

```
#include <stdio.h>
#define MAX 5

int cq[MAX], front = -1, rear = -1;

void enqueue(int x) {
    if ((rear + 1) % MAX == front) { printf("Overflow\n"); return; }
    if (front == -1) front = 0;          /* first element */
    rear = (rear + 1) % MAX;
    cq[rear] = x;
}

int dequeue(void) {
    if (front == -1) { printf("Underflow\n"); return -1; }
    int x = cq[front];
    if (front == rear) front = rear = -1; /* queue becomes empty */
    else front = (front + 1) % MAX;
    return x;
}

void display(void) {
    if (front == -1) { printf("Empty\n"); return; }
    int i = front;
    while (1) {
        printf("%d ", cq[i]);
        if (i == rear) break;
        i = (i + 1) % MAX;
    }
    printf("\n");
}
```

Trace with MAX = 5

Operation	front	rear	Array [0..4]	Note
start	−1	−1	_ _ _ _ _	empty
enqueue 10	0	0	10 _ _ _ _	
enqueue 20	0	1	10 20 _ _ _	
enqueue 30	0	2	10 20 30 _ _	
enqueue 40	0	3	10 20 30 40 _	
enqueue 50	0	4	10 20 30 40 50	full: (4+1)%5 = 0 = front
dequeue → 10	1	4	_ 20 30 40 50	
dequeue → 20	2	4	_ _ 30 40 50	
enqueue 60	2	0	60 _ 30 40 50	rear wraps to 0
enqueue 70	2	1	60 70 30 40 50	full again

A linear queue would have reported overflow at "enqueue 60". The circular queue reuses slots 0 and 1.

Alternative: some books keep one slot always empty and use front = rear = 0 for empty; then full is (rear + 1) % MAX == front and the queue holds at most MAX − 1 elements. Another way is to keep a `count` variable.

Applications: CPU round-robin scheduling, keyboard and I/O buffers, traffic light control, streaming buffers.

Que 2.17 **Implement a queue using a linked list.**

ANSWER

We keep two pointers: **front** (first node, where deletion happens) and **rear** (last node, where insertion happens). Both operations are O(1) and there is no overflow except when memory is exhausted.




```c
#include <stdio.h>
#include <stdlib.h>

struct node { int data; struct node *next; };
struct node *front = NULL, *rear = NULL;

void enqueue(int x) {
    struct node *p = malloc(sizeof(struct node));
    p->data = x;
    p->next = NULL;
    if (rear == NULL) front = rear = p;   /* empty queue */
    else { rear->next = p; rear = p; }
}

int dequeue(void) {
    if (front == NULL) { printf("Underflow\n"); return -1; }
    struct node *p = front;
    int x = p->data;
    front = front->next;
    if (front == NULL) rear = NULL;       /* queue became empty */
    free(p);
    return x;
}

void display(void) {
    for (struct node *p = front; p != NULL; p = p->next) printf("%d ", p->data);
    printf("\n");
}
```

A common mistake is forgetting to set `rear = NULL` when the last node is deleted; the next enqueue would then write through a dangling pointer.

Que 2.18 **What is a deque? Explain input-restricted and output-restricted deques.**

ANSWER

A **deque (double-ended queue)** is a linear list in which elements can be inserted and deleted at **both** ends (front and rear), but not in the middle. It is pronounced "deck". A deque can work both as a stack and as a queue.

Operations: insertFront, insertRear, deleteFront, deleteRear, getFront, getRear, isEmpty, isFull.



Types

1. **Input-restricted deque:** insertion at only one end (rear), deletion at both ends.
2. **Output-restricted deque:** deletion at only one end (front), insertion at both ends.

Circular array implementation (main functions)

NEXT ENGINEER
sheerat

```c
#define MAX 10
int dq[MAX], f = -1, r = -1;

void insertFront(int x) {
    if ((f == 0 && r == MAX - 1) || f == r + 1) { printf("Overflow\n"); return; }
    if (f == -1) f = r = 0;
    else if (f == 0) f = MAX - 1;
    else f = f - 1;
    dq[f] = x;
}

void insertRear(int x) {
    if ((f == 0 && r == MAX - 1) || f == r + 1) { printf("Overflow\n"); return; }
    if (f == -1) f = r = 0;
    else r = (r + 1) % MAX;
    dq[r] = x;
}

int deleteFront(void) {
    if (f == -1) { printf("Underflow\n"); return -1; }
    int x = dq[f];
    if (f == r) f = r = -1;
    else f = (f + 1) % MAX;
    return x;
}

int deleteRear(void) {
    if (f == -1) { printf("Underflow\n"); return -1; }
    int x = dq[r];
    if (f == r) f = r = -1;
    else if (r == 0) r = MAX - 1;
    else r = r - 1;
    return x;
}
```

A doubly linked list also gives an easy deque with O(1) operations at both ends.

Applications: the "undo" list with a size limit (old entries removed from the other end), sliding-window maximum, palindrome checking, job-stealing schedulers.

Que 2.19 **What is a priority queue? Explain its implementations.**

ANSWER

A **priority queue** is a collection in which each element has a **priority**, and elements are deleted in order of priority rather than order of arrival. Two rules apply:

1. An element of higher priority is processed before any element of lower priority.
2. Elements of equal priority are processed in the order they were added (FIFO).

(In some systems a smaller number means higher priority, as in process scheduling.)

Types

- **Ascending (min) priority queue:** the smallest element is removed first.
- **Descending (max) priority queue:** the largest element is removed first.

Implementations

1. Unsorted array or list: insert at the end in O(1); to delete, search for the highest priority in O(n).

2. Sorted array or linked list: keep the elements in order of priority. Insertion must find the correct place, O(n); deletion takes the first element, O(1).

```
struct pnode { int data; int prio; struct pnode *next; };
struct pnode *head = NULL;

/* smaller prio value = higher priority; equal priorities keep FIFO order */
void pq_insert(int x, int pr) {
    struct pnode *p = malloc(sizeof(struct pnode));
    p->data = x; p->prio = pr;
    if (head == NULL || pr < head->prio) { p->next = head; head = p; return; }
    struct pnode *q = head;
    while (q->next != NULL && q->next->prio <= pr) q = q->next;
    p->next = q->next;
    q->next = p;
}

int pq_delete(void) {
    if (head == NULL) return -1;
    struct pnode *p = head;
    int x = p->data;
    head = head->next;
    free(p);
    return x;
}
```

3. Array of queues: if there are only k priority levels, keep one FIFO queue for each level. Delete from the highest non-empty queue.

4. Binary heap (best): a complete binary tree stored in an array in which every parent has priority at least that of its children (Unit 4). Both insert and delete take **O(log n)**.

Implementation	Insert	Delete highest
Unsorted array/list	O(1)	O(n)
Sorted array/list	O(n)	O(1)
Binary heap	O(log n)	O(log n)

Applications: CPU scheduling by priority, Dijkstra's and Prim's algorithms, Huffman coding, event-driven simulation, bandwidth management in routers.

Que 2.20 **How can a queue be implemented using two stacks?**

ANSWER

Use two stacks, S1 for incoming elements and S2 for outgoing elements.

- **Enqueue(x):** push x on S1. O(1).

- **Dequeue():** if S2 is empty, pop every element of S1 and push it on S2 (this reverses the order, so the oldest element comes to the top of S2). Then pop from S2.

Each element is moved from S1 to S2 at most once, so although one dequeue can take O(n), the **amortised** cost per operation is O(1).

Example: enqueue 1, 2, 3 gives S1 = [1, 2, 3] (top 3). Dequeue: S2 is empty, so move all: S2 = [3, 2, 1] (top 1); pop gives 1, which is correct FIFO order.

NEXT ENGINEER
sheerat

Short-Answer Questions (2 marks)

Q1. What is a stack? Why is it called LIFO?

A stack is a linear structure where insertion and deletion happen at one end (top). The last element inserted is the first removed, hence Last In First Out.

Q2. What are overflow and underflow?

Overflow is an attempt to insert into a full structure; underflow is an attempt to delete from an empty one.

Q3. Convert A + B * C − D to postfix.

A B C * + D −.

Q4. Evaluate the postfix expression 5 6 2 + * 12 4 / −.

6 + 2 = 8, 5 × 8 = 40, 12 / 4 = 3, 40 − 3 = **37**.

Q5. What is tail recursion?

Recursion in which the recursive call is the last statement executed, so no work remains after it returns. It can be optimised into a loop.

Q6. How many moves are needed for the Tower of Hanoi with n discs?

$2^n - 1$. For 4 discs, 15 moves.

Q7. What is the drawback of a linear queue? How is it removed?

Space freed at the front by deletions cannot be reused, so the queue may report full while empty slots exist. A circular queue removes this.

Q8. Write the full and empty conditions of a circular queue.

Empty: front == −1. Full: (rear + 1) % MAX == front.

Q9. Define deque and its types.

A deque allows insertion and deletion at both ends. Types: input-restricted (insert at one end only) and output-restricted (delete at one end only).

Q10. What is a priority queue? Give two applications.

A queue where each element has a priority and the highest priority element is removed first. Applications: CPU scheduling, Dijkstra's shortest path algorithm.

Q11. Give the prefix form of (A + B) * (C − D).

* + A B − C D.

Q12. List any four applications of stacks.

Function calls/recursion, expression conversion and evaluation, parenthesis matching, undo operations.

NEXT ENGINEER _sheerat_

Practice Questions

1. Convert the infix expression A * (B + C) ^ D / E − F to postfix using a stack, showing every step.
2. Convert (A − B / C) * (D * E − F) to prefix using a stack.
3. Evaluate the postfix expression 8 2 3 ^ / 2 3 * + 5 1 * − using a stack.
4. Write a C program to implement two stacks in a single array so that neither overflows unless the whole array is full.
5. Write a recursive function to find the GCD of two numbers and convert it into an iterative function.
6. Draw the recursion tree for the Tower of Hanoi with 4 discs and list the first eight moves.
7. Write a C program for a circular queue using a linked list.
8. Write algorithms for insertion and deletion at both ends of a deque using a doubly linked list.
9. Explain how a priority queue can be implemented using a linked list. Insert (A,3), (B,1), (C,2), (D,1) and show the list.
10. Why is the recursive Fibonacci function inefficient? Show how memoization improves it.

NEXT ENGINEER
sheerat

UNIT 3

Searching, Hashing and Sorting

Unit overview

Searching and sorting are the two most common operations performed on data, and a large share of computing time in real systems goes into them. This unit studies the classic searching techniques, hashing (which can search in near-constant time), and seven sorting algorithms with their dry runs and complexity.

Topics in this unit: concept of searching, sequential (linear) search, index sequential search, binary search, concept of hashing, hash functions, collision resolution (open addressing: linear probing, quadratic probing, double hashing; separate chaining), sorting: insertion, selection, bubble, quick, merge, heap and radix sort.

Most asked: a dry run of quick sort, merge sort or heap sort on a given array; hashing numericals with linear and quadratic probing; binary search trace; and the comparison table of sorting algorithms.

Part 1: Searching

Que 3.1 **What is searching? Explain sequential (linear) search with algorithm and complexity.**

ANSWER

Searching is the process of finding whether an element with a given value (the **key**) is present in a collection, and if so, finding its location. A search is **successful** if the key is found and **unsuccessful** otherwise.

Sequential (linear) search

Compare the key with each element of the list, one by one, from the beginning. Stop when a match is found or the list ends. The list does not need to be sorted.

```
LINEAR_SEARCH(A, n, key)
1. Repeat for i = 0 to n − 1
2.     If A[i] = key, return i          // successful
3. Return −1                            // unsuccessful
```

```
int linear_search(int a[], int n, int key) {
    for (int i = 0; i < n; i++)
        if (a[i] == key) return i;
    return -1;
}
```

Sentinel version: put the key at position n (a sentinel) so the loop need not check i < n each time; it is guaranteed to stop. This halves the number of comparisons per step.

```
int sentinel_search(int a[], int n, int key) {   /* a must have room for n + 1 items */
    a[n] = key;
    int i = 0;
    while (a[i] != key) i++;
    return (i < n) ? i : -1;
}
```

Complexity

Case	Comparisons	Order
Best (key at first position)	1	O(1)
Worst (key last or absent)	n	O(n)
Average (key equally likely anywhere)	(n + 1)/2	O(n)

The average comes from (1 + 2 + ... + n)/n = (n + 1)/2.

Advantages: simple, works on unsorted data and on linked lists. **Disadvantage:** slow for large n.

Que 3.2 **Explain binary search with algorithm, an example and complexity analysis.**

ANSWER

Binary search works on a **sorted** array. It compares the key with the middle element:

- If they are equal, the search succeeds.
- If the key is smaller, the search continues in the left half.
- If the key is larger, it continues in the right half.

Each comparison halves the search space.

```
BINARY_SEARCH(A, n, key)
1. Set low = 0, high = n − 1
2. Repeat while low ≤ high
3.     mid = (low + high) / 2      (integer division)
4.     If A[mid] = key, return mid
5.     Else if key < A[mid], set high = mid − 1
6.     Else set low = mid + 1
7. Return −1
```

(The C code, both iterative and recursive, is given in Unit 2, Que 2.12.)

Example: search for 47 in

Index	0	1	2	3	4	5	6	7	8	9
Value	5	12	18	23	31	39	47	56	68	74

Step	low	high	mid	A[mid]	Decision
1	0	9	4	31	47 > 31, low = 5
2	5	9	7	56	47 < 56, high = 6
3	5	6	5	39	47 > 39, low = 6
4	6	6	6	47	found at index 6

Searching for 20 would end with low = 3 > high = 2, an unsuccessful search after 4 comparisons.

Complexity

After k comparisons the remaining size is $n/2^k$. The search stops when this becomes 1, so $k = \log_2 n$. The recurrence is $T(n) = T(n/2) + 1$, giving:

Case	Order
Best	O(1) (key in the middle)
Worst	O(log n), at most $\lfloor \log_2 n \rfloor + 1$ comparisons
Average	O(log n)

For n = 1,000,000 binary search needs at most 20 comparisons, while linear search may need 1,000,000.

Limitations: the array must be sorted, and it needs random access, so it is not efficient on a linked list. Insertions and deletions in a sorted array cost O(n).

Linear vs binary search

Basis	Linear search	Binary search
Data	Sorted or unsorted	Must be sorted
Method	Sequential comparison	Divide the range in half
Worst case	O(n)	O(log n)
Structure	Array or linked list	Array (random access)
Best for	Small or unsorted lists	Large sorted lists searched often

Que 3.3 **What is index sequential search? Explain with an example.**

ANSWER

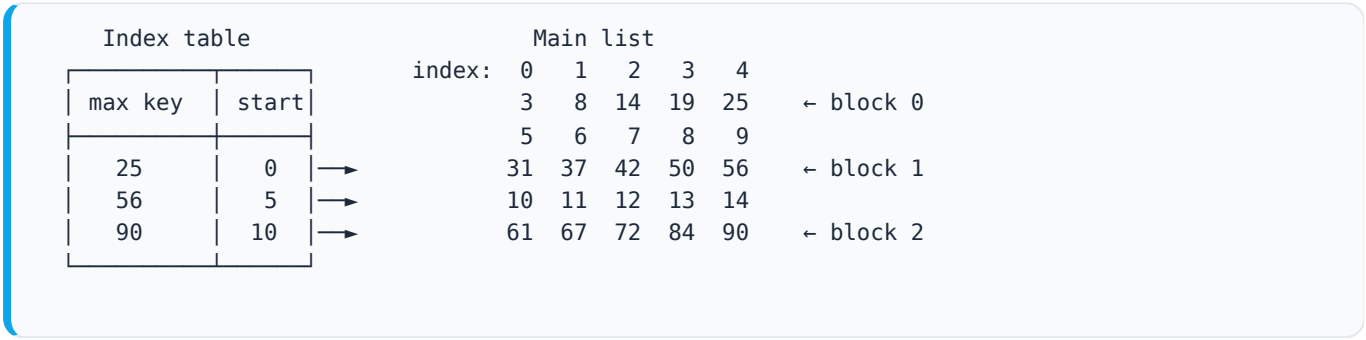
Index sequential search (indexed sequential access) combines an index table with sequential search. The data file is **sorted** and divided into blocks. A smaller **index table** stores, for each block, the largest (or first) key in the block and a pointer to where the block starts.

To search for a key:

1. Search the index table (sequentially or by binary search) to find the block whose range contains the key.

2. Search sequentially only inside that block.

Example: a sorted list of 15 records divided into blocks of 5.



To find 42: in the index, 42 > 25 and 42 ≤ 56, so go to block 1 (start 5) and search positions 5 to 9 sequentially. Found at position 7 after 2 index comparisons and 3 list comparisons, instead of 8 comparisons with linear search.

Complexity: if n records are divided into blocks of size b, there are n/b index entries. Searching takes about n/b + b comparisons in the worst case, which is smallest when b = √n, giving **O(√n)**. With binary search on the index it becomes O(log(n/b) + b).

Use: this is the basis of ISAM (Indexed Sequential Access Method) files in databases, where the index is kept in memory and the blocks on disk. A multi-level index is used for very large files.

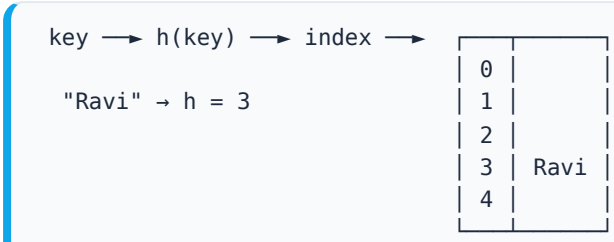
Part 2: Hashing

Que 3.4 **What is hashing? Explain hash tables and the characteristics of a good hash function.**

ANSWER

Hashing is a technique that maps a key directly to a location (an index) in a table using a mathematical function, so that the element can be inserted or found in nearly constant time, O(1), on average, without comparing it with other keys.

- **Hash table:** an array of m slots (buckets) in which records are stored.
- **Hash function h(k):** a function that converts a key k into an index between 0 and m − 1. The value h(k) is called the **hash address** or **home address** of k.
- **Collision:** when two different keys $k_1 \neq k_2$ get the same address, $h(k_1) = h(k_2)$. The keys are called **synonyms**.
- **Load factor α = n/m:** the number of stored keys divided by the table size. It measures how full the table is.



Characteristics of a good hash function

1. It should be **easy and fast** to compute.
2. It should **distribute keys uniformly** over the table, so that every slot is equally likely.
3. It should **minimise collisions**.
4. It should use **all parts** of the key, so that keys that differ slightly get different addresses.
5. It must be **deterministic**: the same key must always give the same address.

Que 3.5 **Explain different hash functions with examples.**

ANSWER

1. Division method

$h(k) = k \bmod m$

where m is the table size. m should be a **prime number not close to a power of 2**, because if $m = 2^p$, h(k) depends only on the lowest p bits of k. Example: m = 11, k = 1234 → 1234 mod 11 = 2 (because 11 × 112 = 1232).

2. Mid-square method

Square the key and take the middle r digits as the address (for a table of size 10^r). Example: k = 3205, k^2 = 10272025. Taking the middle two digits (4th and 5th) gives **72**. It works well because the middle digits depend on all digits of the key.

3. Folding method

Split the key into parts of equal size (the last may be shorter) and add them; ignore the final carry to fit the table size.

- **Shift folding:** add the parts as they are. k = 123456789 split into 123, 456, 789 → 123 + 456 + 789 = 1368 → ignore the carry 1 → **368**.
- **Boundary folding:** reverse alternate parts before adding. 123 + 654 + 789 = 1566 → **566**.

4. Multiplication method

$h(k) = \lfloor m \cdot (k \cdot A \bmod 1) \rfloor$, where $0 < A < 1$ is a constant (Knuth suggests A ≈ 0.6180339887). The fractional part of k·A is multiplied by m. The choice of m is not critical here, so m can be a power of 2. Example: m = 100, k = 123, A = 0.618 → k·A = 76.014 → fractional part 0.014 → h = ⌊1.4⌋ = **1**.

5. Digit extraction (truncation)

Pick some digits of the key in fixed positions. k = 9876543, choosing the 2nd, 4th and 6th digits gives 864.

NEXT ENGINEER _sheerat_

Que 3.6 **What is a collision? Explain collision resolution by open addressing: linear probing, quadratic probing and double hashing.**

ANSWER

A **collision** occurs when the home address of a new key is already occupied by another key. Collisions cannot be avoided completely, because the number of possible keys is far larger than the table size. Collision resolution techniques are of two types: **open addressing (closed hashing)**, in which all keys are stored inside the table itself, and **separate chaining (open hashing)**, in which keys are stored in linked lists outside the table.

In **open addressing**, when the home slot is occupied we **probe** (examine) a sequence of other slots until an empty slot is found. The probe sequence for the i-th attempt (i = 0, 1, 2, ...) is:

1. Linear probing: $h(k, i) = (h(k) + i) \bmod m$. Check the next slot, then the next, wrapping around.

- Simple, and has good cache locality.
- Problem: **primary clustering**: long runs of occupied slots build up, and any key that hashes into a run makes it longer, increasing search time.

2. Quadratic probing: $h(k, i) = (h(k) + c_1 \cdot i + c_2 \cdot i^2) \bmod m$, commonly $h(k, i) = (h(k) + i^2) \bmod m$. Probe positions h, h + 1, h + 4, h + 9, ...

- Removes primary clustering.
- Problem: **secondary clustering**: keys with the same home address follow the same probe sequence. Also, it may fail to find an empty slot even if one exists, unless m is prime and the table is less than half full.

3. Double hashing: $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod m$. A second hash function decides the step size, for example $h_2(k) = R - (k \bmod R)$ where R is a prime smaller than m. $h_2(k)$ must never be 0.

- Different keys with the same home address follow different probe sequences, so it removes both primary and secondary clustering.
- It costs a second hash computation.

Deletion in open addressing: a deleted slot cannot simply be emptied, because that would break the probe chain of later keys. It is marked **DELETED** (a tombstone): searches pass over it, and insertions may reuse it.

Que 3.7 **Insert the keys 72, 27, 36, 24, 63, 81, 92, 101 into a hash table of size 11 using $h(k) = k \bmod 11$ with (a) linear probing, (b) quadratic probing, (c) double hashing with $h_2(k) = 7 - (k \bmod 7)$, and (d) separate chaining.**

ANSWER

Home addresses

Key	72	27	36	24	63	81	92	101
k mod 11	6	5	3	2	8	4	4	2

NEXT ENGINEER _sheerat_

The first six keys go directly to their home slots. Collisions occur for 92 (slot 4 is taken by 81) and 101 (slot 2 is taken by 24).

(a) Linear probing

- 92: slot 4 full → 5 full → 6 full → 7 empty. **92 at 7** (3 collisions).
- 101: slot 2 full → 3 → 4 → 5 → 6 → 7 → 8 all full → 9 empty. **101 at 9** (7 collisions).

Slot	0	1	2	3	4	5	6	7	8	9	10
Key			24	36	81	27	72	92	63	101	

The long run from 2 to 9 is primary clustering at work.

(b) Quadratic probing, $h(k, i) = (h(k) + i^2) \bmod 11$

- 92: i = 0 → 4 full; i = 1 → 5 full; i = 2 → 8 full; i = 3 → 13 mod 11 = 2 full; i = 4 → 20 mod 11 = 9 empty. **92 at 9**.
- 101: i = 0 → 2 full; i = 1 → 3 full; i = 2 → 6 full; i = 3 → 11 mod 11 = 0 empty. **101 at 0**.

Slot	0	1	2	3	4	5	6	7	8	9	10
Key	101		24	36	81	27	72		63	92	

(c) Double hashing, $h(k, i) = (h_1(k) + i \cdot h_2(k)) \bmod 11$, $h_2(k) = 7 - (k \bmod 7)$

- 92: $h_1 = 4$ (full). 92 mod 7 = 1, so $h_2 = 6$. i = 1 → (4 + 6) mod 11 = 10 empty. **92 at 10**.
- 101: $h_1 = 2$ (full). 101 mod 7 = 3, so $h_2 = 4$. i = 1 → 6 full; i = 2 → 10 full (92); i = 3 → 14 mod 11 = 3 full; i = 4 → 18 mod 11 = 7 empty. **101 at 7**.

Slot	0	1	2	3	4	5	6	7	8	9	10
Key			24	36	81	27	72	101	63		92

(d) Separate chaining

Each slot holds a linked list of all keys hashing to it (new keys added at the end).

```
 0 → NULL
 1 → NULL
 2 → [24] → [101] → NULL
 3 → [36] → NULL
 4 → [81] → [92] → NULL
 5 → [27] → NULL
 6 → [72] → NULL
 7 → NULL
 8 → [63] → NULL
 9 → NULL
10 → NULL
```

Que 3.8 **Explain separate chaining. Compare it with open addressing. What is rehashing?**

ANSWER

Separate chaining (open hashing): each slot of the hash table is the head of a linked list. All keys that hash to the same slot are stored in that slot's list. To insert, compute h(k) and add the key to that list (at the front for O(1), or in sorted order). To search, compute h(k) and search that list only.

```
#define M 11
struct node { int key; struct node *next; };
struct node *table[M];                /* all NULL initially */

void chain_insert(int k) {
    int h = k % M;
    struct node *p = malloc(sizeof(struct node));
    p->key = k;
    p->next = table[h];               /* insert at front */
    table[h] = p;
}

struct node *chain_search(int k) {
    for (struct node *p = table[k % M]; p != NULL; p = p->next)
        if (p->key == k) return p;
    return NULL;
}
```

With load factor $\alpha = n/m$, the average chain length is α, so an unsuccessful search takes $\Theta(1 + \alpha)$ and a successful one about $1 + \alpha/2$ comparisons. If m is proportional to n, this is O(1).

Comparison

Basis	Separate chaining	Open addressing
Where keys are stored	In linked lists outside the table	Inside the table
Load factor	Can exceed 1	Must be below 1 (ideally below 0.7)
Deletion	Easy, delete from the list	Needs DELETED markers
Clustering	None	Primary/secondary clustering possible
Extra memory	Pointers in every node	None, but needs empty slots
Cache performance	Poorer	Better (linear probing)
Performance as table fills	Degrades gracefully	Degrades sharply as $\alpha \to 1$

Rehashing: when the load factor becomes too high (for example $\alpha > 0.7$ in open addressing), performance falls. Rehashing creates a new, larger table (usually about twice the size, chosen as a prime), and every key of the old table is **re-inserted** using the new hash function. It costs O(n) at that moment, but happens rarely, so the amortised cost per insertion stays O(1).

Applications of hashing: symbol tables in compilers, database indexing, caches, password storage (with cryptographic hashes), dictionaries and sets in programming languages, duplicate detection.

Part 3: Simple Sorting Algorithms

Que 3.9 **What is sorting? Explain the terms internal sorting, external sorting, stable sorting and in-place sorting.**

ANSWER

Sorting is the process of arranging the elements of a list in a particular order, ascending or descending, according to a key. Sorted data makes searching (binary search), merging and reporting much faster.

- **Internal sorting:** all the data fits in main memory during sorting. Examples: bubble, insertion, quick, heap sort.
- **External sorting:** the data is too large for main memory and is kept on disk; parts are brought into memory, sorted and merged. Example: external (multiway) merge sort.
- **Stable sort:** two records with equal keys keep their original relative order after sorting. This matters when sorting by one key after another (for example by name, then by marks). Stable: bubble, insertion, merge, radix. Not stable: selection, quick, heap.
- **In-place sort:** uses only O(1) extra memory besides the input (a small number of variables). In-place: bubble, insertion, selection, heap, quick (apart from the recursion stack). Not in-place: merge sort (needs O(n)), radix sort.
- **Adaptive sort:** runs faster when the input is already partly sorted. Insertion sort and bubble sort with a flag are adaptive.

Que 3.10 **Explain insertion sort with algorithm, an example and complexity.**

ANSWER

Idea: the array is divided into a sorted part (initially just the first element) and an unsorted part. In each pass, the first element of the unsorted part (the **key**) is taken and inserted into its correct position in the sorted part by shifting larger elements one place to the right. This is how people sort playing cards in their hand.

```
void insertion_sort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;
        while (j >= 0 && a[j] > key) {   /* shift larger elements right */
            a[j + 1] = a[j];
            j--;
        }
        a[j + 1] = key;
    }
}
```

Example: sort 42, 17, 63, 8, 29, 55, 11, 36. (The sorted part is shown before the bar.)

Pass	Key	Array after the pass
1	17	17 42 | 63 8 29 55 11 36
2	63	17 42 63 | 8 29 55 11 36
3	8	8 17 42 63 | 29 55 11 36
4	29	8 17 29 42 63 | 55 11 36
5	55	8 17 29 42 55 63 | 11 36
6	11	8 11 17 29 42 55 63 | 36
7	36	8 11 17 29 36 42 55 63

Complexity

- **Best case** (already sorted): each key is compared once and not moved: n − 1 comparisons, **O(n)**.
- **Worst case** (reverse sorted): the i-th key is compared with all i earlier elements: 1 + 2 + ... + (n − 1) = n(n − 1)/2 comparisons, **O(n^2)**.
- **Average case: O(n^2)**.
- **Space:** O(1). **Stable:** yes. **Adaptive:** yes.

Insertion sort is the best simple sort for small or nearly sorted arrays, and it is used inside faster algorithms (such as hybrid quick sort) for small sub-arrays.

Que 3.11 **Explain selection sort with algorithm, an example and complexity.**

ANSWER

Idea: in pass i, find the smallest element in the unsorted part a[i..n−1] and swap it with a[i]. After pass i, the first i + 1 elements are in their final positions.

```
void selection_sort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int min = i;
        for (int j = i + 1; j < n; j++)
            if (a[j] < a[min]) min = j;
        if (min != i) { int t = a[i]; a[i] = a[min]; a[min] = t; }
    }
}
```

Example: sort 42, 17, 63, 8, 29, 55, 11, 36.

Pass	Minimum found	Swap	Array after the pass
1	8	42 ↔ 8	**8** 17 63 42 29 55 11 36
2	11	17 ↔ 11	**8 11** 63 42 29 55 17 36
3	17	63 ↔ 17	**8 11 17** 42 29 55 63 36
4	29	42 ↔ 29	**8 11 17 29** 42 55 63 36

Pass	Minimum found	Swap	Array after the pass
5	36	42 ↔ 36	**8 11 17 29 36** 55 63 42
6	42	55 ↔ 42	**8 11 17 29 36 42** 63 55
7	55	63 ↔ 55	**8 11 17 29 36 42 55 63**

Complexity

- Comparisons are always (n − 1) + (n − 2) + ... + 1 = n(n − 1)/2, whatever the input. So best, average and worst cases are all **O(n²)**.
- Swaps: at most n − 1, which is the fewest of all simple sorts. This makes selection sort useful when writing to memory is expensive.
- Space O(1). **Not stable** (a swap can jump an element over an equal one; for example 5a 5b 2 becomes 2 5b 5a). **Not adaptive.**

Que 3.12 **Explain bubble sort with algorithm, an example and complexity. How can it be optimised?**

ANSWER

Idea: repeatedly compare adjacent elements and swap them if they are in the wrong order. After each pass, the largest remaining element "bubbles up" to its final position at the end.

Optimised bubble sort: if a complete pass makes no swap, the array is already sorted and we stop.

```
void bubble_sort(int a[], int n) {
    for (int pass = 0; pass < n - 1; pass++) {
        int swapped = 0;
        for (int j = 0; j < n - 1 - pass; j++)
            if (a[j] > a[j + 1]) {
                int t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
                swapped = 1;
            }
        if (!swapped) break;           /* no swaps: already sorted */
    }
}
```

Example: sort 34, 12, 45, 9, 27.

Pass 1

Compare	Action	Array
34, 12	swap	12 34 45 9 27
34, 45	no swap	12 34 45 9 27
45, 9	swap	12 34 9 45 27
45, 27	swap	12 34 9 27 **45**

Pass 2: 12 34 9 27 → (12, 34) no swap; (34, 9) swap → 12 9 34 27; (34, 27) swap → 12 9 27 **34 45**.

Pass 3: (12, 9) swap → 9 12 27; (12, 27) no swap → 9 12 **27 34 45**.

Pass 4: (9, 12) no swap. No swap in this pass, so stop. Sorted: **9 12 27 34 45**.

Complexity

- **Worst case** (reverse sorted): n(n − 1)/2 comparisons and swaps, **$O(n^2)$**.
- **Average case: $O(n^2)$**.
- **Best case** (already sorted, with the flag): one pass of n − 1 comparisons, **O(n)**.
- Space O(1). **Stable:** yes.

Part 4: Divide and Conquer Sorting: Quick Sort and Merge Sort

Que 3.13 **Explain quick sort with the partition algorithm. Sort 42, 17, 63, 8, 29, 55, 11, 36 using quick sort.**

ANSWER

Quick sort, invented by C. A. R. Hoare, is a **divide and conquer** algorithm.

1. **Divide:** choose a **pivot** element and **partition** the array so that all elements less than or equal to the pivot are on its left and all greater elements are on its right. The pivot is now in its final sorted position.
2. **Conquer:** recursively sort the left part and the right part.
3. **Combine:** nothing to do; the array is sorted in place.

Partition (first element as pivot)

1. pivot = a[low], i = low + 1, j = high.
2. Move i right while a[i] ≤ pivot.
3. Move j left while a[j] > pivot.
4. If i < j, swap a[i] and a[j] and repeat from step 2.
5. Otherwise swap a[low] (the pivot) with a[j]. Now j is the pivot's final position.

NEXT ENGINEER _sheerat_

```c
void swap(int *x, int *y) { int t = *x; *x = *y; *y = t; }

int partition(int a[], int low, int high) {
    int pivot = a[low], i = low + 1, j = high;
    while (1) {
        while (i <= high && a[i] <= pivot) i++;
        while (a[j] > pivot) j--;
        if (i < j) swap(&a[i], &a[j]);
        else break;
    }
    swap(&a[low], &a[j]);
    return j;
}

void quick_sort(int a[], int low, int high) {
    if (low < high) {
        int p = partition(a, low, high);
        quick_sort(a, low, p - 1);
        quick_sort(a, p + 1, high);
    }
}
```

Example: a = 42, 17, 63, 8, 29, 55, 11, 36 (indices 0 to 7).

First partition, pivot = 42

Step	i	j	Action	Array
1	2 (63)	7 (36)	i < j, swap 63 and 36	42 17 36 8 29 55 11 63
2	5 (55)	6 (11)	i < j, swap 55 and 11	42 17 36 8 29 11 55 63
3	6 (55)	5 (11)	i > j, swap pivot with a[5]	11 17 36 8 29 **42** 55 63

42 is in its final position (index 5). Left part: 11 17 36 8 29; right part: 55 63.

Left part 11, 17, 36, 8, 29 (pivot 11): i stops at 17 (index 1), j stops at 8 (index 3), swap → 11 8 36 17 29. Next i stops at 36 (index 2), j stops at 8 (index 1); i > j, so swap pivot with a[1] → 8 **11** 36 17 29.

Sub-part 36, 17, 29 (pivot 36): i runs past the end, j stops at 29; swap pivot with 29 → 29 17 **36**. Then 29, 17 (pivot 29) → 17 **29**.

Right part 55, 63 (pivot 55): i stops at 63, j stops at 55 itself, so the pivot stays → **55** 63.

Final sorted array: 8 11 17 29 36 42 55 63

Complexity

- **Best case:** the pivot always splits the array into two equal halves. T(n) = 2T(n/2) + n, which gives **O(n log n)**.
- **Average case: O(n log n)** (about 1.39 n $\log_2$ n comparisons).
- **Worst case:** the pivot is always the smallest or largest element, for example when the array is **already sorted** and the first element is the pivot. Then T(n) = T(n − 1) + n, giving n(n − 1)/2 comparisons, **O(n^2)**.
- **Space:** O(log n) for the recursion stack on average, O(n) in the worst case.

- **Not stable.** In place.

Avoiding the worst case: choose the pivot randomly (**randomised quick sort**), or use the **median of three** (the median of the first, middle and last elements). Switching to insertion sort for very small sub-arrays also speeds it up.

Que 3.14 **Explain merge sort with algorithm and example. Derive its time complexity.**

ANSWER

Merge sort is a **divide and conquer** algorithm, invented by John von Neumann.

1. **Divide:** split the array into two halves.
2. **Conquer:** sort each half recursively by merge sort.
3. **Combine: merge** the two sorted halves into one sorted array.

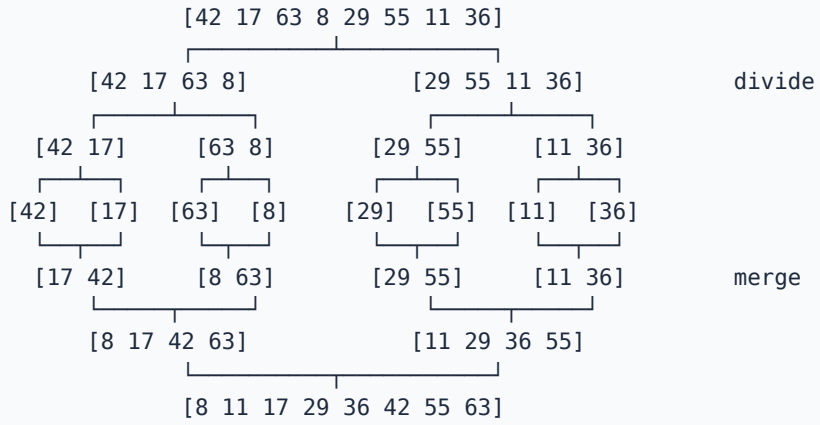
The base case is an array of one element, which is already sorted.

Merging two sorted arrays: compare the front elements of the two halves, copy the smaller one to the output, and advance in that half. When one half is exhausted, copy the rest of the other.

```
void merge(int a[], int low, int mid, int high) {
    int tmp[high - low + 1];
    int i = low, j = mid + 1, k = 0;
    while (i <= mid && j <= high)
        tmp[k++] = (a[i] <= a[j]) ? a[i++] : a[j++];   /* <= keeps it stable */
    while (i <= mid)  tmp[k++] = a[i++];
    while (j <= high) tmp[k++] = a[j++];
    for (k = 0; k < high - low + 1; k++) a[low + k] = tmp[k];
}

void merge_sort(int a[], int low, int high) {
    if (low < high) {
        int mid = (low + high) / 2;
        merge_sort(a, low, mid);
        merge_sort(a, mid + 1, high);
        merge(a, low, mid, high);
    }
}
```

Example: sort 42, 17, 63, 8, 29, 55, 11, 36.



Last merge in detail: [8 17 42 63] and [11 29 36 55]: 8 < 11 → 8; 17 > 11 → 11; 17 < 29 → 17; 42 > 29 → 29; 42 > 36 → 36; 42 < 55 → 42; 63 > 55 → 55; copy 63. Result 8 11 17 29 36 42 55 63.

Complexity

Merging two halves of total size n takes at most n − 1 comparisons, so

$T(n) = 2T(n/2) + cn$, $T(1) = c$.

Expanding: $T(n) = 2T(n/2) + cn = 4T(n/4) + 2cn = 8T(n/8) + 3cn = \ldots = 2^k\, T(n/2^k) + k \cdot cn$.

With $n = 2^k$, that is $k = \log_2 n$: $T(n) = n \cdot T(1) + cn \log_2 n$ = **$O(n \log n)$**.

- Best, average and worst cases are all **$O(n \log n)$**, because the splitting does not depend on the data.
- **Space:** O(n) for the temporary array, plus O(log n) for recursion. Not in place.
- **Stable:** yes.
- Merge sort is ideal for **linked lists** (no extra array needed) and for **external sorting** of huge files.

Quick sort vs merge sort

Basis	Quick sort	Merge sort
Strategy	Hard work in partition (divide)	Hard work in merge (combine)
Worst case	$O(n^2)$	O(n log n)
Average case	O(n log n), usually faster in practice	O(n log n)
Extra space	O(log n) stack, in place	O(n)
Stable	No	Yes
Best use	Arrays in memory	Linked lists, external sorting

Part 5: Heap Sort and Radix Sort

Que 3.15 **What is a heap? Explain heap sort with an example and complexity.**

ANSWER

A **binary heap** is a **complete binary tree** (all levels full except possibly the last, which is filled from the left) that satisfies the **heap property**:

- **Max heap:** every parent is greater than or equal to its children, so the root is the largest element.
- **Min heap:** every parent is less than or equal to its children.

Because it is complete, a heap is stored in an array without pointers. With 0-based indices, for the node at index i:

- left child at 2i + 1, right child at 2i + 2, parent at ⌊(i − 1)/2⌋.

Heap sort idea (ascending order)

1. **Build a max heap** from the array.
2. Repeat for size n down to 2: swap the root (largest) with the last element of the heap, reduce the heap size by one, and **heapify** (sift down) the new root to restore the heap property.

```
void heapify(int a[], int n, int i) {          /* sift down a[i] in a heap of size n */
    int largest = i, l = 2 * i + 1, r = 2 * i + 2;
    if (l < n && a[l] > a[largest]) largest = l;
    if (r < n && a[r] > a[largest]) largest = r;
    if (largest != i) {
        int t = a[i]; a[i] = a[largest]; a[largest] = t;
        heapify(a, n, largest);
    }
}

void heap_sort(int a[], int n) {
    for (int i = n / 2 - 1; i >= 0; i--)       /* build max heap */
        heapify(a, n, i);
    for (int end = n - 1; end > 0; end--) {
        int t = a[0]; a[0] = a[end]; a[end] = t; /* move max to the end */
        heapify(a, end, 0);
    }
}
```

Example: sort 15, 40, 25, 10, 50, 30.

Step 1: build the max heap. Start from the last internal node, index n/2 − 1 = 2.

```
Initial tree:            15
                        /      \
                      40        25
                     /  \      /
                   10    50   30
```

- i = 2 (25): child 30 is larger → swap → 15 40 30 10 50 25.
- i = 1 (40): children 10 and 50; 50 is larger → swap → 15 50 30 10 40 25.
- i = 0 (15): children 50 and 30 → swap with 50 → 50 15 30 10 40 25; continue at index 1 (15): children 10 and 40 → swap with 40 → **50 40 30 10 15 25**.

```
Max heap:                50
                       /      \
                     40        30
                   /  \       /
                 10    15   25
```

Step 2: repeatedly remove the maximum.

Pass	Swap root with last	After heapify (heap | sorted)
1	50 ↔ 25	40 25 30 10 15 | 50
2	40 ↔ 15	30 25 15 10 | 40 50
3	30 ↔ 10	25 10 15 | 30 40 50
4	25 ↔ 15	15 10 | 25 30 40 50
5	15 ↔ 10	10 | 15 25 30 40 50

Sorted: 10 15 25 30 40 50

Complexity

- Heapify on a heap of height h takes O(h) = O(log n).
- Building the heap bottom-up takes **O(n)** in total (most nodes are near the bottom and sift down only a little).
- The n − 1 removals each take O(log n), giving O(n log n).
- So heap sort is **O(n log n)** in the best, average and worst cases.
- **Space:** O(1), in place. **Not stable.**

Que 3.16 **Explain radix sort with an example. What is its complexity?**

ANSWER

Radix sort is a non-comparison sort. It sorts integers digit by digit, starting from the **least significant digit (LSD)** and moving to the most significant. In each pass the numbers are distributed into 10 buckets (0 to 9) according to the current digit, and then collected from bucket 0 to bucket 9. Each pass must be **stable**, which is why the order inside each bucket is kept.

Example: sort 512, 83, 207, 64, 395, 28, 741, 160. The largest number has 3 digits, so 3 passes are needed.

Pass 1 (units digit)

Bucket	0	1	2	3	4	5	6	7	8	9
Numbers	160	741	512	83	64	395		207	28	

Collected: 160, 741, 512, 83, 64, 395, 207, 28

Pass 2 (tens digit)

Bucket	0	1	2	3	4	5	6	7	8	9
Numbers	207	512	28		741		160, 64		83	395

Collected: 207, 512, 28, 741, 160, 64, 83, 395

Pass 3 (hundreds digit)

Bucket	0	1	2	3	4	5	6	7	8	9
Numbers	28, 64, 83	160	207	395		512		741		

Collected: **28, 64, 83, 160, 207, 395, 512, 741** (sorted).

```
void counting_pass(int a[], int n, int exp) {      /* stable sort on one digit */
    int out[n], count[10] = {0};
    for (int i = 0; i < n; i++) count[(a[i] / exp) % 10]++;
    for (int d = 1; d < 10; d++) count[d] += count[d - 1];
    for (int i = n - 1; i >= 0; i--)               /* right to left keeps it stable */
        out[--count[(a[i] / exp) % 10]] = a[i];
    for (int i = 0; i < n; i++) a[i] = out[i];
}

void radix_sort(int a[], int n) {
    int max = a[0];
    for (int i = 1; i < n; i++) if (a[i] > max) max = a[i];
    for (int exp = 1; max / exp > 0; exp *= 10)
        counting_pass(a, n, exp);
}
```

Complexity: with d digits, radix (base) r and n numbers, each pass takes O(n + r), so the total is **O(d(n + r))**. When d is a small constant, this is **O(n)**, linear time. The space needed is O(n + r). Radix sort is stable.

Limitations: it works directly only on integers or fixed-length strings, needs extra memory, and is slow when the keys have many digits.

Que 3.17 **Compare all the sorting algorithms studied.**

ANSWER

Algorithm	Best	Average	Worst	Extra space	Stable	Method
Bubble (with flag)	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Exchange
Selection	$O(n^2)$	$O(n^2)$	$O(n^2)$	O(1)	No	Selection
Insertion	O(n)	$O(n^2)$	$O(n^2)$	O(1)	Yes	Insertion
Quick	O(n log n)	O(n log n)	$O(n^2)$	O(log n)	No	Partition
Merge	O(n log n)	O(n log n)	O(n log n)	O(n)	Yes	Merging
Heap	O(n log n)	O(n log n)	O(n log n)	O(1)	No	Selection (heap)

Algorithm	Best	Average	Worst	Extra space	Stable	Method
Radix	O(d(n + r))	O(d(n + r))	O(d(n + r))	O(n + r)	Yes	Distribution

Which to use

- Small or nearly sorted data: **insertion sort**.
- General-purpose in-memory sorting: **quick sort** (fastest on average).
- Guaranteed O(n log n) with O(1) extra space: **heap sort**.
- Stability needed, linked lists, or external files: **merge sort**.
- Integers with few digits: **radix sort**.

Lower bound: any sorting algorithm that works only by comparing elements needs **Ω(n log n)** comparisons in the worst case (because a decision tree with n! leaves has height at least $\log_2(n!) \approx n \log_2 n$). Radix sort beats this bound because it does not compare elements.

NEXT ENGINEER
sheerat

Short-Answer Questions (2 marks)

Q1. What is the condition for applying binary search?

The list must be sorted and must allow direct (random) access to the middle element, as an array does.

Q2. What is the worst case complexity of linear and binary search?

Linear search O(n); binary search O(log n).

Q3. Define hashing and hash function.

Hashing stores and finds records by computing their table address from the key. A hash function h(k) maps a key k to an index in the range 0 to m − 1.

Q4. What is a collision? Name two techniques to resolve it.

A collision occurs when two keys hash to the same address. Techniques: linear probing (open addressing) and separate chaining.

Q5. What is primary clustering?

In linear probing, occupied slots form long continuous runs; any key hashing into a run lengthens it, which increases the search time.

Q6. What is the load factor of a hash table?

$\alpha = n/m$, the number of keys stored divided by the number of slots.

Q7. Which sorting algorithms are stable?

Bubble, insertion, merge and radix sort.

Q8. What is the worst case of quick sort and when does it occur?

$O(n^2)$. It occurs when the pivot is always the smallest or largest element, for example on an already sorted array with the first element as pivot.

Q9. Why is merge sort not an in-place sort?

Because merging two halves needs a temporary array of size n.

Q10. What is the time complexity of building a heap?

O(n) using the bottom-up method.

Q11. How many passes does bubble sort need for n elements in the worst case?

n − 1 passes.

Q12. Using h(k) = k mod 7, where do keys 50 and 64 go?

50 mod 7 = 1 and 64 mod 7 = 1, so both hash to slot 1 (a collision).

Practice Questions

1. Sort 25, 57, 48, 37, 12, 92, 86, 33 using quick sort, showing each partition.

NEXT ENGINEER _sheerat_

2. Sort the same list using merge sort and heap sort, showing every step.
3. Sort 329, 457, 657, 839, 436, 720, 355 using radix sort.
4. Insert 89, 18, 49, 58, 69 into a table of size 10 using h(k) = k mod 10 with linear probing, quadratic probing and double hashing with $h_2(k) = 7 - (k \bmod 7)$.
5. Write a recursive C function for binary search and trace it for key 23 in 2, 5, 8, 12, 16, 23, 38, 56, 72, 91.
6. Explain index sequential search. Why is the block size $\sqrt{n}$ optimal?
7. Show that the worst case of insertion sort is $O(n^2)$ and the best case is $O(n)$.
8. Explain why selection sort is not stable with an example.
9. Derive the average number of comparisons in a successful linear search.
10. Explain the mid-square and folding hash functions with examples.

NEXT ENGINEER
sheerat

UNIT 4

Trees

Unit overview

A tree stores data in a **hierarchy**, like a family tree or the folders on a computer. Trees are the most heavily examined topic in this subject, because they give many numerical questions: constructing a tree from its traversals, inserting and deleting in a binary search tree, AVL rotations, B-tree insertion and deletion, Huffman codes and heaps.

Topics in this unit: basic terminology, binary trees, array and linked representation, binary search tree, strictly binary tree, complete binary tree, extended binary tree, inorder, preorder and postorder traversal, constructing a binary tree from traversals, insertion, deletion, searching and modification in a BST, threaded binary trees and their traversal, Huffman coding, AVL trees, B-trees and binary heaps.

Most asked: construct a tree from inorder and preorder/postorder; BST deletion with all three cases; AVL insertion with rotations; B-tree of order m; Huffman codes for given frequencies; threaded binary tree diagram.

Part 1: Tree Terminology and Binary Trees

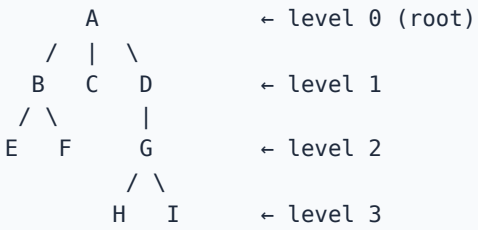
Que 4.1 **Define a tree. Explain the basic terminology used with trees.**

ANSWER

A **tree** is a finite set of one or more nodes such that:

1. There is a specially designated node called the **root**.
2. The remaining nodes are partitioned into n ≥ 0 disjoint sets T_1, T_2, ..., T_n, each of which is itself a tree. These are called the **subtrees** of the root.

This definition is recursive. A tree with n nodes has exactly n − 1 edges, and there is exactly one path between any two nodes (no cycles).



Terminology (using the tree above)

- **Root:** the topmost node with no parent: A.

- **Parent and child:** a node directly above another is its parent; B is the parent of E and F, and E and F are children of B.
- **Siblings:** children of the same parent: B, C, D are siblings.
- **Leaf (terminal / external node):** a node with no children: E, F, C, H, I.
- **Internal (non-terminal) node:** a node with at least one child: A, B, D, G.
- **Degree of a node:** the number of its children. deg(A) = 3, deg(B) = 2, deg(C) = 0.
- **Degree of a tree:** the maximum degree of any node: 3.
- **Edge:** a link between a parent and a child.
- **Path:** a sequence of nodes connected by edges, such as A → D → G → H. Its **length** is the number of edges (3).
- **Ancestors** of a node: all nodes on the path from the root to it. Ancestors of H: G, D, A. **Descendants** of D: G, H, I.
- **Level:** the root is at level 0, its children at level 1, and so on. (Some books start at level 1.)
- **Depth of a node:** the length of the path from the root to it. Depth of G = 2.
- **Height of a node:** the length of the longest path from it down to a leaf. **Height of the tree** = height of the root = 3.
- **Forest:** a set of disjoint trees. Removing the root of a tree gives a forest.

Que 4.2 **What is a binary tree? Explain strictly binary, complete, full and extended binary trees. State the important properties of binary trees.**

ANSWER

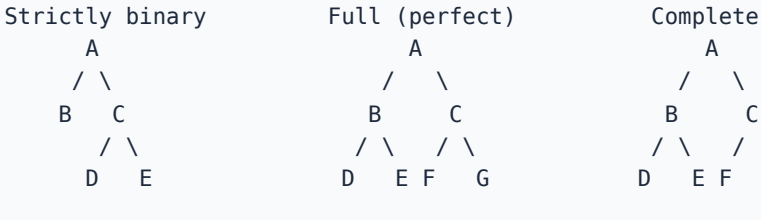
A **binary tree** is a finite set of nodes that is either empty, or consists of a root and two disjoint binary trees called the **left subtree** and the **right subtree**. Each node therefore has at most two children, and the left and right children are distinguished: a tree with only a left child is different from one with only a right child.

Types of binary trees

1. Strictly binary tree (proper or 2-tree): every node has either 0 or 2 children. A strictly binary tree with n leaves has exactly $2n - 1$ nodes.

2. Full (perfect) binary tree: every internal node has two children and all leaves are at the same level. A full binary tree of height h has $2^{h+1} - 1$ nodes.

3. Complete binary tree: every level is completely filled except possibly the last, and the nodes of the last level are as far **left** as possible. Heaps are complete binary trees. A complete binary tree with n nodes has height $\lfloor \log_2 n \rfloor$.



4. Skewed binary tree: every node has only a left child (left-skewed) or only a right child (right-skewed). It behaves like a linked list.

5. Extended binary tree (2-tree): a binary tree in which every empty subtree is replaced by a special **external node** (drawn as a square). The original nodes are called **internal nodes**. Every internal node then has exactly two children.



If there are n internal nodes, there are **n + 1 external nodes**. If I is the **internal path length** (sum of the depths of the internal nodes) and E is the **external path length**, then **E = I + 2n**. In the example, n = 4, I = 0 + 1 + 1 + 2 = 4 and E = 2 + 2 + 2 + 3 + 3 = 12 = 4 + 8. Extended binary trees are used in Huffman coding and in analysing search algorithms.

Properties of binary trees

1. The maximum number of nodes at level l is $\mathbf{2^l}$ (root at level 0).
2. The maximum number of nodes in a binary tree of height h is $\mathbf{2^{h+1} - 1}$.
3. A binary tree with n nodes has height at least $\mathbf{\lceil \log_2(n + 1) \rceil - 1}$, and at most n − 1.
4. In any non-empty binary tree, if n_0 is the number of leaves and n_2 the number of nodes of degree 2, then $\mathbf{n_0 = n_2 + 1}$.

Proof of property 4: let $n = n_0 + n_1 + n_2$ be the total number of nodes. Every node except the root has exactly one incoming edge, so the number of edges is n − 1. Each node of degree 1 gives one edge and each node of degree 2 gives two, so the number of edges is also $n_1 + 2n_2$. Hence $n_0 + n_1 + n_2 - 1 = n_1 + 2n_2$, which gives $n_0 = n_2 + 1$.

Que 4.3 **Explain the array and linked list representations of a binary tree.**

ANSWER

1. Array (sequential) representation

The nodes are stored level by level, left to right, in an array. With 0-based indexing, for a node at index i:

- left child at **2i + 1**, right child at **2i + 2**,
- parent at $\mathbf{\lfloor (i - 1)/2 \rfloor}$.

(With 1-based indexing: left 2i, right 2i + 1, parent $\lfloor i/2 \rfloor$.)

NEXT ENGINEER _sheerat_

```
          A              index: 0  1  2  3  4  5  6
        /   \            value: A  B  C  D  E  -  F
       B     C
      / \     \
     D   E     F
```

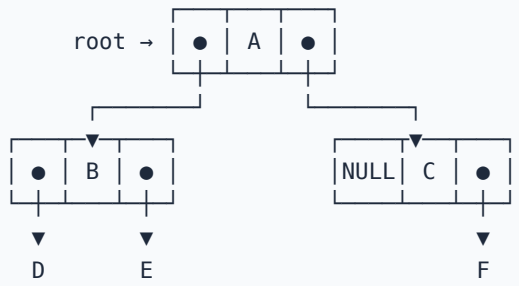
Empty positions (index 5 here) are left blank.

- **Advantages:** no pointers; parent and children found by arithmetic; ideal for complete binary trees and heaps.
- **Disadvantages:** wastes a lot of space for sparse or skewed trees (a right-skewed tree of height h needs $2^{h+1} - 1$ slots for h + 1 nodes), and the size is fixed.

2. Linked representation

Each node is a structure with a data field and two pointers:

```
struct tnode {
    struct tnode *left;
    int data;
    struct tnode *right;
};
```



- **Advantages:** no wasted space for missing nodes, dynamic size, easy insertion and deletion.
- **Disadvantages:** extra memory for pointers; there is no direct way to reach the parent (a third `parent` pointer can be added); in a tree with n nodes, n + 1 of the 2n pointer fields are NULL (this waste is used by threaded trees).

Part 2: Tree Traversals

Que 4.4 **Explain the inorder, preorder and postorder traversals of a binary tree with recursive algorithms and an example.**

ANSWER

Traversal means visiting every node of the tree exactly once in a fixed order. The three depth-first orders differ only in **when the root is visited** relative to its subtrees:

Traversal	Order	Memory aid
Preorder	Root, Left, Right	NLR
Inorder	Left, Root, Right	LNR
Postorder	Left, Right, Root	LRN

```
void preorder(struct tnode *t) {
    if (t == NULL) return;
    printf("%d ", t->data);   /* N */
    preorder(t->left);        /* L */
    preorder(t->right);       /* R */
}

void inorder(struct tnode *t) {
    if (t == NULL) return;
    inorder(t->left);
    printf("%d ", t->data);
    inorder(t->right);
}

void postorder(struct tnode *t) {
    if (t == NULL) return;
    postorder(t->left);
    postorder(t->right);
    printf("%d ", t->data);
}
```

Example



- **Preorder:** A B D E G C F H
- **Inorder:** D B G E A C H F
- **Postorder:** D G E B H F C A

Level-order traversal (breadth-first): visit nodes level by level using a queue: A B C D E F G H.

```
void level_order(struct tnode *root) {
    struct tnode *q[100]; int f = 0, r = 0;
    if (root) q[r++] = root;
    while (f < r) {
        struct tnode *t = q[f++];
        printf("%d ", t->data);
        if (t->left)  q[r++] = t->left;
        if (t->right) q[r++] = t->right;
    }
}
```

Expression trees: for the expression tree of (a + b) * (c – d), preorder gives the prefix form `* + a b - c d`, inorder gives the infix form `a + b * c - d` (brackets must be added), and postorder gives the postfix form `a b + c d - *`.

Complexity: each node is visited once, so every traversal takes **O(n)** time. The recursive versions use O(h) stack space, where h is the height.

Que 4.5 **Write a non-recursive (iterative) algorithm for inorder traversal using a stack.**

ANSWER

The recursion is removed by keeping the path in an explicit stack.

1. Set current = root.
2. While current is not NULL or the stack is not empty: - While current is not NULL: push current and go to its left child. - Pop a node, visit it, and set current to its right child.

```
void inorder_iter(struct tnode *root) {
    struct tnode *st[100]; int top = -1;
    struct tnode *cur = root;
    while (cur != NULL || top != -1) {
        while (cur != NULL) { st[++top] = cur; cur = cur->left; }
        cur = st[top--];
        printf("%d ", cur->data);
        cur = cur->right;
    }
}
```

Iterative preorder: push the root; repeatedly pop a node, visit it, push its right child and then its left child (so the left is processed first).

Que 4.6 **How is a binary tree constructed from its traversals? Construct the tree whose inorder is D B G E A C H F and preorder is A B D E G C F H.**

ANSWER

A binary tree can be uniquely constructed from **inorder together with preorder**, or **inorder together with postorder**. Preorder and postorder alone are not enough (except for strictly binary trees), because they do not tell left from right.

Method (inorder + preorder)

1. The **first** element of the preorder is the root.
2. Find the root in the inorder. Elements to its left form the left subtree and elements to its right form the right subtree.
3. Take the next elements of the preorder for the left subtree (as many as its size) and then the right subtree, and repeat recursively.

(With postorder, the **last** element is the root, and we build the right subtree before the left when reading backwards.)

Solution

- Preorder starts with **A** → root A. Inorder: [D B G E] **A** [C H F].
- Left subtree: inorder D B G E, preorder B D E G. Root **B**. Inorder: [D] **B** [G E].
- Left of B: **D**.
- Right of B: inorder G E, preorder E G. Root **E**; G is to its left in inorder, so **G** is the left child of E.
- Right subtree: inorder C H F, preorder C F H. Root **C**. Inorder: (nothing) **C** [H F], so C has no left child.
- Right of C: inorder H F, preorder F H. Root **F**; H is left of F in inorder, so **H** is the left child of F.



Check: the postorder of this tree is D G E B H F C A. If a question gives inorder D B G E A C H F and postorder D G E B H F C A, the same tree results: the last element A is the root, and so on.

C function (inorder + preorder)

```
int pre_idx = 0;

int find(char in[], int lo, int hi, char x) {
    for (int i = lo; i <= hi; i++) if (in[i] == x) return i;
    return -1;
}

struct tnode *build(char in[], char pre[], int lo, int hi) {
    if (lo > hi) return NULL;
    struct tnode *t = malloc(sizeof(struct tnode));
    t->data = pre[pre_idx++];
    t->left = t->right = NULL;
    if (lo == hi) return t;
    int k = find(in, lo, hi, t->data);
    t->left  = build(in, pre, lo, k - 1);
    t->right = build(in, pre, k + 1, hi);
    return t;
}
```

Part 3: Binary Search Trees

Que 4.7 **What is a binary search tree? Write algorithms for searching and inserting in a BST. Construct a BST for 50, 30, 70, 20, 40, 60, 80, 35, 45, 65.**

ANSWER

A **binary search tree (BST)** is a binary tree in which, for every node X:

1. all keys in the left subtree of X are **less than** the key of X,
2. all keys in the right subtree of X are **greater than** the key of X, and
3. the left and right subtrees are also BSTs.

(Duplicate keys are usually not allowed, or are consistently placed on one side.) **The inorder traversal of a BST gives the keys in sorted order.**

Searching: start at the root. If the key equals the root's key, stop. If it is smaller, search the left subtree; if larger, the right subtree.

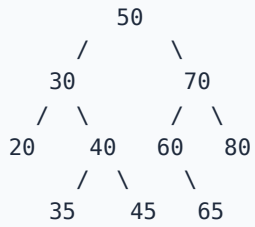
```
struct tnode *bst_search(struct tnode *t, int key) {
    while (t != NULL && t->data != key)
        t = (key < t->data) ? t->left : t->right;
    return t;                       /* NULL if not found */
}
```

Insertion: search for the key; when we reach a NULL link, put the new node there. A new key always becomes a leaf.

```
struct tnode *bst_insert(struct tnode *t, int key) {
    if (t == NULL) {
        t = malloc(sizeof(struct tnode));
        t->data = key;
        t->left = t->right = NULL;
    } else if (key < t->data)
        t->left = bst_insert(t->left, key);
    else if (key > t->data)
        t->right = bst_insert(t->right, key);
    return t;                       /* duplicates ignored */
}
```

Construction for 50, 30, 70, 20, 40, 60, 80, 35, 45, 65

- 50 becomes the root.
- 30 < 50 → left of 50. 70 > 50 → right of 50.
- 20 < 50, < 30 → left of 30. 40 < 50, > 30 → right of 30.
- 60 > 50, < 70 → left of 70. 80 → right of 70.
- 35 < 50, > 30, < 40 → left of 40. 45 → right of 40.
- 65 > 50, < 70, > 60 → right of 60.



Inorder: 20 30 35 40 45 50 60 65 70 80 (sorted).

Complexity of search and insert: O(h), where h is the height. For a balanced tree h = O(log n); for a skewed tree (keys inserted in sorted order) h = n − 1, so the worst case is **O(n)**. This is why balanced trees such as AVL trees are needed.

Finding the minimum and maximum: the minimum is the leftmost node (keep going left); the maximum is the rightmost node.

Que 4.8 **Explain deletion in a binary search tree with all cases. Delete 20, 60 and 50 (in this order) from the BST of Que 4.7.**

ANSWER

To delete a key, first search for its node X. There are three cases:

Case 1: X is a leaf. Simply remove it: set the corresponding child pointer of its parent to NULL.

Case 2: X has one child. Replace X by its only child: link X's parent directly to X's child, then free X.

Case 3: X has two children. Find the **inorder successor** S of X (the smallest key in X's right subtree, found by going right once and then left as far as possible). Copy S's key into X, and then delete S from the right subtree. S has no left child, so its deletion is case 1 or case 2. (The **inorder predecessor**, the largest key in the left subtree, can be used instead.)

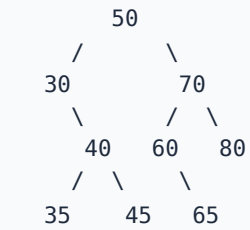
NEXT ENGINEER _sheerat_

```
struct tnode *find_min(struct tnode *t) {
    while (t->left != NULL) t = t->left;
    return t;
}

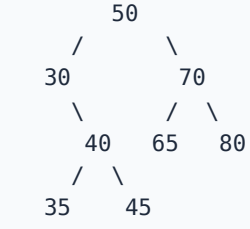
struct tnode *bst_delete(struct tnode *t, int key) {
    if (t == NULL) return NULL;                      /* not found */
    if (key < t->data) t->left = bst_delete(t->left, key);
    else if (key > t->data) t->right = bst_delete(t->right, key);
    else {
        if (t->left == NULL) {                       /* case 1 or 2 */
            struct tnode *r = t->right; free(t); return r;
        }
        if (t->right == NULL) {                      /* case 2 */
            struct tnode *l = t->left; free(t); return l;
        }
        struct tnode *s = find_min(t->right);        /* case 3 */
        t->data = s->data;
        t->right = bst_delete(t->right, s->data);
    }
    return t;
}
```

Example

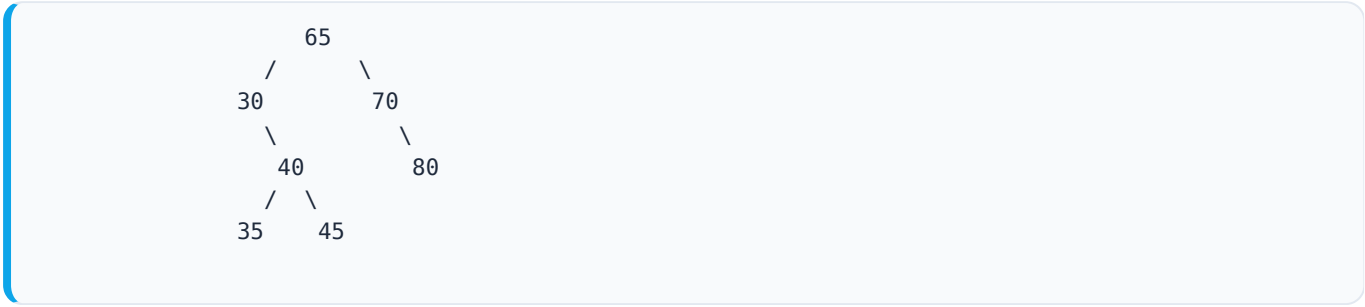
Delete 20 (leaf, case 1): remove it; 30's left becomes NULL.



Delete 60 (one child, case 2): 70's left pointer now points to 65.



Delete 50 (two children, case 3): the inorder successor of 50 is the smallest key in its right subtree: go right to 70, then left as far as possible, reaching **65**. Copy 65 into the root and delete the old 65, which is a leaf.



The inorder is now 30 35 40 45 65 70 80, still sorted, so the BST property holds. Using the inorder predecessor instead (45, the largest key in the left subtree) would also give a valid BST.

Modification of data in a BST: changing a key may break the BST property, so a key is modified by **deleting** the old key and **inserting** the new one. Non-key data in a node can be updated in place after a search.

Complexity: O(h): O(log n) for a balanced tree and O(n) in the worst case.

Part 4: Threaded Binary Trees

Que 4.9 **What is a threaded binary tree? Explain its types and representation. Write an algorithm for inorder traversal of a threaded binary tree.**

ANSWER

In a linked binary tree with n nodes, there are 2n pointer fields but only n − 1 are used for edges, so **n + 1 pointers are NULL**. A **threaded binary tree** uses these NULL pointers to store **threads**: pointers to the inorder predecessor or successor of the node. This was suggested by A. J. Perlis and C. Thornton.

- A NULL **right** pointer is replaced by a pointer to the node's **inorder successor**.
- A NULL **left** pointer is replaced by a pointer to the node's **inorder predecessor**.

To tell a thread from a real child pointer, each node has two Boolean flags:

```
struct thnode {
    int lthread;              /* 1 if left is a thread  */
    struct thnode *left;
    int data;
    struct thnode *right;
    int rthread;              /* 1 if right is a thread */
};
```

Types

1. **One-way (right) threaded:** only the NULL right pointers are threads (to the inorder successor).
2. **Two-way (fully) threaded:** both NULL left and NULL right pointers are threads.

The left thread of the first inorder node and the right thread of the last inorder node have nowhere to point, so a **header (dummy) node** is usually added; these two threads point to the header, and the whole tree is the left subtree of the header.

Example: for the tree below, inorder is D B E A C.

```
              A
            /   \
           B     C
          / \
         D   E

  Threads (dashed):  D.left → header   D.right → B
                     E.left → B        E.right → A
                     C.left → A        C.right → header
```

Inorder traversal without a stack

The inorder successor of a node p is found as follows:

- If p's right is a thread, the successor is p→right.
- Otherwise go to p→right and then follow left pointers (real ones) as far as possible.

```
struct thnode *leftmost(struct thnode *p) {
    while (p != NULL && p->lthread == 0 && p->left != NULL) p = p->left;
    return p;
}

void inorder_threaded(struct thnode *root) {        /* right threads; last node's right = NULL */
    struct thnode *p = leftmost(root);
    while (p != NULL) {
        printf("%d ", p->data);
        if (p->rthread) p = p->right;               /* follow the thread       */
        else p = leftmost(p->right);                /* go to the right subtree */
    }
}
```

Advantages

- Inorder traversal needs **no stack and no recursion**, so it is faster and uses O(1) extra space.
- The inorder successor and predecessor of any node can be found directly.
- The NULL pointers are put to use.

Disadvantages

- Insertion and deletion are more complicated, because threads must be updated.
- Each node needs two extra flag bits.

Part 5: Huffman Coding

Que 4.10 **What is Huffman coding? Explain the algorithm and find the Huffman codes for the characters A, B, C, D, E, F with frequencies 8, 3, 12, 5, 20, 2.**

ANSWER

Huffman coding is a lossless data compression technique, devised by David Huffman in 1952. It assigns **variable-length binary codes** to characters: frequent characters get short codes and rare

characters get long codes. The codes are **prefix-free**: no code is a prefix of another, so an encoded bit string can be decoded without separators. The codes are found from a binary tree (an extended binary tree with minimum weighted external path length).

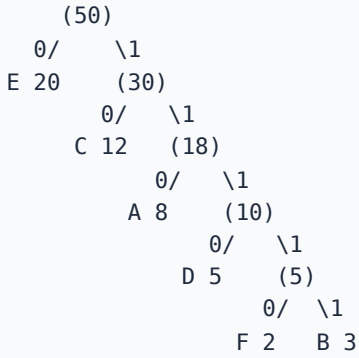
Algorithm

1. Create a leaf node for each character, with its frequency as weight. Put all nodes in a min priority queue.
2. Repeat until only one node remains: - Remove the two nodes with the smallest weights, x and y. - Create a new internal node z with weight w(x) + w(y), with x as the left child and y as the right child. - Insert z into the priority queue.
3. The remaining node is the root. Label every left edge 0 and every right edge 1. The code of a character is the sequence of labels on the path from the root to its leaf.

Solution

Sorted by frequency: F 2, B 3, D 5, A 8, C 12, E 20 (total 50).

Step	Two smallest	New node	Queue after the step
1	F 2, B 3	N1 = 5	D 5, N1 5, A 8, C 12, E 20
2	D 5, N1 5	N2 = 10	A 8, N2 10, C 12, E 20
3	A 8, N2 10	N3 = 18	C 12, N3 18, E 20
4	C 12, N3 18	N4 = 30	E 20, N4 30
5	E 20, N4 30	Root = 50	Root 50



Codes

Char	Freq	Code	Bits	Freq × bits
E	20	0	1	20
C	12	10	2	24
A	8	110	3	24
D	5	1110	4	20
F	2	11110	5	10

Char	Freq	Code	Bits	Freq × bits
B	3	11111	5	15
	50			**113**

- Total bits with Huffman coding: **113**. Average code length = 113/50 = **2.26 bits** per character.
- With a fixed-length code, 6 characters need 3 bits each: 50 × 3 = **150 bits**.
- Saving = (150 − 113)/150 ≈ **24.7%**.

Decoding: read bits from the left and walk down the tree from the root (0 = left, 1 = right); on reaching a leaf output its character and restart at the root. For example, 110 0 10 decodes to A E C.

(When two weights are equal, either can be chosen first; the codes may differ but the total length stays the same.)

Complexity: with a binary heap as the priority queue, there are n − 1 merges, each O(log n), so building the tree takes **O(n log n)**.

Part 6: AVL Trees

Que 4.11 **What is an AVL tree? Explain balance factor and the four rotations used to balance it.**

ANSWER

An **AVL tree** (named after its inventors Adelson-Velsky and Landis, 1962) is a **height-balanced binary search tree**: for every node, the heights of its left and right subtrees differ by at most 1.

Balance factor of a node:

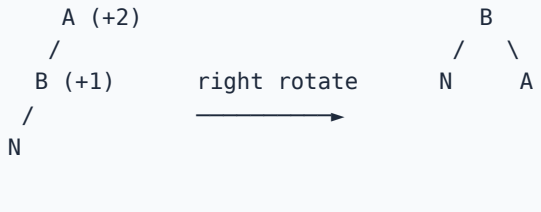
BF = height(left subtree) – height(right subtree)

In an AVL tree every node has BF ∈ {−1, 0, +1}. If an insertion or deletion makes some node's BF equal to +2 or −2, the tree is rebalanced by **rotations**. Because the height of an AVL tree with n nodes is always O(log n) (at most about 1.44 $\log_2 n$), search, insertion and deletion all take **O(log n)** time in the worst case.

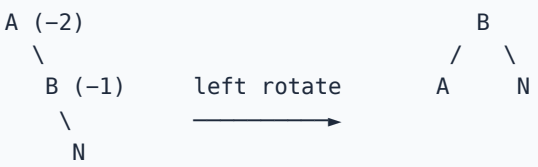
The four rotations

Let A be the nearest ancestor of the new node whose BF has become ±2.

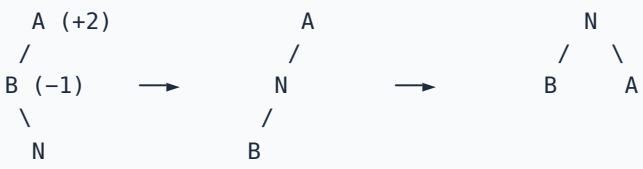
1. LL rotation (single right rotation): the new node is in the **left subtree of the left child** of A.



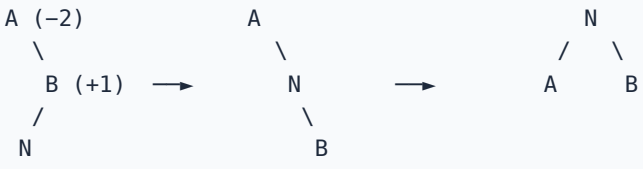
2. RR rotation (single left rotation): the new node is in the **right subtree of the right child** of A.



3. LR rotation (double rotation): the new node is in the **right subtree of the left child** of A. First rotate left at B, then rotate right at A.



4. RL rotation (double rotation): the new node is in the **left subtree of the right child** of A. First rotate right at B, then rotate left at A.



In every rotation the subtrees hanging off the moved nodes are re-attached so that the BST order is kept. The middle key of the three nodes always becomes the new root of the subtree.

NEXT ENGINEER _sheerat_

```c
struct avl { int key, height; struct avl *left, *right; };

int h(struct avl *t) { return t ? t->height : -1; }
int max(int a, int b) { return a > b ? a : b; }
void update(struct avl *t) { t->height = 1 + max(h(t->left), h(t->right)); }

struct avl *rotate_right(struct avl *a) {        /* LL case */
    struct avl *b = a->left;
    a->left = b->right;
    b->right = a;
    update(a); update(b);
    return b;
}

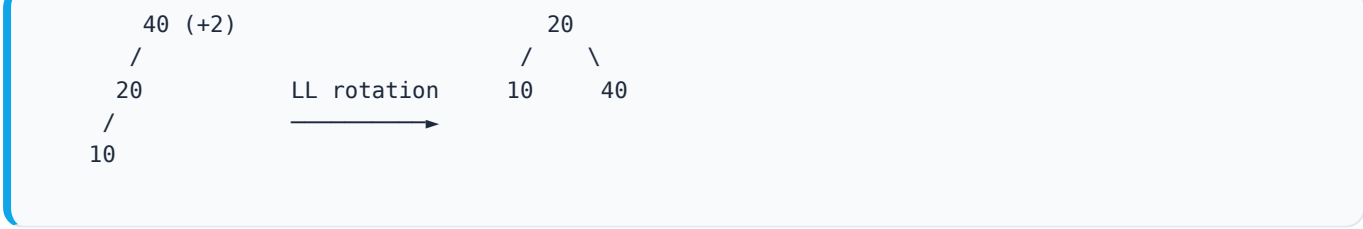
struct avl *rotate_left(struct avl *a) {         /* RR case */
    struct avl *b = a->right;
    a->right = b->left;
    b->left = a;
    update(a); update(b);
    return b;
}

struct avl *avl_insert(struct avl *t, int key) {
    if (t == NULL) {
        t = malloc(sizeof(struct avl));
        t->key = key; t->height = 0; t->left = t->right = NULL;
        return t;
    }
    if (key < t->key) t->left = avl_insert(t->left, key);
    else if (key > t->key) t->right = avl_insert(t->right, key);
    else return t;
    update(t);
    int bf = h(t->left) - h(t->right);
    if (bf > 1 && key < t->left->key) return rotate_right(t);             /* LL */
    if (bf < -1 && key > t->right->key) return rotate_left(t);            /* RR */
    if (bf > 1)  { t->left = rotate_left(t->left);   return rotate_right(t); } /* LR */
    if (bf < -1) { t->right = rotate_right(t->right); return rotate_left(t); } /* RL */
    return t;
}
```

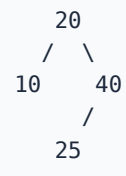
Que 4.12 **Construct an AVL tree by inserting 40, 20, 10, 25, 30, 22, 50 in this order.**

ANSWER

Insert 40, 20, 10. After inserting 10, node 40 has BF = +2 and the new node is in the left subtree of its left child: **LL case**. Rotate right at 40.



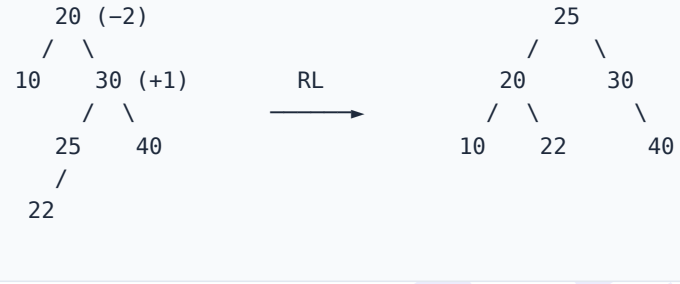
Insert 25. 25 > 20, 25 < 40 → left child of 40. All balance factors are within limits (20: −1, 40: +1).



Insert 30. 30 > 20, < 40, > 25 → right child of 25. Now 40 has BF = +2 and the new node is in the right subtree of its left child: **LR case**. Rotate left at 25, then right at 40. 30 becomes the root of this subtree.

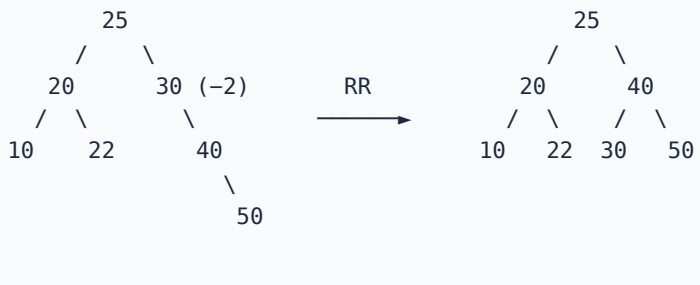


Insert 22. 22 > 20, < 30, < 25 → left child of 25. Now node 20 has left height 0 and right height 2, so BF = −2. The path from 20 goes right (30) and then left (25): **RL case**. Rotate right at 30, then left at 20. Node 25 becomes the root.



(During the rotation, 25's left child 22 becomes the right child of 20, and 25's right subtree, which is empty, becomes the left child of 30.)

Insert 50. 50 > 25, > 30, > 40 → right child of 40. Node 30 now has BF = −2, with the new node in the right subtree of its right child: **RR case**. Rotate left at 30.



Final AVL tree (all balance factors 0): root 25; left subtree 20 (10, 22); right subtree 40 (30, 50). Its inorder is 10 20 22 25 30 40 50, which confirms it is still a BST.

Deletion in an AVL tree: delete as in a BST, then walk back up towards the root, updating heights. At every node whose BF becomes ±2, apply the rotation that matches the heavier side (look at the BF of

the child on the heavy side: same sign or 0 → single rotation, opposite sign → double rotation). Unlike insertion, deletion may need rotations at several levels, but the total is still O(log n).

Part 7: B-Trees

Que 4.13 **What is a B-tree? State its properties. Why are B-trees used?**

ANSWER

A **B-tree of order m** is a balanced **m-way search tree** used mainly for data stored on disk. It was introduced by Bayer and McCreight in 1972. Its properties are:

1. Every node has **at most m children** and therefore at most **m − 1 keys**.
2. Every node except the root has **at least ⌈m/2⌉ children**, that is at least **⌈m/2⌉ − 1 keys**.
3. The root has at least 2 children unless it is a leaf.
4. **All leaves are at the same level.**
5. A node with k children has exactly k − 1 keys, kept in sorted order. If its keys are $K_1 < K_2 < \ldots < K_{k-1}$, then all keys in child C_i lie between K_{i-1} and K_i (like a BST, but with many keys per node).

For **order 5**: maximum 4 keys and 5 children per node; minimum 2 keys (3 children) for non-root nodes.

Why B-trees are used

- Reading a disk block is thousands of times slower than reading memory. A B-tree node is made the size of one disk block and holds many keys, so the tree is very **short and wide**. With m = 100 and 10 million keys, the height is only about 4, so a search needs about 4 disk reads.
- The tree always stays balanced, so search, insertion and deletion are **O(log n)** (more precisely O(log_m n) disk accesses).
- They are used in database indexes and file systems (the B+ tree variant, in which all data is in the leaves and leaves are linked, is the most common).

Searching: in a node, search the keys (linearly or by binary search). If the key is found, stop; otherwise go down into the child between the two keys that surround it.

Que 4.14 **Explain insertion in a B-tree. Construct a B-tree of order 5 by inserting 10, 20, 30, 40, 50, 60, 70, 80, 90, 25, 35, 5.**

ANSWER

Insertion algorithm

1. Search for the key to find the **leaf** where it belongs. Insert the key into the leaf in sorted order.
2. If the leaf now has m keys (overflow), **split** it: the **median** key moves up into the parent, the keys to its left form one node and the keys to its right form another.
3. If the parent overflows as a result, split it in the same way. If the root splits, a new root is created and the height grows by one. (A B-tree grows upwards, at the root.)

Order 5 → a node overflows when it gets 5 keys.

NEXT ENGINEER _sheerat_

Insert 10, 20, 30, 40: all go into the root, which is a leaf: [10 20 30 40].

Insert 50: [10 20 30 40 50] overflows. The median 30 moves up to a new root.

```
          [30]
         /    \
[10 20]       [40 50]
```

Insert 60, 70: go to the right leaf → [40 50 60 70].

Insert 80: [40 50 60 70 80] overflows; the median 60 moves up into the root.

```
           [30 60]
         /    |    \
[10 20]  [40 50]  [70 80]
```

Insert 90: → [70 80 90]. **Insert 25:** 25 < 30 → [10 20 25]. **Insert 35:** 30 < 35 < 60 → [35 40 50]. **Insert 5:** → [5 10 20 25].

Final B-tree

```
                  [30 60]
          /          |          \
[5 10 20 25]    [35 40 50]    [70 80 90]
```

Que 4.15 **Explain deletion in a B-tree with examples. Delete 90, 80, 40 and 70 from the B-tree of Que 4.14.**

ANSWER

Deletion algorithm

1. If the key is in a **leaf**, remove it.
2. If the key is in an **internal node**, replace it with its inorder predecessor (largest key in the left child's subtree) or inorder successor (smallest key in the right child's subtree), which is always in a leaf, and delete that key from the leaf.
3. If a leaf now has fewer than the minimum number of keys (**underflow**; for order 5, fewer than 2): - **Borrow (redistribute):** if an adjacent sibling has more than the minimum, move the separating key from the parent down into the deficient node and move the nearest key of the sibling up into the parent. - **Merge:** if both adjacent siblings have only the minimum, merge the deficient node, the separating key from the parent and a sibling into one node. The parent loses a key; if the parent underflows, fix it the same way. If the root becomes empty, its only child becomes the new root and the height falls by one.

Starting tree: root [30 60]; children [5 10 20 25], [35 40 50], [70 80 90].

Delete 90: leaf [70 80 90] → [70 80], which still has 2 keys. No underflow.

Delete 80: [70 80] → [70], underflow. The left sibling [35 40 50] has 3 keys (more than 2), so **borrow**: the separator 60 moves down into the deficient node and 50 moves up into the root.

```
                  [30 50]
          /          |          \
[5 10 20 25]      [35 40]      [60 70]
```

Delete 40: [35 40] → [35], underflow. The left sibling [5 10 20 25] has 4 keys, so borrow: the separator 30 moves down and 25 moves up.

```
               [25 50]
        /        |        \
[5 10 20]    [30 35]    [60 70]
```

Delete 70: [60 70] → [60], underflow. Its only adjacent sibling [30 35] has the minimum 2 keys, so **merge**: [30 35] + separator 50 + [60] = [30 35 50 60]. The root loses 50.

```
            [25]
          /      \
[5 10 20]    [30 35 50 60]
```

The root has 1 key, which is allowed for the root, so the deletion is complete.

Part 8: Binary Heaps and Conversion of General Trees

Que 4.16 **Explain insertion and deletion in a binary heap. Insert 20, 15, 30, 5, 40 into a max heap and then delete the maximum.**

ANSWER

A **binary heap** is a complete binary tree stored in an array, with the heap property (max heap: parent ≥ children). It is the standard implementation of a **priority queue** (Unit 2) and the basis of **heap sort** (Unit 3).

Insertion (sift up / heapify up)

1. Put the new key at the end of the array (the next free position of the last level), so the tree stays complete.
2. While the key is greater than its parent, swap it with the parent.

Deletion of the maximum (sift down / heapify down)

1. The maximum is at the root. Remove it.
2. Move the last element to the root and reduce the size by one.
3. While this element is smaller than the larger of its children, swap it with that child.

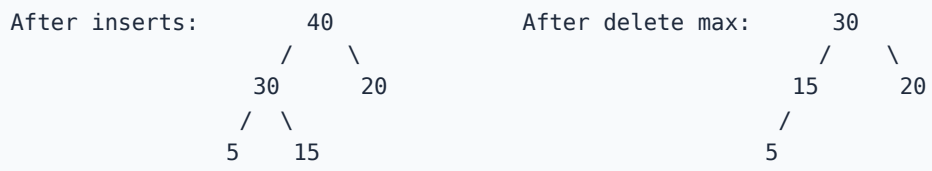
```c
int heap[100], size = 0;

void heap_insert(int x) {
    int i = size++;
    heap[i] = x;
    while (i > 0 && heap[(i - 1) / 2] < heap[i]) {
        int p = (i - 1) / 2;
        int t = heap[p]; heap[p] = heap[i]; heap[i] = t;
        i = p;
    }
}

int heap_delete_max(void) {
    int max = heap[0];
    heap[0] = heap[--size];
    int i = 0;
    while (1) {
        int l = 2 * i + 1, r = 2 * i + 2, big = i;
        if (l < size && heap[l] > heap[big]) big = l;
        if (r < size && heap[r] > heap[big]) big = r;
        if (big == i) break;
        int t = heap[i]; heap[i] = heap[big]; heap[big] = t;
        i = big;
    }
    return max;
}
```

Example

Operation	Steps	Heap array
insert 20	first element	20
insert 15	15 < 20, stays	20 15
insert 30	30 > parent 20, swap	30 15 20
insert 5	5 < parent 15, stays	30 15 20 5
insert 40	40 > parent 15, swap; 40 > parent 30, swap	40 30 20 5 15
delete max	remove 40, move 15 to root; 15 < 30, swap; 15 > child 5, stop	30 15 20 5



Complexity: the height of a heap with n nodes is $\lfloor \log_2 n \rfloor$, so insertion and deletion take **O(log n)**; finding the maximum takes O(1). A heap can be built from n given elements in **O(n)** with bottom-up heapify (Unit 3, Que 3.15).

Que 4.17 How is a general tree converted into a binary tree?

ANSWER

A general tree may have any number of children per node, which is awkward to store. It is converted to a binary tree using the **left child, right sibling** rule:

1. The **left** pointer of each node points to its **first (leftmost) child**.
2. The **right** pointer of each node points to its **next sibling** to the right.



The root of the resulting binary tree never has a right child. The preorder of the binary tree equals the preorder of the general tree, and its inorder equals the postorder of the general tree. A **forest** can be converted the same way by treating the roots of the trees as siblings.

NEXT ENGINEER _sheerat_

Short-Answer Questions (2 marks)

Q1. What is the maximum number of nodes in a binary tree of height h?

$2^{h+1} - 1$ (with the root at height 0).

Q2. If a binary tree has 15 leaves, how many nodes of degree 2 does it have?

$n_2 = n_o - 1 = 14$.

Q3. Differentiate between a complete and a full binary tree.

In a full binary tree all levels are completely filled. In a complete binary tree all levels except possibly the last are filled, and the last level is filled from the left.

Q4. What is an extended binary tree?

A binary tree in which every empty subtree is replaced by an external node, so every original node has two children. n internal nodes give n + 1 external nodes.

Q5. Which traversal of a BST gives sorted output?

Inorder traversal.

Q6. Can a binary tree be built uniquely from preorder and postorder?

Not in general, because they do not show which child is left and which is right. Inorder with either of them is needed.

Q7. What is a threaded binary tree?

A binary tree in which NULL left and right pointers are replaced by pointers (threads) to the inorder predecessor and successor, allowing traversal without a stack.

Q8. Define the balance factor of an AVL tree node.

BF = height of left subtree − height of right subtree; it must be −1, 0 or +1.

Q9. What is the minimum number of keys in a non-root node of a B-tree of order 7?

$\lceil 7/2 \rceil - 1 = 3$ keys.

Q10. What is a prefix code in Huffman coding?

A code in which no codeword is a prefix of another, so encoded text can be decoded uniquely without separators.

Q11. What is the worst case time to search a BST? When does it occur?

O(n), when the tree is skewed, for example when keys are inserted in sorted order.

Q12. Give the array index of the parent and children of node i in a heap (0-based).

Parent $\lfloor (i - 1)/2 \rfloor$, left child 2i + 1, right child 2i + 2.

NEXT ENGINEER _sheerat_

Practice Questions

1. Construct the binary tree whose postorder is H I D E B F G C A and inorder is H D I B E A F C G. Write its preorder.
2. Insert 45, 32, 67, 12, 38, 54, 89, 36, 40, 60 into an empty BST. Then delete 32 and 67, drawing the tree after each deletion.
3. Construct an AVL tree by inserting the months JAN, FEB, MAR, APR, MAY, JUN, JUL, AUG in this order (comparing names alphabetically), naming each rotation.
4. Construct a B-tree of order 3 by inserting 1 to 10 in order.
5. Find the Huffman codes for a:45, b:13, c:12, d:16, e:9, f:5 and the average code length.
6. Draw the fully threaded binary tree for the tree of Que 4.4 and trace its inorder traversal.
7. Prove that a strictly binary tree with n leaves has 2n − 1 nodes.
8. Write a C function to count the leaves and find the height of a binary tree.
9. Build a max heap from 12, 7, 25, 3, 18, 30, 9 using bottom-up heapify.
10. Convert a general tree with root R and children P (children X, Y, Z), Q and S (child T) into a binary tree.

NEXT ENGINEER
sheerat

UNIT 5

Graphs

Unit overview

A graph is the most general data structure: it models any set of objects with connections between them, such as cities and roads, computers in a network, web pages and links, or people in a social network. This unit covers how graphs are stored, how they are traversed, and three classic problems on them: minimum spanning trees, transitive closure and shortest paths.

Topics in this unit: graph terminology, representations (adjacency matrix, adjacency list, incidence matrix and adjacency multilist), depth first search, breadth first search, connected components, spanning trees, minimum cost spanning trees by Prim's and Kruskal's algorithms, transitive closure by Warshall's algorithm, and single-source shortest paths by Dijkstra's algorithm.

Most asked: BFS and DFS traces on a given graph, Prim's and Kruskal's algorithms with step tables, Dijkstra's algorithm step by step, Warshall's algorithm matrices, and adjacency matrix versus adjacency list.

Part 1: Graph Terminology and Representation

Que 5.1 **Define a graph. Explain the terminology used with graphs.**

ANSWER

A **graph** G = (V, E) consists of a finite non-empty set **V** of **vertices** (nodes) and a set **E** of **edges**, where each edge connects a pair of vertices.



Terminology

- **Undirected graph:** edges have no direction; edge (u, v) is the same as (v, u). G1 above.
- **Directed graph (digraph):** every edge has a direction, written ⟨u, v⟩, from the **tail** u to the **head** v. G2 above.
- **Weighted graph (network):** every edge carries a number (weight or cost), such as distance or time.

- **Adjacent vertices:** u and v are adjacent if there is an edge between them. The edge is **incident** on u and v.
- **Degree** of a vertex (undirected): the number of edges incident on it. In G1, deg(A) = 3.
- **In-degree and out-degree** (directed): the number of edges coming in and going out. In G2, in-degree(B) = 1, out-degree(B) = 1.
- **Handshaking lemma:** the sum of the degrees of all vertices is **2|E|**, because every edge adds 1 to the degree of each of its two ends. Hence the number of odd-degree vertices is always even.
- **Path:** a sequence of vertices $v_1, v_2, \ldots, v_k$ in which each consecutive pair is joined by an edge. Its length is the number of edges.
- **Simple path:** a path in which no vertex is repeated.
- **Cycle:** a path that starts and ends at the same vertex, with no other repetition, such as A → B → C → A in G1.
- **Acyclic graph:** a graph with no cycles. A directed acyclic graph is called a **DAG**.
- **Connected graph:** an undirected graph in which there is a path between every pair of vertices.
- **Strongly connected:** a directed graph in which there is a directed path from every vertex to every other vertex. G2 is strongly connected (A → B → C → D → A).
- **Weakly connected:** a directed graph that is connected if the directions are ignored.
- **Connected component:** a maximal connected subgraph of an undirected graph.
- **Complete graph:** every pair of distinct vertices is joined by an edge. A complete undirected graph on n vertices has **n(n − 1)/2** edges; a complete digraph has n(n − 1).
- **Subgraph:** G' = (V', E') with V' ⊆ V and E' ⊆ E.
- **Self-loop:** an edge from a vertex to itself. **Multigraph:** a graph with more than one edge between the same pair of vertices. A **simple graph** has neither.
- **Tree:** a connected acyclic undirected graph. A tree with n vertices has exactly n − 1 edges.
- **Sparse and dense graphs:** a graph with $|E|$ much smaller than $|V|^2$ is sparse; one with $|E|$ close to $|V|^2$ is dense.

Que 5.2 **Explain the adjacency matrix and adjacency list representations of a graph. Compare them.**

ANSWER

1. Adjacency matrix

For a graph with n vertices, the adjacency matrix is an n × n matrix A where

- A[i][j] = 1 if there is an edge from vertex i to vertex j, and 0 otherwise.
- For a weighted graph, A[i][j] = weight of the edge, and 0 or ∞ if there is no edge.

For an undirected graph the matrix is **symmetric**. The degree of vertex i is the sum of row i. For a digraph, the row sum is the out-degree and the column sum is the in-degree.

Using the undirected graph G1 (A–B, A–C, A–D, B–C, C–D):

	A	B	C	D
A	0	1	1	1
B	1	0	1	0
C	1	1	0	1
D	1	0	1	0

2. Adjacency list

An array of n lists, one per vertex. The list of vertex i contains all vertices adjacent to i (for a weighted graph, each list node also stores the weight).

```
A → B → C → D → NULL
B → A → C → NULL
C → A → B → D → NULL
D → A → C → NULL
```

```
struct adjnode { int vertex; int weight; struct adjnode *next; };
struct adjnode *adj[MAXV];          /* adj[i] is the list of vertex i */

void add_edge(int u, int v, int w) {   /* undirected: add both directions */
    struct adjnode *p = malloc(sizeof(struct adjnode));
    p->vertex = v; p->weight = w; p->next = adj[u]; adj[u] = p;
    p = malloc(sizeof(struct adjnode));
    p->vertex = u; p->weight = w; p->next = adj[v]; adj[v] = p;
}
```

In an undirected graph each edge appears twice, so the lists have 2|E| nodes in total; in a digraph they have |E| nodes.

Comparison

Basis	Adjacency matrix	Adjacency list
Space	$O(V^2)$	O(V + E)
Is (u, v) an edge?	O(1)	O(degree of u)
List all neighbours of u	O(V)	O(degree of u)
Add an edge	O(1)	O(1)
Add a vertex	$O(V^2)$ (rebuild)	O(1)
BFS/DFS time	$O(V^2)$	O(V + E)
Best for	Dense graphs, quick edge tests	Sparse graphs (most real graphs)

Que 5.3 **Explain the incidence matrix and adjacency multilist representations.**

ANSWER

1. Incidence matrix

For a graph with n vertices and e edges, the incidence matrix is an n × e matrix M where

- (undirected) M[i][j] = 1 if edge j is incident on vertex i, otherwise 0;
- (directed) M[i][j] = +1 if edge j leaves vertex i, −1 if it enters vertex i, and 0 otherwise.

Each column has exactly two non-zero entries. For G1 with edges e1 = AB, e2 = AC, e3 = AD, e4 = BC, e5 = CD:

	e1	e2	e3	e4	e5
A	1	1	1	0	0
B	1	0	0	1	0
C	0	1	0	1	1
D	0	0	1	0	1

It takes O(V × E) space and is used mainly in electrical network analysis.

2. Adjacency multilist

In an adjacency list of an undirected graph, every edge is stored twice. In an **adjacency multilist** each edge is stored **once**, as a node shared by the lists of both its end vertices:

```
Edge node:  ┌──────┬────┬────┬──────────────┬──────────────┐
            │ mark │ v1 │ v2 │ next for v1  │ next for v2  │
            └──────┴────┴────┴──────────────┴──────────────┘
```

- `mark` records whether the edge has been processed (useful in algorithms that must look at each edge once).
- `next for v1` points to the next edge incident on v1, and `next for v2` to the next edge incident on v2.
- Each vertex has a header pointer to its first edge.

This saves memory and makes operations like marking or deleting an edge easy, because there is only one copy of each edge.

Part 2: Graph Traversal

Que 5.4 **Explain breadth first search (BFS) with algorithm, an example and complexity.**

ANSWER

Breadth first search visits the vertices of a graph level by level. Starting from a source vertex s, it first visits all neighbours of s, then all unvisited neighbours of those vertices, and so on. It uses a **queue** and a `visited` array so that no vertex is visited twice.

Algorithm BFS(G, s)

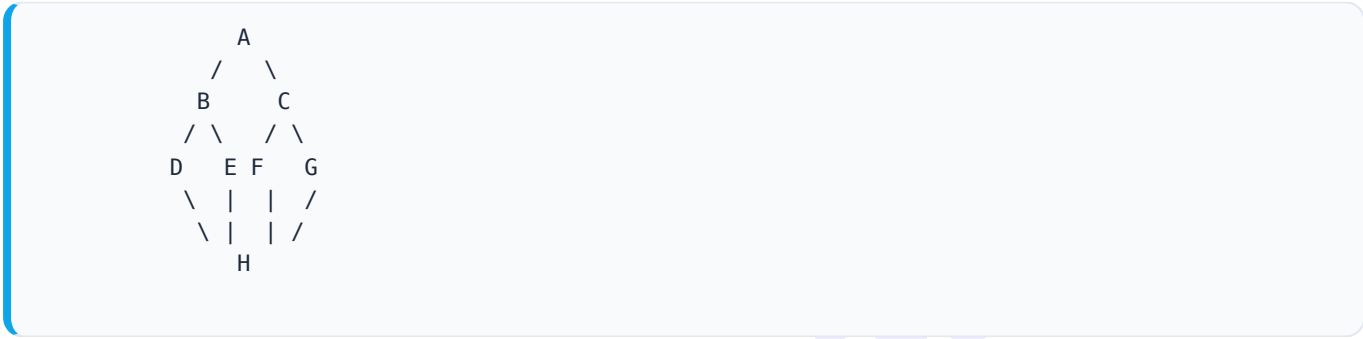
1. Mark all vertices unvisited. Mark s visited and insert s into the queue.

2. While the queue is not empty: - Delete a vertex u from the queue and process (print) it. - For each vertex v adjacent to u: if v is unvisited, mark it visited and insert it into the queue.

```
int visited[MAXV];

void bfs(int s, int n, int a[][MAXV]) {         /* adjacency matrix version */
    int q[MAXV], f = 0, r = 0;
    for (int i = 0; i < n; i++) visited[i] = 0;
    visited[s] = 1; q[r++] = s;
    while (f < r) {
        int u = q[f++];
        printf("%c ", 'A' + u);
        for (int v = 0; v < n; v++)
            if (a[u][v] && !visited[v]) { visited[v] = 1; q[r++] = v; }
    }
}
```

Example graph (used for both BFS and DFS):



Adjacency (alphabetical order): A: B, C; B: A, D, E; C: A, F, G; D: B, H; E: B, H; F: C, H; G: C, H; H: D, E, F, G.

BFS from A

Step	Vertex removed	Unvisited neighbours added	Queue after the step
0			A
1	A	B, C	B C
2	B	D, E	C D E
3	C	F, G	D E F G
4	D	H	E F G H
5	E	(H already visited)	F G H
6	F		G H
7	G		H
8	H		(empty)

BFS order: A B C D E F G H

The edges used to discover new vertices (A–B, A–C, B–D, B–E, C–F, C–G, D–H) form the **BFS spanning tree**.

Complexity: each vertex is inserted into and deleted from the queue once, and each adjacency list is scanned once. With adjacency lists the time is **O(V + E)**; with an adjacency matrix it is **O(V^2)**. Space O(V) for the queue and visited array.

Applications of BFS

- Shortest path (fewest edges) in an **unweighted** graph.
- Finding connected components.
- Testing whether a graph is bipartite.
- Web crawlers, finding people within k connections in a social network, broadcasting in networks.

Que 5.5 **Explain depth first search (DFS) with algorithm, an example and complexity. Compare BFS and DFS.**

ANSWER

Depth first search goes as deep as possible along one path before backtracking. From the current vertex it moves to an unvisited neighbour, then to an unvisited neighbour of that vertex, and so on; when a vertex has no unvisited neighbours, it backtracks to the previous vertex. It uses a **stack**, either the system stack through recursion or an explicit one.

Recursive algorithm DFS(u)

1. Mark u visited and process (print) it.
2. For each vertex v adjacent to u: if v is unvisited, call DFS(v).

```
void dfs(int u, int n, int a[][MAXV]) {
    visited[u] = 1;
    printf("%c ", 'A' + u);
    for (int v = 0; v < n; v++)
        if (a[u][v] && !visited[v]) dfs(v, n, a);
}
```

Iterative version with a stack: push s; while the stack is not empty, pop u; if u is unvisited, mark and print it and push all its unvisited neighbours (in reverse order, if the same order as the recursive version is wanted).

DFS from A on the graph of Que 5.4 (neighbours in alphabetical order)

Step	Current vertex	Action	Recursion stack
1	A	visit A, go to B	A
2	B	visit B, A done, go to D	A B
3	D	visit D, go to H	A B D
4	H	visit H, D done, go to E	A B D H
5	E	visit E; B and H visited, backtrack	A B D H E

Step	Current vertex	Action	Recursion stack
6	H	next neighbour F	A B D H
7	F	visit F, go to C	A B D H F
8	C	visit C; A, F visited, go to G	A B D H F C
9	G	visit G; C, H visited, backtrack	A B D H F C G
10		backtrack to A; all done	(empty)

DFS order: A B D H E F C G

The tree edges A–B, B–D, D–H, H–E, H–F, F–C, C–G form the **DFS spanning tree**.

Complexity: O(V + E) with adjacency lists, **$O(V^2)$** with an adjacency matrix. Space O(V) for the stack and visited array.

Applications of DFS: detecting cycles, topological sorting of a DAG, finding connected and strongly connected components, solving mazes and puzzles, finding articulation points and bridges.

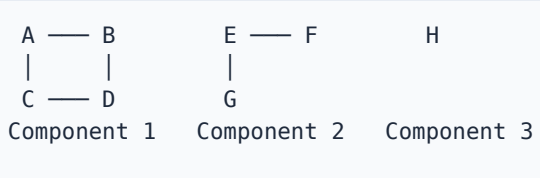
BFS vs DFS

Basis	BFS	DFS
Data structure	Queue	Stack (or recursion)
Order	Level by level	Deep along a path, then backtrack
Shortest path in unweighted graph	Yes	No
Memory	Can be large for wide graphs (whole level in queue)	Proportional to depth
Spanning tree produced	Short and wide	Long and narrow
Typical uses	Shortest paths, bipartite test, broadcasting	Cycle detection, topological sort, mazes
Time	O(V + E)	O(V + E)

Que 5.6 **What is a connected component? How are connected components found?**

ANSWER

A **connected component** of an undirected graph is a maximal set of vertices such that there is a path between every pair of them. A connected graph has exactly one component.



Algorithm: run BFS or DFS from any unvisited vertex; every vertex it reaches belongs to the same component. Repeat from the next unvisited vertex, increasing a counter each time.

```
int count_components(int n, int a[][MAXV]) {
    int c = 0;
    for (int i = 0; i < n; i++) visited[i] = 0;
    for (int i = 0; i < n; i++)
        if (!visited[i]) {
            c++;
            printf("\nComponent %d: ", c);
            dfs(i, n, a);           /* prints all vertices of this component */
        }
    return c;
}
```

For the graph above the function prints three components: {A, B, D, C}, {E, F, G} and {H}. Time **O(V + E)**.

For directed graphs, **strongly connected components** are found with Kosaraju's algorithm (two DFS passes, one on the reversed graph) or Tarjan's algorithm, also in O(V + E).

Part 3: Spanning Trees and Minimum Cost Spanning Trees

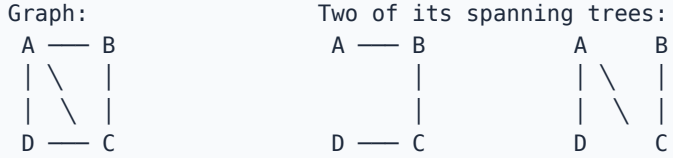
Que 5.7 **What is a spanning tree? What is a minimum cost spanning tree? State its properties and applications.**

ANSWER

A **spanning tree** of a connected undirected graph G = (V, E) is a subgraph T that

1. includes **all the vertices** of G,
2. is a **tree** (connected and without cycles), and therefore
3. has exactly **|V| − 1 edges**.

A graph can have many spanning trees. A complete graph K_n has $\mathbf{n^{n-2}}$ spanning trees (Cayley's formula); for example K_4 has 16.



A **minimum cost spanning tree (MST)** of a connected **weighted** graph is a spanning tree whose total edge weight is as small as possible.

Properties

- Adding any edge to a spanning tree creates exactly one cycle; removing any edge disconnects it.
- If all edge weights are distinct, the MST is unique.

- **Cut property:** the lightest edge crossing any cut (division of the vertices into two groups) belongs to some MST. Both Prim's and Kruskal's algorithms are greedy algorithms based on this property.

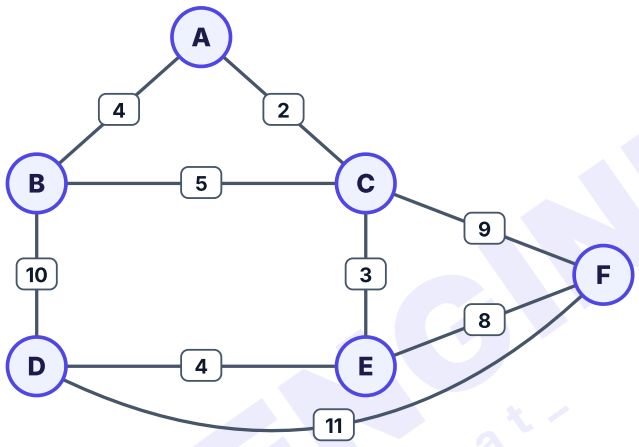
Applications: designing networks with least cable or pipe length (telephone, electricity, water, computer networks), clustering, approximation of the travelling salesman problem, image segmentation.

Que 5.8 **Explain Kruskal's algorithm for the minimum spanning tree. Apply it to the weighted graph given in the answer.**

ANSWER

The graph (vertices A to F) has these edges:

Edge	AB	AC	BC	BD	CE	DE	DF	EF	CF
Weight	4	2	5	10	3	4	11	8	9



Kruskal's algorithm builds the MST edge by edge, always choosing the **cheapest remaining edge that does not form a cycle**:

1. Sort all edges in increasing order of weight.
2. Start with a forest in which each vertex is its own tree.
3. For each edge (u, v) in sorted order: if u and v are in **different trees**, add the edge to the MST and merge the two trees; otherwise reject it (it would form a cycle).
4. Stop when the MST has V − 1 edges.

Whether u and v are in the same tree is checked with a **disjoint-set (union-find)** structure.

Solution

Sorted edges: AC 2, CE 3, AB 4, DE 4, BC 5, EF 8, CF 9, BD 10, DF 11.

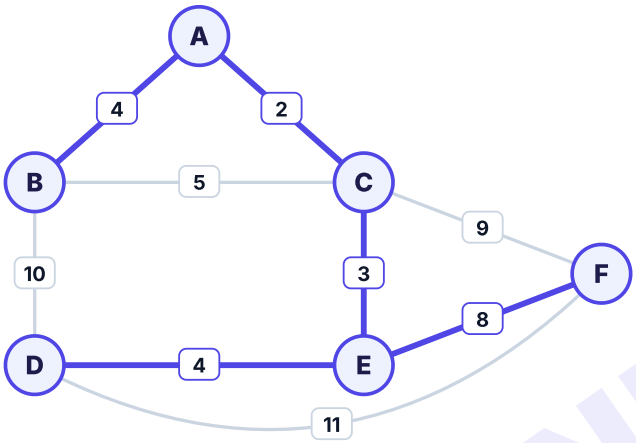
Step	Edge	Weight	Decision	Components after the step
1	AC	2	accept	{A, C} {B} {D} {E} {F}
2	CE	3	accept	{A, C, E} {B} {D} {F}

NEXT ENGINEER _sheerat_

Step	Edge	Weight	Decision	Components after the step
3	AB	4	accept	{A, B, C, E} {D} {F}
4	DE	4	accept	{A, B, C, D, E} {F}
5	BC	5	reject: B and C already connected (cycle A-B-C)	unchanged
6	EF	8	accept	{A, B, C, D, E, F}

Five edges (V − 1 = 5) have been accepted, so we stop.

MST edges: AC, CE, AB, DE, EF. **Total cost = 2 + 3 + 4 + 4 + 8 = 21.**



The highlighted edges form the MST.

```
int parent[MAXV];
int find(int x) { while (parent[x] != x) x = parent[x]; return x; }

struct edge { int u, v, w; };

int kruskal(struct edge e[], int m, int n) {      /* e[] sorted by weight */
    int cost = 0, taken = 0;
    for (int i = 0; i < n; i++) parent[i] = i;
    for (int i = 0; i < m && taken < n - 1; i++) {
        int ru = find(e[i].u), rv = find(e[i].v);
        if (ru != rv) {                           /* no cycle */
            parent[ru] = rv;
            cost += e[i].w;
            taken++;
            printf("%c-%c (%d)\n", 'A' + e[i].u, 'A' + e[i].v, e[i].w);
        }
    }
    return cost;
}
```

Complexity: sorting takes **O(E log E)**; the union-find operations are nearly O(1) each with path compression and union by rank. So Kruskal's algorithm runs in **O(E log E)**, which equals O(E log V). It is preferred for **sparse** graphs.

Que 5.9 **Explain Prim's algorithm for the minimum spanning tree and apply it to the graph of Que 5.8 starting from A.**

ANSWER

Prim's algorithm grows a single tree from a starting vertex. At every step it adds the **cheapest edge that connects a vertex in the tree to a vertex outside the tree**.

1. Put the start vertex in the tree.
2. Repeat V − 1 times: among all edges (u, v) with u in the tree and v outside, choose the one with the least weight; add v and the edge to the tree.

In practice we keep for every vertex v outside the tree a value `key[v]` = the weight of the cheapest edge joining v to the tree, and `parent[v]` = the tree vertex at the other end of that edge.

Solution (start at A)

Step	Tree vertices	Candidate edges (to vertices outside)	Chosen edge
1	A	AB 4, AC 2	**AC 2**
2	A, C	AB 4, BC 5, CE 3, CF 9	**CE 3**
3	A, C, E	AB 4, BC 5, CF 9, DE 4, EF 8	**AB 4** (tie with DE 4)
4	A, B, C, E	BD 10, CF 9, DE 4, EF 8	**DE 4**
5	A, B, C, D, E	CF 9, DF 11, EF 8	**EF 8**

MST edges: AC, CE, AB, DE, EF. **Total cost = 21**, the same tree as Kruskal's.

```
#define INF 99999

int prim(int g[][MAXV], int n) {          /* g[u][v] = weight, 0 = no edge */
    int key[MAXV], parent[MAXV], in[MAXV], cost = 0;
    for (int i = 0; i < n; i++) { key[i] = INF; in[i] = 0; parent[i] = -1; }
    key[0] = 0;                           /* start at vertex 0 (A) */
    for (int k = 0; k < n; k++) {
        int u = -1;
        for (int v = 0; v < n; v++)       /* pick the cheapest vertex outside the tree */
            if (!in[v] && (u == -1 || key[v] < key[u])) u = v;
        in[u] = 1;
        cost += key[u];
        if (parent[u] != -1) printf("%c-%c (%d)\n", 'A' + parent[u], 'A' + u, key[u]);
        for (int v = 0; v < n; v++)       /* update keys of neighbours */
            if (g[u][v] && !in[v] && g[u][v] < key[v]) { key[v] = g[u][v]; parent[v] = u; }
    }
    return cost;
}
```

Complexity: with an adjacency matrix and a simple scan, **$O(V^2)$**, which is good for **dense** graphs. With adjacency lists and a binary heap, **$O(E \log V)$**.

Prim vs Kruskal

Basis	Prim	Kruskal
Approach	Grows one tree from a start vertex	Adds cheapest edges, merging a forest of trees
Intermediate result	Always a single connected tree	May be a forest (disconnected)
Data structure	Priority queue / key array	Sorted edge list + union-find
Time	$O(V^2)$ or $O(E \log V)$	$O(E \log E)$
Better for	Dense graphs	Sparse graphs
Works on disconnected graph	Gives MST of one component only	Gives a minimum spanning forest

Part 4: Transitive Closure and Shortest Paths

Que 5.10 **What is the transitive closure of a graph? Explain Warshall's algorithm with an example.**

ANSWER

The **transitive closure** of a directed graph G with n vertices is an n × n Boolean matrix **T** (also called the **path matrix** or **reachability matrix**) where

T[i][j] = 1 if there is a path of length one or more from vertex i to vertex j, and 0 otherwise.

Warshall's algorithm computes T from the adjacency matrix A in $O(n^3)$. It builds a sequence of matrices $R^{(0)} = A$, $R^{(1)}$, ..., $R^{(n)} = T$, where $R^{(k)}[i][j] = 1$ if there is a path from i to j whose **intermediate vertices are all among the first k vertices**. The rule is:

$R^{(k)}[i][j] = R^{(k-1)}[i][j]$ OR ($R^{(k-1)}[i][k]$ AND $R^{(k-1)}[k][j]$)

that is, i reaches j using vertices up to k either if it already could without k, or if i reaches k and k reaches j.

```
void warshall(int r[][MAXV], int n) {     /* r starts as the adjacency matrix */
    for (int k = 0; k < n; k++)
        for (int i = 0; i < n; i++)
            for (int j = 0; j < n; j++)
                r[i][j] = r[i][j] || (r[i][k] && r[k][j]);
}
```

Shortcut for hand working: in pass k, for every row i that has a 1 in column k, OR row k into row i.

Example: a digraph with edges A → B, B → D, D → A, D → C.

$R^{(0)}$ (adjacency matrix)

	A	B	C	D
A	0	1	0	0
B	0	0	0	1

	A	B	C	D
C	0	0	0	0
D	1	0	1	0

k = A: only row D has a 1 in column A. Row D = D OR row A = (1, 1, 1, 0).

$R^{(1)}$	A	B	C	D
A	0	1	0	0
B	0	0	0	1
C	0	0	0	0
D	1	1	1	0

k = B: rows A and D have a 1 in column B. Row A = A OR row B = (0, 1, 0, 1). Row D = D OR row B = (1, 1, 1, 1).

$R^{(2)}$	A	B	C	D
A	0	1	0	1
B	0	0	0	1
C	0	0	0	0
D	1	1	1	1

k = C: only row D has a 1 in column C, but row C is all zeros, so nothing changes. $R^{(3)} = R^{(2)}$.

k = D: rows A, B and D have a 1 in column D. Row A = A OR row D = (1, 1, 1, 1). Row B = B OR row D = (1, 1, 1, 1).

$R^{(4)} = T$	A	B	C	D
A	1	1	1	1
B	1	1	1	1
C	0	0	0	0
D	1	1	1	1

Interpretation: A, B and D lie on the cycle A → B → D → A, so each reaches every vertex (including itself), and through D they also reach C. C has no outgoing edge, so it reaches nothing.

Complexity: three nested loops over n vertices: **$O(n^3)$** time and $O(n^2)$ space.

Floyd–Warshall (extension): replacing OR/AND by min/+ on a weighted matrix gives the all-pairs shortest path algorithm: D[i][j] = min(D[i][j], D[i][k] + D[k][j]), also $O(n^3)$.

Que 5.11 **Explain Dijkstra's single-source shortest path algorithm. Apply it to the weighted graph of Que 5.8 with source A.**

ANSWER

The **single-source shortest path problem** is: given a weighted graph and a source vertex s, find the minimum-cost path from s to every other vertex. **Dijkstra's algorithm** (Edsger Dijkstra, 1959) solves it when **all edge weights are non-negative**. It is a greedy algorithm.

Idea: keep a tentative distance d[v] for every vertex (0 for s, ∞ for others) and a set S of vertices whose shortest distance is final. Repeatedly:

1. Pick the vertex u not in S with the **smallest d[u]**, and add it to S (its distance is now final).
2. **Relax** every edge (u, v): if d[u] + w(u, v) < d[v], set d[v] = d[u] + w(u, v) and pred[v] = u.

Stop when all vertices are in S.

```
void dijkstra(int g[][MAXV], int n, int s) {     /* g[u][v] = weight, 0 = no edge */
    int d[MAXV], done[MAXV], pred[MAXV];
    for (int i = 0; i < n; i++) { d[i] = INF; done[i] = 0; pred[i] = -1; }
    d[s] = 0;
    for (int k = 0; k < n; k++) {
        int u = -1;
        for (int v = 0; v < n; v++)
            if (!done[v] && (u == -1 || d[v] < d[u])) u = v;
        done[u] = 1;
        for (int v = 0; v < n; v++)
            if (g[u][v] && !done[v] && d[u] + g[u][v] < d[v]) {
                d[v] = d[u] + g[u][v];
                pred[v] = u;
            }
    }
    for (int v = 0; v < n; v++) printf("%c: %d\n", 'A' + v, d[v]);
}
```

Solution (source A)

Edges: AB 4, AC 2, BC 5, BD 10, CE 3, DE 4, DF 11, EF 8, CF 9 (undirected).

Step	Vertex added to S	d[A]	d[B]	d[C]	d[D]	d[E]	d[F]
Init		0	∞	∞	∞	∞	∞
1	A (0)	**0**	4	2	∞	∞	∞
2	C (2)		4	**2**	∞	5	11
3	B (4)		**4**		14	5	11
4	E (5)				9	**5**	11
5	D (9)				**9**		11
6	F (11)						**11**

Working:

- Step 2 (u = C): B: 2 + 5 = 7 is not less than 4, so no change; E: 2 + 3 = 5; F: 2 + 9 = 11.
- Step 3 (u = B): D: 4 + 10 = 14.
- Step 4 (u = E): D: 5 + 4 = 9 < 14, update; F: 5 + 8 = 13 is not less than 11.
- Step 5 (u = D): F: 9 + 11 = 20 is not less than 11.

Result

Vertex	Shortest distance from A	Path
B	4	A → B
C	2	A → C
D	9	A → C → E → D
E	5	A → C → E
F	11	A → C → F

The paths are read backwards from the pred[] array, for example pred[D] = E, pred[E] = C, pred[C] = A.

Complexity: with an adjacency matrix and linear search for the minimum, **$O(V^2)$**. With adjacency lists and a binary heap (priority queue), **$O((V + E) \log V)$**.

Why negative weights are not allowed: once a vertex is added to S its distance is never revised. A negative edge found later could give a shorter path to an already-final vertex, so the answer may be wrong. The **Bellman–Ford** algorithm handles negative weights (but not negative cycles) in O(V·E).

Applications: route finding in maps and GPS, network routing protocols such as OSPF, robot path planning, flight connection planning.

NEXT ENGINEER _sheerat_

Short-Answer Questions (2 marks)

Q1. Define a graph. How is it different from a tree?

A graph G = (V, E) is a set of vertices with edges joining pairs of them. A tree is a special graph that is connected and has no cycles; a graph may have cycles and may be disconnected.

Q2. What is the maximum number of edges in a simple undirected graph with n vertices?

$n(n-1)/2$.

Q3. What is the sum of degrees of a graph with 12 edges?

$2 \times 12 = 24$ (handshaking lemma).

Q4. Which data structures are used in BFS and DFS?

BFS uses a queue; DFS uses a stack (or recursion).

Q5. What is the time complexity of BFS with an adjacency list?

$O(V + E)$.

Q6. What is a spanning tree? How many edges does it have?

A subgraph that is a tree and includes all vertices of the graph. It has $V - 1$ edges.

Q7. How many spanning trees does K_4 have?

$4^{4-2} = 16$ (Cayley's formula).

Q8. Which MST algorithm is better for sparse graphs and why?

Kruskal's, because its running time $O(E \log E)$ depends mainly on the number of edges.

Q9. What is transitive closure?

A matrix T where T[i][j] = 1 if there is a path from vertex i to vertex j; computed by Warshall's algorithm in $O(n^3)$.

Q10. Why does Dijkstra's algorithm fail with negative edge weights?

Because a vertex's distance is fixed once it is selected, and a negative edge found later could make a shorter path to it.

Q11. Differentiate between a strongly and weakly connected graph.

In a strongly connected digraph, every vertex can reach every other along directed paths. A weakly connected digraph is connected only when edge directions are ignored.

Q12. Write two applications of graphs.

Road networks and route finding; computer and social networks.

NEXT ENGINEER _sheerat_

Practice Questions

1. Represent a directed graph with 5 vertices and 7 edges of your choice using an adjacency matrix, adjacency list and incidence matrix.
2. Write a C program for BFS using adjacency lists and trace it on a graph of 7 vertices.
3. Write a non-recursive DFS using an explicit stack.
4. Find the MST of a graph with edges (1,2) 28, (1,6) 10, (2,3) 16, (2,7) 14, (3,4) 12, (4,5) 22, (4,7) 18, (5,6) 25, (5,7) 24 using both Prim's and Kruskal's algorithms.
5. Apply Dijkstra's algorithm on a directed graph of 6 vertices of your choice and show the distance table.
6. Find the transitive closure of the digraph with edges 1→2, 2→3, 3→4, 4→2 using Warshall's algorithm.
7. Prove that a tree with n vertices has n − 1 edges.
8. Explain how DFS can be used to detect a cycle in an undirected graph.
9. Compare Prim's and Kruskal's algorithms in a table.
10. Write a short note on the adjacency multilist representation.

NEXT ENGINEER
sheerat

Model Question Paper with Answer Guide

This paper follows the university pattern (70 marks, 3 hours). Next to each question is the place in this book where the full answer is given. Attempt it in 3 hours with the book closed, then check your answers.

Section A: attempt all questions (7 × 2 = 14 marks)

No.	Question	Answer in
a	Define time-space trade-off with an example.	Que 1.8
b	Convert A + B * C ^ D − E to postfix.	Que 2.6 method
c	What is the drawback of a linear queue?	Unit 2, Q7
d	Define load factor and primary clustering.	Unit 3, Q5, Q6
e	Prove that $n_0 = n_2 + 1$ in a binary tree.	Que 4.2
f	What is a threaded binary tree?	Que 4.9
g	Write the handshaking lemma.	Que 5.1

Section B: attempt any three (3 × 7 = 21 marks)

No.	Question	Answer in
a	Derive the row-major and column-major address formulae for a 3-D array and find the address of A[5][−1][8] in A[2:8, −4:1, 6:10] with base 200 and W = 4.	Que 1.11, 1.12
b	Write the algorithm to convert infix to postfix and convert (A + B) * C − D / E ^ F + G.	Que 2.6
c	Sort 42, 17, 63, 8, 29, 55, 11, 36 using quick sort and give its best and worst case complexity.	Que 3.13
d	Construct an AVL tree for 40, 20, 10, 25, 30, 22, 50.	Que 4.12
e	Find the MST of a weighted graph using Kruskal's algorithm.	Que 5.8

Section C: attempt one part of each question (5 × 7 = 35 marks)

No.	Question	Answer in
3 (a)	Write C functions to insert and delete a node in a doubly linked list.	Que 1.17
3 (b)	Add two polynomials using linked lists with an algorithm and example.	Que 1.19
4 (a)	Explain the Tower of Hanoi problem and derive the number of moves.	Que 2.13
4 (b)	Write a C program for a circular queue and trace it.	Que 2.16
5 (a)	Insert 72, 27, 36, 24, 63, 81, 92, 101 in a table of size 11 using linear and quadratic probing.	Que 3.7
5 (b)	Explain heap sort with an example.	Que 3.15

No.	Question	Answer in
6 (a)	Construct a binary tree from its inorder and preorder traversals.	Que 4.6
6 (b)	Construct a B-tree of order 5 and explain deletion.	Que 4.14, 4.15
7 (a)	Apply Dijkstra's algorithm to find shortest paths from a source.	Que 5.11
7 (b)	Find the transitive closure using Warshall's algorithm.	Que 5.10

NEXT ENGINEER
sheerat

Last-Day Revision Checklist

- Asymptotic notation definitions with c and n_0, and the growth order $1 < \log n < n < n \log n < n^2 < 2^n$.
- Row-major and column-major formulae for 2-D and 3-D arrays (write M, N, P and effective indices first).
- Triplet form of a sparse matrix.
- Linked list insertion and deletion code, including the order of pointer updates.
- Polynomial addition trace table.
- Infix to postfix table, postfix and prefix evaluation.
- Tower of Hanoi: recurrence $T(n) = 2T(n - 1) + 1$ and answer $2^n - 1$.
- Circular queue full and empty conditions.
- Linear, quadratic and double hashing probe formulae.
- Quick, merge and heap sort traces; the full complexity comparison table.
- BST deletion's three cases; AVL's four rotations; B-tree split and merge.
- Huffman tree construction and average code length.
- BFS and DFS orders; Prim's and Kruskal's tables; Dijkstra's distance table; Warshall's matrices.

NEXT ENGINEER
sheerat